Looped Language Models: A Survey and Meta-Analysis of Recurrent-Depth Transformers

Seyedarmin Azizi* Erfan Baghaei Potraghloo Massoud Pedram
University of Southern California (USC), USA

Abstract

Looped language models apply one shared block of Transformer layers repeatedly to the same tokens. Their effective depth, the number of layer applications per token, can grow without adding parameters and can be chosen at inference time. Recurrent depth is thus a way to scale test-time compute in latent space instead of in generated tokens. The idea appears under many names, including looped Transformers, LoopLMs, recurrent-depth models, Universal Transformers, Mixture-of-Recursions, and recursive reasoning models such as HRM and TRM. In two years it has moved from synthetic tasks to open models such as Huginn and Ouro and to pretraining on up to 28 trillion tokens. According to press reports, it is also used in a frontier-lab model. Its literature has split across architecture, training, systems, theory and safety. We present the first survey dedicated to looped language models, built on a systematic corpus of 560 core papers screened from 7,374 candidates. We introduce a unified prelude–core–coda formalism that makes explicit how parameters, compute, memory and depth are counted. We organize the field from design, training and scaling to inference systems, theory, mechanisms, capabilities, other modalities and safety. A meta-analysis classifies 822 verified empirical claims by comparison regime. It shows that most apparent contradictions about whether "a recurrence is worth a layer" come from comparing parameter-matched with compute-matched results. The evidence fits one reading, in which a recurrence is worth less than a fresh block of parameters. Looping therefore helps when parameters are the limiting resource and hurts when it replaces them at fixed compute. We derive a research agenda from measured gaps, including the absence of work on robustness, alignment and privacy. We release the corpus, the extraction records with verbatim evidence for every number, and the claims ledger.

Living repository: https://github.com/ArminAzizi98/looped-lm-survey • Machine-readable corpus, extraction records and claims ledger released with the paper.

*Corresponding author: seyedarm@usc.edu

Key findings

- In dense models at matched training compute and effective depth, a recurrence is worth about $r^{0.46}$ fresh blocks, so looping loses [433]. Looping helps when parameters are the limiting resource: with unique parameters fixed, with limited data, or with sparse experts that keep the total parameter count (Sections 7.1 and 15.3).
- Most apparent contradictions in the field are differences in what each comparison holds fixed. Only 19% of the empirical claims we verified are matched on compute, and 31% are shown at a billion parameters or more (Section 15).
- At language-model scale, running more loops than were used in training does not help. Gains saturate at or before the training depth (Section 7.2).
- In small-scale controlled studies, looping does not add knowledge storage. A reasoning advantage at matched compute has been shown only for sparse models (Section 7.3).
- Most reported savings from adaptive depth are proxies, such as average loop counts. For looped language models, measured speedups so far require batching or engine support, and none has been compared with fixed-loop models retrained at the same compute (Sections 5.5 and 8).
- Puzzle results of small recursive models depend heavily on the evaluation protocol, and independent reruns vary widely (Section 11.2).
- One study has measured chain-of-thought monitorability in looped models, and it finds no systematic penalty but does not test training that rewards hiding computation. No study has measured their robustness, alignment or privacy, and knowledge editing has been studied only at toy scale (Section 14).

Table 20 grades the evidence behind these and other recurring claims.

Contents

1 Introduction **5**

2 Scope, Terminology, and Methodology **9**
2.1 What counts as a looped model . . . 9
2.2 One idea, many names . . . 9
2.3 Search and screening protocol . . . 9

3 A Unified View: Formalism and Compute Accounting **11**
3.1 The prelude–core–coda template . . . 11
3.2 Topologies as choices of (k_P, k_R, k_C) and of what is shared . . . 13
3.3 Sharing maps . . . 13
3.4 Accounting identities . . . 13
3.5 Comparison regimes . . . 13
3.6 Three axes of serial computation . . . 14

4 Lineage: From Adaptive Computation to Looped Language Models **15**
4.1 Adaptive computation and shared layers (2016–2021) . . . 15
4.2 Recurrence as algorithmic generalization (2021–2024) . . . 16
4.3 Reasoning in latent depth (2025) . . . 17
4.4 Industrialization and scrutiny (2026) . . . 17

5 The Design Space of Looped Models **18**
5.1 What is looped . . . 18
5.2 How state is carried . . . 21
5.3 How much is shared . . . 22
5.4 Hybrids . . . 23
5.5 How many loops: loop-count policies . . . 23

6 Training Looped Models **27**
6.1 Pretraining from scratch . . . 28
6.2 Keeping the recurrence stable . . . 29
6.3 Gradients through depth . . . 30
6.4 Building looped models from existing checkpoints . . . 31
6.5 Post-training . . . 32

7 Scaling **33**
7.1 Scaling laws and what they hold fixed . . . 33
7.2 Test-time depth . . . 34
7.3 Knowledge versus reasoning . . . 35
7.4 How many loops? . . . 36

8 Inference and Systems **37**
8.1 The KV cache . . . 37
8.2 The latency of sequential loops . . . 38
8.3 Serving adaptive depth . . . 39
8.4 Quantization and numerics . . . 39
8.5 Decoding with early iterations . . . 40

9 Theory **41**
9.1 Expressivity: loop count as circuit depth . . . 41
9.2 Looped models versus chain of thought . . . 42
9.3 Looped models as iterative algorithms . . . 43
9.4 Length generalization and algorithmic extrapolation . . . 43
9.5 The theory–practice gap . . . 44

10 Mechanisms and Interpretability **45**
10.1 Limiting dynamics: fixed points, cycles and attractors . . . 45
10.2 What each iteration does . . . 46
10.3 Reading intermediate iterations . . . 47

11 Capabilities and Empirical Evidence **48**
11.1 Looped language models . . . 48
11.2 Recursive reasoning models . . . 49

12 Beyond Language **52**
12.1 Diffusion and generative models . . . 52
12.2 Vision and multimodal models . . . 54
12.3 Speech . . . 55
12.4 Reinforcement learning, robotics and control . . . 55
12.5 Recommendation . . . 55

12.6 Science and other domains . . . 56

13 Relation to Neighboring Paradigms **56**
13.1 Sequence-axis latent reasoning . . . 56
13.2 Diffusion language models . . . 57
13.3 Adaptive depth without sharing . . . 58
13.4 Deep equilibrium and implicit models . . . 58
13.5 How standard Transformers use depth . . . 58
13.6 Prior surveys and the scaling-axis claim . . . 58

14 Safety, Oversight, and Trustworthiness **59**
14.1 Chain-of-thought monitorability . . . 60
14.2 Reading latent computation directly . . . 62
14.3 Capability elicitation . . . 62
14.4 What has not been studied . . . 63

15 Meta-Analysis: What the Evidence Supports **63**
15.1 One pair of models, two verdicts . . . 64
15.2 The shape of the evidence . . . 64
15.3 Is a recurrence worth a layer? . . . 65
15.4 Graded claims . . . 65
15.5 Limitations of the ledger . . . 66
15.6 Recommendations . . . 66

16 Open Problems and Research Agenda **68**
16.1 Measurement and accounting . . . 68
16.2 Reliability of depth . . . 68
16.3 Safety and governance . . . 69
16.4 Capabilities . . . 70
16.5 Systems and theory . . . 70

17 Resources **71**

18 Conclusion **72**

A Corpus Growth Data **122**

B Reporting Checklist for Looped-Model Papers **122**

C Claims Ledger Excerpt **123**

D Core Papers by Topic **128**

1 Introduction

For most of the history of large language models (LLMs), there were two ways to increase capability. One could train a larger model on more data. More recently, one could also let a fixed model generate more tokens before it answers. A third way is to *let the model apply its own layers again*, which spends more test-time compute in latent space instead of in generated tokens. A **looped language model** reuses one block of parameters for several consecutive steps of computation on the same tokens. The number of layer applications per token is the model's *effective depth*. In a looped model, effective depth can be chosen independently of the number of parameters. Often it can also be chosen independently at training time and at inference time.

The idea is old, and several earlier methods used it (Section 4 traces this history). Adaptive Computation Time let a recurrent network "learn how many computational steps to take between receiving an input and emitting an output" [187]. The Universal Transformer applied one Transformer layer recurrently over depth with "a dynamic per-position halting mechanism" [120]. ALBERT made BERT smaller by sharing "all parameters across layers" [285]. Recurrent networks trained on easy instances were also shown to solve harder ones "simply by performing additional recurrences during inference" [431]. Recent work differs from this earlier work in its scale and ambition. Huginn, a recurrent-depth model with 3.5 billion parameters trained on 800 billion tokens, improved on reasoning benchmarks "up to a computation load equivalent to 50 billion parameters" [175]. The Ouro family scaled looped pretraining to 7.7 trillion tokens and reports that its 1.4B and 2.6B models "match 4B and 8B standard transformers on most benchmarks" [604]. This comparison is against public checkpoints trained on different data (Section 11.1). A 3B looped model has since been pretrained "from scratch on 28T tokens" [368]. Looped mixture-of-experts models with up to 20B parameters have been released [169]. At the other end of the scale, the Tiny Recursive Model (TRM) has 7M parameters and reports 45% on ARC-AGI-1 [245], although such puzzle scores depend strongly on the evaluation protocol (Section 11.2). The architecture may also have reached frontier labs. In September 2026, press coverage reported that a new OpenAI model uses "recurrent depth" [483]. In the same month, safety researchers warned that latent recurrence could weaken chain-of-thought monitoring [155].

The literature has grown very quickly, and it is now hard for anyone to read all of it. Our systematic search (Section 2) identified 560 papers on looped models. Of these, 60% were posted in the twelve months before our October 2026 snapshot (Figure 2). The work appears under at least nine names (Table 1). It covers language, vision, speech, reinforcement learning and puzzle solving. It also comes from communities that rarely cite each other. Complexity theorists study what loops can express, systems researchers work to reduce the memory cost of looping, and reasoning researchers train recursive puzzle solvers. Existing surveys treat looped models as one subsection of *latent reasoning* (Table 17). We matched the titles of our core papers against the full texts of the three most recent surveys. By this count, which is a lower bound, these surveys cite 12, 14 and 6 of the 560 core papers we identify [83, 305, 603].

The central empirical question of the field is whether *a recurrence is worth as much as a fresh layer*. Fragmentation would matter less if the field agreed on the answer, but at least on the surface it does not. An iso-depth scaling study finds that "replacing unique blocks with shared recurrences increases validation loss at matched training compute" [433]. A training-recipe study finds that a 1.4B looped model "outperforms a parameter-matched dense model trained on the same data and token budget on all 12 evaluated benchmarks" [343]. A looped mixture-of-experts series reports that its model "substantially outperforms vanilla Transformer baselines trained with the same compute budget" [169]. These three statements do not conflict, because each of them holds a different quantity fixed. The first uses dense blocks and compares at matched *training compute*. The second

compares at matched *parameters and data*. The third compares at matched training time, which is measured as step time and not as floating-point operations (FLOPs), and it places sparse experts inside the loop. For this sparse setting, another study finds that looped models "scale better than the standard baseline while dense looped models do not" [288]. If the comparison regime is stated explicitly, an apparent contradiction of this kind shows when looping helps and when it does not. Most apparent disagreements in the field are of this kind, and resolving them is the main analytical goal of this survey (Figure 1). Our answer, developed in Section 15, is that a recurrence is worth less than a fresh block of parameters. Looping therefore hurts when it replaces unique parameters at fixed compute. It helps when parameters are the limiting resource, for example under a memory budget, with limited data, or with sparse experts.

Contributions. To our knowledge, this is the first survey dedicated to looped language models, and it makes the following contributions.

- **A unified formalism** (Section 3). We show that every method we review is an instance of one template. The template consists of a prelude, a shared core with an injection operator, a coda and a loop-count policy. A sharing map sets how strictly the parameters of the core are tied. We derive the accounting identities for parameters, FLOPs, memory and effective depth. We also show that parameter-matched and compute-matched comparisons are mutually exclusive at fixed width.
- **A full-stack taxonomy** (Figure 3). The taxonomy covers architecture (Section 5), training (Section 6), scaling (Section 7), inference systems (Section 8), theory (Section 9), mechanisms (Section 10), capabilities (Section 11), other modalities (Section 12) and safety (Section 14).
- **A meta-analysis** (Section 15). We classify 822 verified empirical claims of the corpus by comparison regime and scale. We sort the controlled evidence on the value of a recurrence by what each comparison holds fixed (Table 19). We also grade the evidence behind the claims that recur across the field (Table 20), and we identify which disagreements are real.
- **A safety evidence map** (Table 18). We collect what is known about the monitorability, robustness and alignment of looped models, and we show which of these properties no study has examined.
- **A measured research agenda** (Section 16). The open problems in this agenda are backed by corpus counts (Table 21), and they include topics with no published work at the time of the snapshot.
- **Verified evidence and open resources** (Section 17). Every quotation and number in this survey is checked automatically against the text of the cited paper, and every sentence that cites a paper was checked against that paper (Section 2.3). We provide a table of released checkpoints and code, and we release the machine-readable corpus, screening decisions, extraction records and claims ledger.

How to read this survey. Readers new to the area can read Sections 3 to 5 for the main ideas and Section 15 for the state of the evidence. Practitioners who build or serve looped models will find recipes in Sections 6 and 8. Theorists can start at Section 9, and safety researchers can start at Section 14. Readers who want to know what looped models achieve can read Sections 7, 11 and 12. Section 10 covers what each iteration computes, Section 13 compares looped models with neighboring paradigms, and Section 16 collects the open problems.

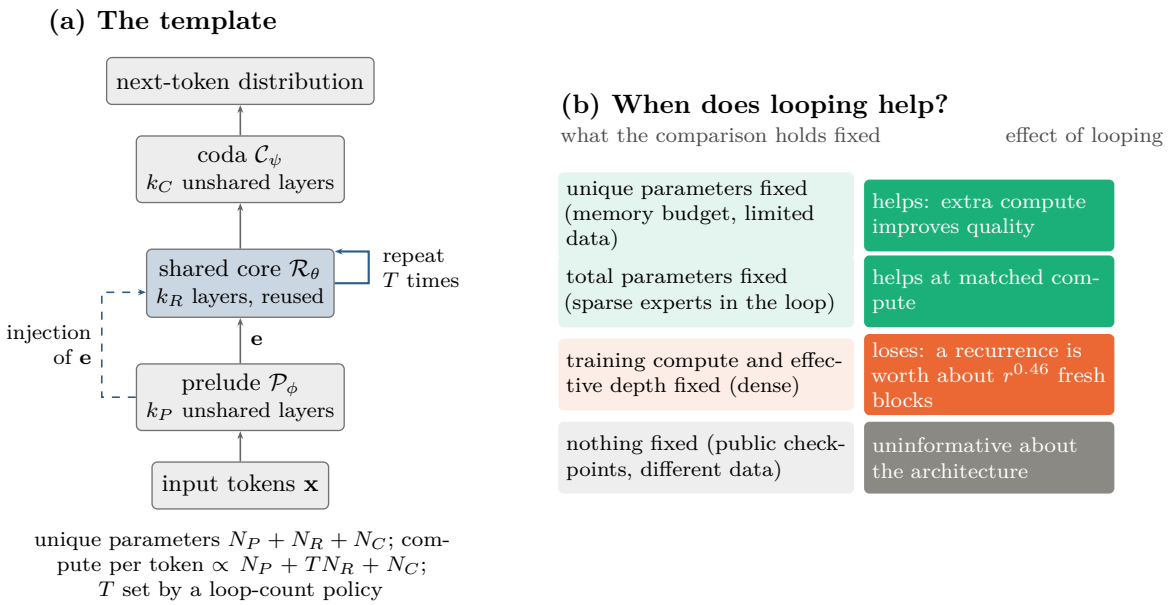


Figure 1: Overview. (a) Every method in this survey is an instance of one template: an unshared prelude, a shared core applied T times with the input encoding **e** injected, and an unshared coda (Section 3). Looping multiplies the compute spent in the core without adding parameters. (b) The controlled evidence on whether a recurrence is worth a layer, sorted by what each comparison holds fixed (Table 19). The value $r^{0.46}$ comes from an iso-depth scaling study [433]. A recurrence adds compute that is worth less than a fresh block of parameters, so looping helps when parameters are the limiting resource and loses when it replaces them at fixed compute.

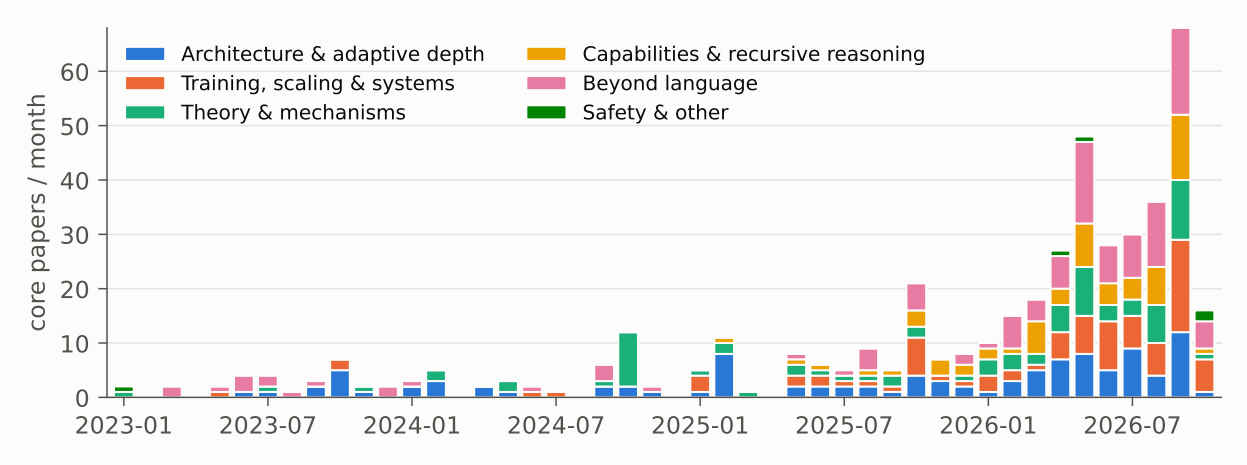


Figure 2: Number of core looped-model papers per month in our corpus, from January 2023 to the October 2026 snapshot (October is a partial month), stacked by area. The data are in Section A.

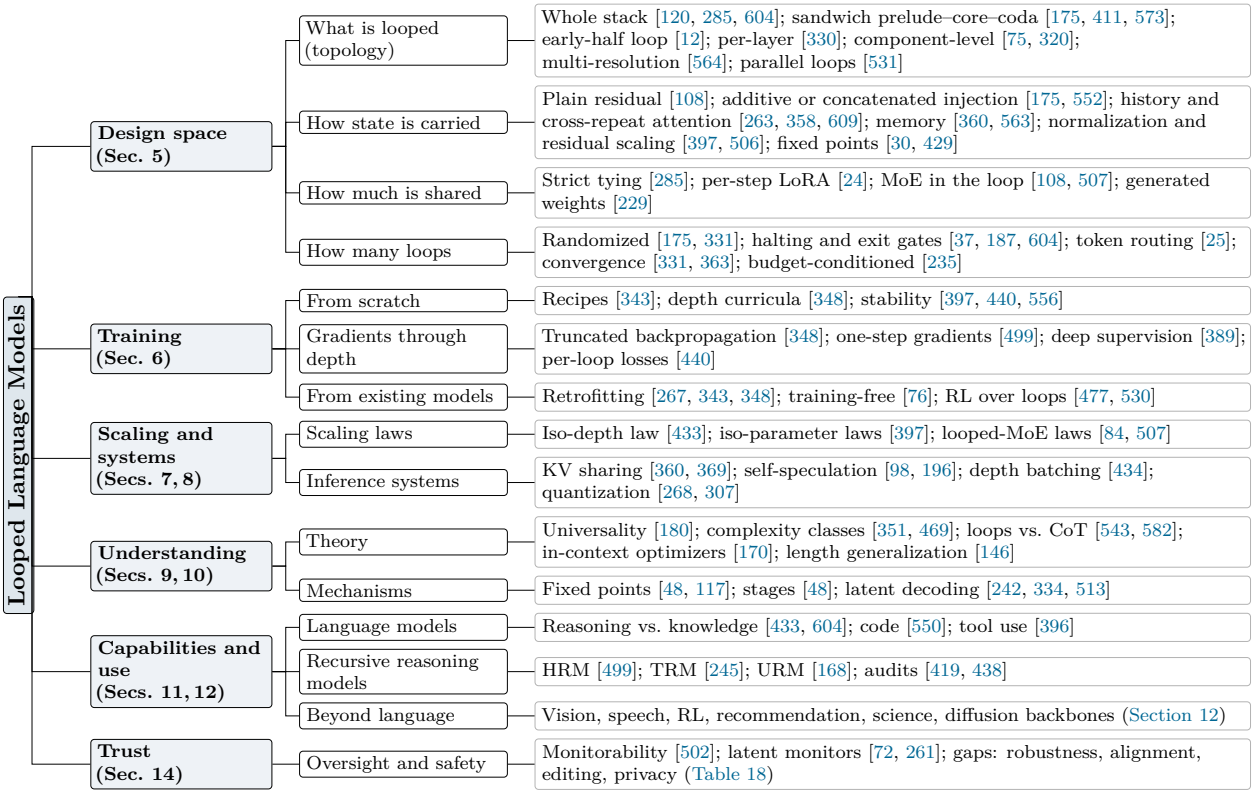


Figure 3: Taxonomy of looped language models used to organize this survey.

2 Scope, Terminology, and Methodology

This section fixes what we count as a looped model (Section 2.1), reconciles the names used in the literature (Section 2.2) and describes how we built the corpus on which the survey rests (Section 2.3).

2.1 What counts as a looped model

We use the term *looped model* for any network in which a parameter-shared block is applied repeatedly to the same positions along the *depth* (iteration) axis of a single forward computation. The number of applications may be chosen at inference time, but this is not required. The shared block can be the whole network, a sub-stack (the "core" of Section 3), a single layer, or a single sub-module such as the token mixer. We include partial sharing, such as low-rank or adapter deviations per iteration and mixture-of-experts layers whose routing changes across iterations. We also include nested recursion at several timescales, training-free re-execution of the layers of a pretrained model, and retrofitting recurrence into a pretrained model.

Three neighboring families are *not* looped in this sense, and we discuss them only in comparison (Section 13). The first is recurrence along the *sequence* axis, where a hidden state is fed back as the next input (continuous or latent chain of thought). The second is refinement over *denoising* steps (diffusion language models). The third is adaptive depth *without* sharing (early exit, layer skipping, Mixture-of-Depths).

2.2 One idea, many names

Table 1 lists the names under which the same construction appears. The large number of names causes practical problems. A search for one name misses most of the literature filed under the other names, and at least one name now collides with an unrelated line of work.

A name collision. The name "Recursive Language Models" refers to an *inference paradigm* in which a model treats a long prompt as an external environment and recursively calls itself over pieces of it [577]. The same phrase has since been used for depth-recurrent language models [117]. The two are unrelated, because the former recurses over *context* and the latter recurses over *depth*. For the latter, we recommend the terms *looped language model* or *recurrent-depth language model*, and we use them throughout this survey.

Terms used in this survey. *Loop*, *iteration*, *recurrence* and *recursion* all refer to one application of the shared core. *Effective depth* is the number of layer applications per token (Eq. (5)). The *training horizon* is the largest number of iterations seen during training. *Extrapolation* means running more iterations than that at test time. We use *training depth* for the loop count used in training, or for its mean when the loop count is sampled.

2.3 Search and screening protocol

We built the corpus systematically and release it with the paper. We did this to make the survey reproducible and to support the quantitative analyses of Sections 15 and 16.

Identification. (i) We queried the arXiv API for 56 phrases, with one phrase per query and with the abstract and title fields searched separately. The phrases cover every name in Table 1, plus adaptive-computation, fixed-point and latent-reasoning terms. (ii) We ran forward and backward

Table 1: Names for depth recurrence in the literature, with a representative usage. "Representative" is not a claim of priority.

Term	Representative usage	Source
Universal Transformer	a Transformer whose layer is applied recurrently over depth, with optional adaptive halting	[120]
Cross-layer parameter sharing	sharing one layer's parameters with all layers, or relaxed forms of this	[285, 470]
Looped Transformer	weights programmed or trained and run by "placing them in a loop" to execute iterative algorithms	[180, 552]
Recursive Transformer	an LLM converted to share parameters across layers ("layer tying"), optionally relaxed with low-rank adapters (LoRA)	[24]
Mixture-of-Recursions	a Recursive Transformer with routers that assign a recursion depth to each token	[25]
Recurrent depth / depth-recurrent	"iterating a recurrent block, thereby unrolling to arbitrary depth at test-time"	[175]
Looped Language Model (LoopLM)	pretrained language models that build iterative latent computation into pretraining	[604]
Pondering	"repeatedly invoking the forward process within a single token generation step"	[575]
Recursive reasoning models	small networks that recurse at one or several frequencies on puzzle-like tasks: the Hierarchical Reasoning Model (HRM), the Tiny Recursive Model (TRM) and the Universal Reasoning Model (URM)	[168, 245, 499]

citation snowballing from 32 verified seed papers through the Semantic Scholar Graph API. (iii) We added every entry of a community-maintained paper list on looped Transformers.[1] (iv) We also added the arXiv identifiers cited in the full texts of included papers. (v) Before release, we repeated the arXiv queries for papers submitted between 2026-09-28 and 2026-10-06 and screened the new records on their full texts.

Screening. We first removed duplicates by arXiv identifier and normalized title. We then filtered the candidates for name collisions, such as "human-in-the-loop", "closed-loop" and loop integrals in physics. The candidates were screened on title and abstract against the criteria of Section 2.1, and every decision was recorded together with a one-line reason. Papers were labeled *core* (looped in the sense above), *adjacent* (used in comparison sections) or *out*.

Extraction and verification. For every core paper, we extracted the design choices of Section 3 and Section 5, the comparison regime of each main claim, and every reported number used in this survey. Each extracted item comes with a verbatim quotation from the paper. An automated check confirms that every quotation is a substring of the paper's text. Numbers that fail this check are not reported. Claims whose regime was unclear after extraction were re-read against the full text in a separate pass. This pass assigned a regime to the claim, or marked it as a within-model or non-comparative observation, in both cases with a verified supporting quotation. 21 claims remain unclear and are excluded from regime statistics. We also checked every sentence of this survey that cites a paper against the text of that paper. Bibliographic entries are generated from arXiv metadata.

[1] https://github.com/ngocdaobao/A-Survey-on-Looped-Transformers, accessed 2026-10-05.

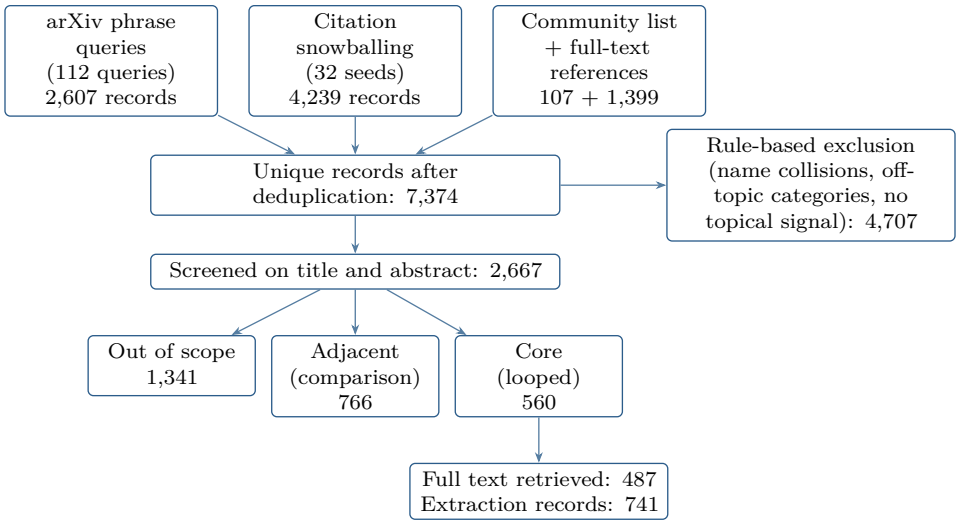


Figure 4: Corpus construction (snapshot 2026-10-05). Every count is generated from the released data files.

Counts. Figure 4 summarizes the pipeline. Of 7,374 unique records, 2,667 were screened on title and abstract. 560 records were judged core and 766 were judged adjacent.

Limitations of the protocol. The corpus is centered on arXiv, so work published only in venues without an arXiv version may be missing. Work posted after the snapshot date may also be missing. Coverage of the citation graph is incomplete for papers from the last weeks before the snapshot. Screening on title and abstract can misclassify papers whose abstracts do not describe the architecture. We reduced this risk by screening every candidate connected to a seed paper, regardless of keywords.

With the scope and the corpus fixed, Section 3 introduces the notation that we use to describe and compare the models in the corpus.

3 A Unified View: Formalism and Compute Accounting

Papers on looped models use at least nine names for overlapping ideas (Section 2) and many incompatible notations. This section defines one notation that covers every method in the survey. Its more important goal is to state explicitly the accounting identities that determine what a "fair" comparison between a looped and a non-looped model can mean. Most apparent disagreements in the field (Section 15) disappear once these identities are written down.

3.1 The prelude–core–coda template

Let $\mathbf{x} = (x_1, \ldots, x_n)$ be an input sequence and let Emb map tokens to $\mathbb{R}^{n \times d}$. A looped model is defined by five components.

1. A **prelude** $\mathcal{P}_\phi$: $k_P \geq 0$ unshared layers that produce an *input encoding* $\mathbf{e} = \mathcal{P}_\phi(\mathrm{Emb}(\mathbf{x}))$.
2. A **core** (the recurrent block) $\mathcal{R}_\theta$: $k_R \geq 1$ layers whose parameters θ are reused at every iteration.
3. An **injection operator** $\mathcal{I}$ that combines the running state with the input encoding (and optionally with earlier states).

4. A **coda** $\mathcal{C}_\psi$: $k_C \geq 0$ unshared layers followed by the unembedding.
5. A **loop-count policy** π that sets the number of iterations T globally, per sequence or per token.

Let $\mathbf{s}^{(t)} \in \mathbb{R}^{n \times d}$ be the state after t iterations. The forward computation is

$$\mathbf{s}^{(0)} = \text{init}(\mathbf{e}), \tag{1}$$

$$\mathbf{s}^{(t)} = \mathcal{R}_{\theta \oplus \delta_t}(\mathcal{I}(\mathbf{s}^{(t-1)}, \mathbf{e}, \mathbf{s}^{(<t-1)}),\ c_t), \qquad t = 1, \ldots, T, \tag{2}$$

$$p(\cdot \mid \mathbf{x}) = \text{softmax}(W_U\, \mathcal{C}_\psi(\mathbf{s}^{(T)})), \qquad T \sim \pi(\cdot \mid \mathbf{s}^{(1)}, \mathbf{s}^{(2)}, \ldots). \tag{3}$$

Here c_t is optional step conditioning, such as a timestep embedding, a routing mask or a budget token. The term $\theta \oplus \delta_t$ denotes optional step-specific deviations from strict sharing (Section 3.3). Studies disagree about whether step conditioning is useful. Huginn found that a core that takes the step index as input "interacts badly with path independence, leading to models that cannot extrapolate" [175]. A later controlled study instead proposes timestep conditioning combined with history-state injection as a fix that improves robustness [609] (Section 5.2). Setting $k_P = k_C = 0$, $\mathcal{I}(\mathbf{s}, \mathbf{e}, \cdot) = \mathbf{s}$, $\delta_t = 0$ and a fixed T gives the simplest *whole-stack* looped Transformer, in which the entire network is applied T times. Setting $T = 1$ gives a standard Transformer with $k_P + k_R + k_C$ layers.

Two instances. Huginn is a sandwich model with $(k_P, k_R, k_C) = (2, 4, 2)$, concatenation injection and a random initial state. It is trained with a mean recurrence of $\bar{r} = 32$, so its effective depth at the mean is $2 + 4 \cdot 32 + 2 = 132$ layers [175]. Ouro is a whole-stack model. It applies one stack of layers for up to four recurrent steps, and a learned exit gate, trained with an entropy-regularized objective, sets T for each token [604]. Table 5 describes further models in the same terms.

Initial state. In most whole-stack designs, init is the identity. Recurrent-depth models in the deep-thinking tradition instead draw $\mathbf{s}^{(0)} \sim \mathcal{N}(0, \sigma^2 I)$ independently of the input. They rely on injection to bring the input in at every step. Huginn uses this design because of the observation that input injection and random initialization promote convergence to a steady state that is independent of the initialization ("path independence") [175]. This choice affects whether the core can be iterated beyond the training horizon (Section 6.2).

Injection. The operator $\mathcal{I}$ strongly affects extrapolation in depth, and studies disagree about how to choose it (Section 5.2). The corpus contains the following variants:

- *none*: $\mathcal{I}(\mathbf{s}, \mathbf{e}) = \mathbf{s}$. The input reaches later iterations only through the residual stream.
- *additive*: $\mathcal{I}(\mathbf{s}, \mathbf{e}) = \mathbf{s} + \mathbf{e}$. This is the default in looped Transformers for in-context learning, where removing injection makes the model "quickly diverge after the trained loop iterations" [552], and in looped models for length generalization [146].
- *concatenate-and-project*: $\mathcal{I}(\mathbf{s}, \mathbf{e}) = A[\mathbf{s}; \mathbf{e}]$ with a learned adapter $A : \mathbb{R}^{2d} \to \mathbb{R}^d$. Huginn reports that addition "works equally well for smaller models" but concatenation "works best at scale" [175].
- *gated or learned-scalar* mixing: $\mathcal{I}(\mathbf{s}, \mathbf{e}) = \alpha \odot \mathbf{s} + (1 - \alpha) \odot \mathbf{e}$.
- *history or trajectory* injection, which feeds a summary of $\mathbf{s}^{(<t-1)}$ in addition to $\mathbf{e}$.

3.2 Topologies as choices of (k_P, k_R, k_C) and of what is shared

The template covers every topology in Section 5. A *sandwich* model has $k_P, k_C > 0$. A *per-layer* ("immediate") loop repeats each layer before it moves to the next one, so it is a stack of cores with $k_R = 1$ each. A *cyclic* pattern shares M distinct blocks in a fixed order $(B_1, \ldots, B_M, B_1, \ldots)$. A *component-level* loop repeats only one sub-module, for example the token mixer, while the feed-forward network runs once. A *hierarchical* or *multi-timescale* model nests two recurrences, in which a fast inner core is iterated several times for each step of a slow outer core. The recursive reasoning models discussed in Section 11.2 have this structure. Recurrence along the *sequence* axis, which feeds a hidden state back as the next input embedding, is not depth recurrence. We treat it as a neighboring paradigm (Section 13).

3.3 Sharing maps

Let $\ell \in \{1, \ldots, D_{\text{eff}}\}$ index layers along the unrolled depth and let $\sigma(\ell)$ give the parameters used at depth ℓ. In a standard Transformer, σ is injective. Strict looping sets $\sigma(\ell) = \theta_{(\ell \bmod k_R)}$ inside the core. Relaxed sharing adds a step-specific low-rank or adapter term, $\sigma(\ell) = \theta + \delta_{t(\ell)}$. A mixture-of-experts core shares the expert pool across iterations, but the router can select different experts at each iteration. So the *active* parameters can differ across iterations even though the *stored* parameters are shared. Input-conditioned weight generation makes δ_t a function of the state. These choices cover the range between a fully looped model (σ constant on the core) and a standard model (σ injective).

3.4 Accounting identities

Let N_P, N_R and N_C be the parameter counts of the prelude, core and coda (excluding embeddings), and let T be the number of iterations. We write KV for the key–value cache of attention. Then, to first order,

$$\text{unique parameters} \quad N = N_P + N_R + N_C, \tag{4}$$

$$\text{effective depth} \quad D_{\text{eff}} = k_P + T k_R + k_C, \tag{5}$$

$$\text{forward FLOPs per token} \quad F_{\text{fwd}} \approx 2\,(N_P + T N_R + N_C) \;+\; \text{attention terms} \propto D_{\text{eff}}, \tag{6}$$

$$\text{KV cache per token} \quad M_{\text{KV}} \propto D_{\text{eff}} \;\; \text{unless caches are shared across iterations}, \tag{7}$$

$$\text{training activation memory} \quad \propto T \;\; \text{under full backpropagation through depth}. \tag{8}$$

Equations (4) and (6) express the defining property of the family. **Looping multiplies the compute spent in the core by T and leaves the number of parameters unchanged.** When the core holds most of the parameters, compute per parameter therefore grows by about T. Equation (7), for the KV cache, expresses the property that is most often overlooked when papers compare models. Unless the cache is shared or compressed, **the inference memory of a looped model grows with effective depth and not with the number of parameters** (Section 8.1).

3.5 Comparison regimes

Because of Eqs. (4) and (6), a looped model of width d cannot match a standard model of the same width on parameters and on compute at the same time. Every empirical comparison therefore uses one of the regimes in Table 2, explicitly or implicitly.

Consider a claim of the form "a looped model of size N matches a standard model of size kN". It is an ISO-PARAM statement about the looped model and an *iso-quality* statement about the

Table 2: Comparison regimes. Each regime fixes one quantity of the baseline, and a win means something different in each.

Regime	Baseline is matched on	What a win means
ISO-PARAM	unique parameters N	looping improves quality by using extra compute; says nothing about compute efficiency
ISO-DEPTH	effective depth D_{eff} (baseline has $\approx T\times$ the core parameters)	how close shared depth comes to unique depth; a win is surprising
ISO-INFER-FLOP	forward FLOPs per token	a compute-efficiency claim at inference
ISO-TRAIN-FLOP	total training compute	the standard scaling-law notion of efficiency
ISO-DATA	training tokens	a data-efficiency claim, often combined with ISO-PARAM
UNMATCHED	nothing (e.g. against public checkpoints trained on different data)	shows competitiveness but says nothing about the architecture

baseline. Unless training and inference FLOPs are also reported, the claim implies nothing about which model is cheaper to train or to serve. Section 15 classifies the main claims of the corpus by regime. It shows that most apparent contradictions are differences between regimes and not between results. Two further kinds of statement are not comparisons between architectures. A *within-model* statement compares settings of one model, for example one checkpoint run at different loop counts. A non-comparative statement reports an observation, such as a property of the learned dynamics. The meta-analysis counts both kinds separately.

Effective parameters. The effective-parameter form introduced for iso-depth scaling laws [433] gives a useful summary of how much a recurrence is worth,

$$N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}, \qquad \varphi \in [0, 1], \tag{9}$$

where N_{once} counts parameters applied once and N_{rec} counts parameters applied r times. We keep the symbol of the original study for this recurrence count, which plays the role of the number of iterations in Eq. (5). A value of $\varphi = 1$ means that a recurrence is as good as fresh parameters, and $\varphi = 0$ means that it adds no value. Section 7 reports the measured values of φ and how training choices change it.

3.6 Three axes of serial computation

A Transformer can spend additional serial computation on a fixed input along three axes. The first is *depth* (more layer applications per token), which is the subject of this survey. The second is *sequence* (more generated tokens), as in explicit or continuous chain of thought (CoT). The third is *refinement* (more denoising or editing steps over a whole sequence), as in diffusion language models. Table 3 compares them on the properties that matter for deployment and oversight, and Section 13 discusses the comparison in more detail. The axes are sometimes treated as if they were the same. One safety study groups Huginn with Coconut and CODI as models that "feed continuous hidden states back as inputs" [406]. However, Huginn iterates over depth within a token, while Coconut iterates over sequence positions. The same study also evaluates "only Coconut-style continuous thought models" [406]. Results on one axis should not be assumed to transfer to the other.

Table 3: Three axes along which a Transformer can spend extra serial computation on a fixed input.

	Depth recurrence (looped models)	Sequence recurrence (explicit / continuous CoT)	Refinement recurrence (diffusion LMs)
State carried between steps	full hidden state at every position	generated tokens or fed-back hidden states in the context	the whole (partially masked) sequence
Memory growth with more steps	KV cache grows with depth unless shared	context grows with every step	fixed sequence length
Supervision of intermediate steps	none required; optional per-iteration losses	explicit traces, or distillation into latents	denoising objective
Human readability of steps	none by default	explicit CoT is readable; continuous CoT is not	partially, via intermediate drafts
Where extra compute is spent	per token, possibly adaptively	per problem, sequentially	per sequence, in parallel

Takeaways

- Every method in this survey is an instance of Eqs. (1) to (3), with a prelude, a shared core with an injection operator, a coda and a loop-count policy.
- Looping multiplies compute per parameter by about T. Unless caches are shared, it also multiplies the inference memory by the same factor.
- At fixed width, a comparison cannot be both parameter-matched and compute-matched. A result can only be interpreted once its comparison regime is known.

4 Lineage: From Adaptive Computation to Looped Language Models

Looped language models build on four lines of work that developed mostly in parallel. These are adaptive computation in recurrent networks, parameter sharing in Transformers, recurrence as a way to reach algorithmic generalization, and the theory of looped Transformers as general-purpose computers. This section describes how the four lines came together, and Table 4 lists the milestones. The first two subsections follow the four lines up to 2024. The last two describe how they merged around reasoning in latent depth in 2025, and how the field moved to production-scale pretraining and systematic scrutiny in 2026.

4.1 Adaptive computation and shared layers (2016–2021)

Adaptive Computation Time (ACT) introduced a question that is still central to much of the field. It asked how a network can "learn how many computational steps to take between receiving an input and emitting an output" [187]. PonderNet later reformulated the stopping decision in probabilistic terms and learned "end-to-end the number of computational steps" [37]. The Universal Transformer brought recurrence in depth to the Transformer by applying one layer repeatedly with a per-position halting mechanism in the style of ACT [120]. A later analysis summarized its appeal as "better compositional generalization than Vanilla Transformers (VTs) in formal language tasks" [475].

Weight tying across depth was used before Transformers existed. A recursive convolutional network was used to separate the effects of depth, width and parameter count. This study found that depth helps when the number of parameters is constant. Untied layers beat tied ones at equal depth and width, but only because they have more parameters [131]. A residual network with shared weights was shown to be an unrolled recurrent network that performs close to its untied counterpart with far fewer parameters [318]. Value iteration networks applied one planning step repeatedly inside a policy [472].

A second motivation was model size. Work on machine translation shared all layers to obtain compact models. A translation model that stacks one shared layer six times matches an unshared six-layer model [112]. A more extensive study of this recurrent stacking finds that it approaches the unshared stack [113]. ALBERT's default was "to share all parameters across layers" to reduce BERT's memory footprint [285]. Staircase attention trades compute for quality at a fixed number of parameters, and its extreme Ladder variant repeats one shared Transformer at every step [247]. Later work relaxed full sharing into partial assignment strategies ("Sequence, Cycle, and Cycle (rev)") [470]. This work was the first systematic study of the sharing map defined in Section 3.3. Deep equilibrium models took recurrence to its limit by treating a weight-tied network as infinitely deep. They find the equilibrium points of this network "via root-finding" [30]. The fixed-point view appears again in modern analyses of looped models (Section 10).

4.2 Recurrence as algorithmic generalization (2021–2024)

A third line of work asked what recurrence adds beyond saving parameters. Recurrent networks trained on easy problems could solve harder ones "simply by performing additional recurrences during inference" [431]. Recurrent models were also found to "closely emulate the behavior of non-recurrent deep models, often doing so with far fewer parameters" [432]. Graph networks arrived at the same idea with learned halting. One iterative module "executes the same GNN layer repeatedly until a learned stopping criterion is met" and extrapolates shortest paths from small training graphs to graphs of thousands of nodes [478]. Iterating far beyond the training regime revealed a degeneration, which the authors named overthinking. Their fix was "a recall architecture that keeps an explicit copy of the problem instance in memory so that it cannot be forgotten" [38]. This recall architecture is the ancestor of the input injection used by almost every modern looped model (Section 3.1). Later work linked path independence (Section 3.1) to upward generalization, which is the ability to solve harder instances with more iterations. It also showed that input injection promotes path independence, which gave the rationale for Huginn's design [18]. An explicit pre-, looped- and post-transformer layout appeared a year before Huginn's prelude, core and coda [167].

In parallel, theoretical work showed that looped Transformers are universal. With programmed weights, they execute iterative algorithms by "placing them in a loop" [180]. A later analysis formalized the link between recursive neural networks and the Neural Data Router, a weight-shared Transformer variant that performs well on algorithmic tasks [101]. Experiments with trained models followed. Trained looped Transformers match standard ones on in-context data-fitting "while utilizing less than 10% of the parameter count" [552]. A later study reports that "looped Transformers with an adaptive number of steps significantly improve length generalization" [146]. CoTFormer made the connection to chain of thought explicit by observing that chain of thought "resembles employing a deeper transformer through re-applying the model multiple times" [358].

The obstacle to using these ideas for language modeling was the ratio of parameters to compute. As the MoEUT authors put it, "no previous work has succeeded in proposing a shared-layer Transformer design that is competitive in parameter count-dominated tasks such as language modeling" [108]. Two responses appeared in 2023–2024. The first put mixture-of-experts (MoE)

layers inside the shared block [108, 475]. The second converted pretrained models into looped ones with per-iteration low-rank relaxations [24].

4.3 Reasoning in latent depth (2025)

In 2025 the motivation shifted from efficiency to reasoning. One analysis argued that "many reasoning problems require a large depth but not necessarily many parameters". On synthetic tasks, it showed that "a k-layer transformer looped L times nearly matches the performance of a kL-layer non-looped model" [426]. Huginn showed the idea at scale with a 3.5B-parameter recurrent-depth model trained on 800B tokens [175]. PonderLM fed a probability-weighted embedding back for another forward pass, "repeatedly invoking the forward process within a single token generation step" [575]. Mixture-of-Recursions combined sharing with token-level adaptivity and used "lightweight routers" to assign "different recursion depths to individual tokens" [25]. On puzzle benchmarks, the Hierarchical Reasoning Model (HRM) reached 40.3% on ARC-AGI with 27M parameters and about 1,000 training examples [499]. The Tiny Recursive Model simplified it to one 2-layer network with 7M parameters and reported 45% on ARC-AGI-1 [245]. The Universal Reasoning Model reported "53.8% pass@1 on ARC-AGI 1 and 16.0% pass@1 on ARC-AGI 2" [168]. The year ended with Ouro, a family of pretrained looped language models trained on 7.7T tokens [604].

4.4 Industrialization and scrutiny (2026)

In 2026, looped models moved from proofs of concept to production-scale pretraining. At the same time, they became the subject of systematic scrutiny. On the production side, one work trained a family of "7B PLT coders with different loop counts, from scratch on 18T tokens" [550]. Other works built looped mixture-of-experts models with 20B total and 2B active parameters [169] and a 3B looped model trained on 28T tokens [368]. Another architecture has a globally shared feed-forward memory that attention "reasoners" query repeatedly. Its 7B model "achieves similar downstream score as a 7B Transformer baseline with 62.6% of baseline's training data" [75]. On the scrutiny side, scaling laws expressed the value of a recurrence as an equivalent number of unique parameters [84, 433]. Stability analyses traced loss spikes to "large spectral norms in their injection parameters" [397]. Inference systems addressed the memory and latency cost of looping (Section 8). The first evaluation of chain-of-thought monitorability for looped models also appeared [502].

Across these periods, the main motivation for looping moved from adaptive computation and smaller models to algorithmic generalization and then to reasoning. The central question changed with it. Earlier work asked whether shared layers could be competitive at all. Current work asks what a recurrence is worth at a fixed budget, which is the subject of Sections 7 and 15. Section 5 turns from this history to the design choices that looped models make.

Table 4: Milestones in the development of looped models. Dates are first arXiv submission.

Date	Work	Contribution
2016-03	ACT [187]	learned number of computation steps
2018-07	Universal Transformer [120]	recurrence over depth with per-position halting
2019-09	ALBERT [285]; DEQ [30]	cross-layer sharing at pretraining scale; infinite-depth fixed points
2021-04	Lessons on sharing [470]	partial sharing maps (sequence, cycle)
2021-06	Deep Thinking [431]	easy-to-hard generalization by extra recurrences
2021-10	Neural Data Router [107]	gated shared layers for systematic generalization
2022-02	Recall + progressive training [38]	input re-injection against overthinking
2023-01	Looped Transformers as computers [180]	universality of looped Transformers
2023-10	CoTFormer [358]; SUT [475]	loops vs. chain of thought; sparse experts in shared layers
2024-05	MoEUT [108]	shared-layer MoE competitive in language modeling
2024-10	Relaxed Recursive Transformers [24]	converting pretrained LLMs with per-step LoRA
2025-02	Huginn [175]; latent thoughts [426]	3.5B recurrent-depth model; depth over parameters for reasoning
2025-06	HRM [499]	multi-timescale recursion for puzzles
2025-07	Mixture-of-Recursions [25]	token-level recursion depth
2025-10	TRM [245]; Ouro [604]	7M-parameter recursion; LoopLMs at 7.7T tokens
2026-04	Parcae [397]; iso-depth laws [433]	stable looping; value of a recurrence in unique parameters
2026-06/07	LoopCoder-v2 [550]; Loopie [169]; Nanbeige4.2 [368]	18T–28T-token looped pretraining; looped MoE
2026-10	Monitorability of LoopLMs [502]	first oversight evaluation

5 The Design Space of Looped Models

Section 3 reduced every looped model to a small number of choices: what is looped, how state is carried between iterations, how strictly parameters are shared, and how many iterations are run. This section reviews the options that the corpus has explored for each choice and what is known about their effects. It also covers hybrids that combine looping with other backbones (Section 5.4). The training methods that make these designs work are the subject of Section 6.

5.1 What is looped

Table 5 describes representative models in terms of the template of Section 3, and Figure 5 shows the main topologies. Papers in the corpus have tried every topology that the template allows. Most models in the table use strict sharing. They differ mainly in which layers are looped, how the input re-enters the loop and how the loop count is set, and the rest of this section discusses these choices.

Whole stack versus sandwich. The earliest models loop the entire network [120, 285], and some later designs keep this form. One example is a recurrent-depth Transformer with parameters shared across layers, derived from energy minimization on the hypersphere [217]. For generative

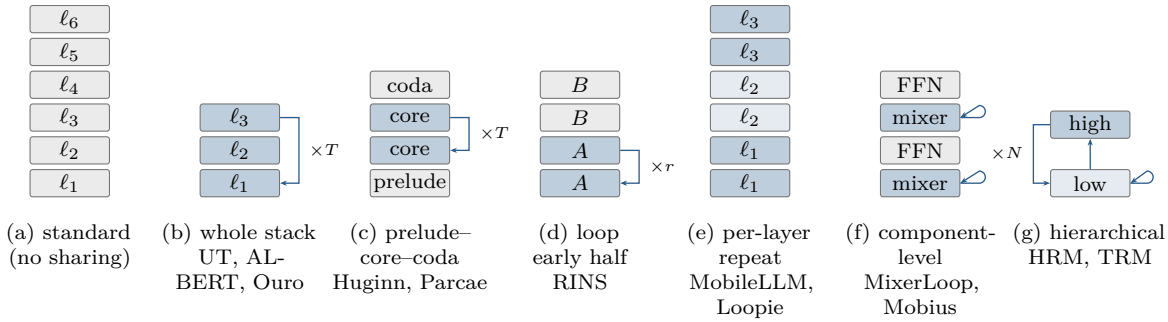


Figure 5: Loop topologies, each written as an instance of the template in Section 3. Gray boxes are unshared layers and blue boxes are shared layers, where the same shade means the same parameters. Arrows mark repeated application, and example models are listed under each panel.

models, naive sharing of all layers hurt performance, and keeping the first and last layers unique ("sandwich" sharing) restored a good balance [411]. CoTFormer also kept unshared layers at the beginning and the end [358], and the prelude–core–coda layout became standard with Huginn [175]. Boundary layers have measurable benefits. A coda reduces the degradation that comes from running fewer loops than in training, and a prelude helps reasoning when the model runs more loops than in training, although knowledge and overall scores still decline [609]. The fraction of layers inside the loop appears to have an optimum between the two extremes. In one sweep, loss is U-shaped in this fraction, with a minimum at 30–40% [564].

Placement of the unique layers. Studies disagree on where the unique layers should go. The best-controlled study of loop placement is a sweep of 59 models at matched parameters and training compute. It finds that looping the *first* half and keeping the second half unique beats both repeating the whole stack and a Huginn-style middle loop [12]. Other designs keep the loop in the middle. They use a 25/50/25 split [573], follow the Huginn-style middle-loop pattern [223], or cycle only the middle layers between unshared first and last layers [299]. One recent architecture drops fixed boundary layers entirely and loops the whole stack with mixture-of-experts (MoE) layers and attention over earlier depth states [263].

Block size and loop count. Physical depth and loop count are not interchangeable when effective depth is fixed, but two studies point in different directions. In MoEUT, smaller recurrent groups with more steps beat larger groups with fewer steps [108]. A controlled study of depth extrapolation prefers the larger block. At 20 effective layers, a 2-layer block looped ten times peaks early and degrades under extrapolation, while a 4-layer block looped five times keeps improving [609]. MoEUT measures quality at the trained depth and the second study measures behavior beyond it, so the two results need not conflict.

Finer and coarser granularities. The repeated unit can be smaller or larger than a block of layers. At the fine end, per-layer repetition applies each block twice before moving on to the next one, and it suits on-device models [330]. Between the extremes, some designs loop a group of layers. One loop network repeats a subset of GPT-2 layers with residual connections and improves language modeling [371]. A translation model loops its Transformer encoder and decoder blocks in depth with shared parameters [302]. At the coarse end, multi-resolution loops iterate at different sequence resolutions [564], and hierarchical recursion nests two loops at different timescales (Section 11.2).

Table 5: Representative looped models as instances of the prelude–core–coda template (k_P / $k_R \times T$ / k_C). "Whole" means the entire stack is looped.

Model	Topology	Injection / state	Sharing	Loop policy; largest scale
Universal Transformer [120]	whole	none; time-step embedding at each step	strict	fixed or ACT; Transformer-base size
ALBERT [285]	whole	none	strict	fixed; 233M
Subformer [411]	1 / shared / 1 (sandwich)	none	partial	fixed
CoTFormer [358]	2 / 21×5 / 1	later repeats attend to all earlier repeats	strict	fixed or routed
MoEUT [108]	whole (groups of 2–4 layers)	none; peri-layernorm	MoE in loop	fixed; 1.04B
Relaxed Recursive [24]	whole (cycle, 2 loops)	none	per-loop LoRA	fixed; converted from 2B
Huginn [175]	2 / 4×r / 2	concatenation adapter; random initial state	strict	randomized; 3.5B, 800B tokens
RINS [12]	0 / first half×r / second half	none	strict + per-r adapters	stochastic; 1B
PonderLM [575]	whole	probability-weighted embedding added to input	strict	fixed or random; 2.8B
Mixture-of-Recursions [25]	shared stack	none	strict	token-level routing
Ouro [604]	whole (24 layers ×4)	none	strict	learned exit gate; 2.6B, 7.7T tokens
MeSH [563]	3 / 6×3 / 3	routed multi-slot memory	strict	fixed; 1.4B
Parcae [397]	middle third looped	spectrally constrained injection	strict	randomized; 1.3B
Hyperloop [573]	25% / 50%×3 / 25%	loop-level hyper-connections [602]	strict	fixed
LoopCoder-v2 [550]	parallel loops	shared first-loop KV, gated window	strict	fixed; 7B, 18T tokens
Intern-S2-Mobius [75]	component (shared FFN memory)	hidden state as carrier	shared FFN pool	7B from scratch
LoopMoE [82]	2 / 2×4 / 2	no re-injection; iteration-conditioned normalization	MoE in loop	fixed; 9B
Loopie [169]	per-layer, each layer ×2	none	strict, MoE layers	fixed; 20B total, 2B active
Attractor Models [152]	backbone + small weight-tied attractor	persistent additive injection; fixed point	strict	convergence; 770M

Per-layer versus whole-block repetition. Studies also disagree on whether to repeat each layer in place or to cycle through a block. The largest controlled topology ablation uses a 6B MoE model. In this ablation, repeating each layer in place beats looping the whole stack, but only after about 1.2T tokens [169]. Before that point in training, the whole-stack loop leads. Smaller studies favor interleaved block cycling over adjacent repetition [108]. When recurrence is applied to individual layers of a small language model, it helps most on the earlier layers [374]. The ranking of repetition patterns also differs between tasks, and the authors suggest that task differences or

the placement of normalization may explain this [470]. The training horizon, normalization and sparsity are plausible moderators of these results. They also agree with the observation that the loop count alone does not describe a looped model, because the looped operator and the order of interleaving also matter [320].

Component-level loops. Some designs loop only part of a layer. One shares a global feed-forward memory that attention layers query repeatedly [75]. Another repeats only the token mixer while the feed-forward network runs once [320]. The repeated part can also be the high-entropy attention heads [87], the expert sublayer with a router per iteration [519], or the feed-forward network with token-adaptive loop counts [377]. One design loops only the efficient-attention half of the network [466]. A dual-path design runs a looped sublayer and a once-applied sublayer in parallel. At matched FLOPs, the best mixture of the two beats both pure width and pure looping, and pure looping is the worst [159].

5.2 How state is carried

Plain chaining. In the simplest design, a shared block runs on its own output with no additional mechanism. This is not enough for deep iteration, because the model tends to diverge or stall as iterations accumulate. A chained looped model "fails to converge after 32 iterations" whereas a model with an inter-iteration residual converges [13]. A gated variant reports that without its gate "recursive layer application makes the model strictly worse" [229]. Normalizing the residual stream between iterations is a common fix, because residual scales "can become incompatible across depths" [263]. Another fix constrains the update itself. One design replaces layer stacking with a single shared block iterated by a contractive update inspired by ordinary differential equations, in which a step size controls stability and update magnitude [181]. Mechanisms of this kind are older than recurrent-depth language models. The Neural Data Router is a weight-shared Transformer whose copy gate lets a position pass its state through an iteration unchanged. It reaches "100% length generalization accuracy on the classic compositional table lookup task" [107], provided the shared layer is applied at least as many times as the computation has steps. For example, "the 12-layer model generalizes almost perfectly, but the 10-layer one does not" [107].

Input injection. Studies disagree about the value of input injection and about the best way to do it. Huginn re-injects the input at every step and finds that concatenation works best at scale (Section 3.1). In contrast, a controlled study finds that, for knowledge under depth extrapolation, dense initial-state injection of this kind "causes a catastrophic performance collapse" [609]. In the same study, lightweight scalar or channel injection gives negligible gains over no injection. Another study argues that additive re-injection creates a bias in the recurrence, and it replaces this injection with a recurrence on the deviation from an anchor state [256]. For fixed-point and mixture-of-experts looped models, the evidence is mixed. On one side, persistent additive injection lets 99.7% of tokens converge, while injecting only at initialization lets 12.4% converge [152]. Removing embedding injection also raises validation perplexity from 18.62 to 26.05 in a looped MoE [205]. On the other side, another looped MoE removes per-step re-injection because its authors argue that it suppresses differences between iterations [82]. Several strong models also use no re-injection at all [12, 108, 121, 169]. One plausible interpretation is that injection matters mainly for extrapolation beyond the trained depth, and that high-capacity injection can overfit to the loop count used in training.

Initial state. Studies also disagree about the best initial state for the loop. Huginn starts from random noise (Section 3.1), whereas in attractor models a warm start from a backbone proposal outperforms zero or noise initialization [152].

Access to earlier iterations. In looped language models, mechanisms that give later iterations access to earlier iterations help consistently. Letting later repeats attend to the states of earlier repeats beats plain looping at equal size and an equal number of repeats [358]. A routed multi-slot memory replaces residual and anchor schemes, and it beats a vanilla 1.4B model while using about a third fewer non-embedding parameters [563]. Attention over earlier depth states improves data efficiency [263]. In a controlled study, history-state injection combined with loop gating, a form of timestep conditioning, is the best combination [609]. Unbounded access is not always better on algorithmic tasks. One study compares recurrent-depth Transformers that carry only the current state, an expanding cache of earlier states (CoTFormer) or a bounded gated memory. On a cellular-automaton task, the current-state model extrapolates to unseen depths more reliably than CoTFormer, and the gated memory improves depth extrapolation and delayed recall over both [252]. These mechanisms also have a memory cost that papers rarely report.

Step conditioning. The evidence on conditioning the core on the step index is also contested. Huginn found that this conditioning breaks extrapolation (Section 3.1). However, the Universal Transformer uses a time-step embedding by default [120], and a sinusoidal step embedding is a strong baseline at small scale [256]. The step signal can also enter through the weights, as in a single shared Transformer block applied repeatedly with depth-specific low-rank level signals [211]. Depth embeddings help only adaptive routing in CoTFormer [358]. Timestep gating helps extrapolation in some configurations but not in others [609]. No study tests step conditioning for both fixed-depth quality and extrapolation on the same task.

5.3 How much is shared

Feed-forward capacity. Compared with an unshared model of the same architecture, a strictly shared model has too few parameters, and the feed-forward network (FFN) is the bottleneck. The ablation in ALBERT locates the cost and finds that "most of the performance drop appears to come from sharing the FFN-layer parameters" [285]. All of the field's fixes restore feed-forward capacity. The most direct fix widens the shared network. In translation Transformers that share one feed-forward network across layers, widening it restores accuracy [392]. The other fixes add experts inside the loop or relax the sharing, as the next two paragraphs describe.

Mixture-of-experts inside the loop. Placing expert layers in the shared block lets different iterations route to different experts, so the active parameters differ across iterations even though the stored parameters are shared (Section 3.3). An early design of this kind shares one Transformer block across depth and uses a mixture of experts as its feed-forward layer [547]. Experts in the shared block later made shared-layer models competitive in language modeling. MoEUT slightly beats parameter-matched dense models up to 1B parameters [108], and the same idea is the basis of the looped-MoE scaling results of Section 7 [84, 288, 507]. In looped MoE models, the expert pool is the cheapest component to share. Tying experts across groups of four consecutive layers cuts total parameters by up to 52% with negligible loss [233]. Other designs also share experts across layers. Expert pools shared across adjacent layers, with depth embeddings, beat standard experts [404]. A shared pool of universal experts can be reached from many layers, which enables recursive reuse [89]. The 7B component-level design described in Section 5.1 shares a global feed-forward pool in a similar

way [75]. Studies disagree on which component must stay untied. One design gives each loop its own router and shares attention and experts [205]. Another unties attention and shares experts and routers [509]. A third finds that untying attention matters an order of magnitude more than untying the router [233].

Relaxed sharing. Per-iteration low-rank adapters relax strict tying and add few parameters. When Gemma 2B is converted, a rank-512 relaxation recovers 58.4 average accuracy, compared with 58.6 for the original model [24]. Per-step weights that are generated from the input relax the tying further [229]. It is unclear whether per-step parameters are needed at all. A later conversion study finds that shared weights and per-step adapters perform within two points of each other (Section 6.4). Shared weights plus a small per-layer term are also common outside looped language models. Speech Transformer encoders share full-rank weights between adjacent layers and add per-layer low-rank components [516]. A post-training compression method shares weights between neighboring Transformer blocks and trains low-rank differences [354]. An on-device sequence-to-sequence Transformer shares layers and adds per-layer adaptation to recover quality [174]. A more general form builds network blocks as linear combinations of a shared set of neural bases [497].

5.4 Hybrids

Looping can also be combined with other backbones and with iterative solvers. With linear-time token mixers, a hybrid looped model matches the average zero-shot accuracy of a full-attention loop and has about 5.7× its decode throughput at 8K context and batch size 8 [121]. Depth recurrence has also been applied to state-space models [151, 429] and to diffusion language models [291]. It also works with cyclic block orders, in which different workers pass through shared blocks in different orders [606].

Other hybrids treat the loop as an iterative solver. Attractor models solve a small weight-tied block to a fixed point, with a warm start from a backbone proposal (Section 5.2). The authors report that the benefit is largely internalized, and the readout after zero or one iteration is already strong [152]. Viewing each loop transition as an optimizer step suggests transition designs with weight decay and adaptive step sizes. These designs improve exact-match accuracy over non-looped models at matched training compute, with about half the parameters [223].

5.5 How many loops: loop-count policies

The loop-count policy π of Eq. (3) decides how many iterations each sequence or token receives. Table 6 summarizes the policy families in the corpus. Uniform extra depth is wasteful and sometimes harmful, which motivates adaptive depth. However, most reported savings from adaptive depth are *proxies*, such as average loop counts or nominal FLOPs. When the same papers also time their models, the proxies and the measured times often disagree.

Uniform depth. With a uniform depth, some inputs get too many iterations and others get too few. Extra iterations repair some predictions and break others. In one study, a second iteration corrects 8.7% of predictions but turns 2.1% of correct predictions into errors [162]. In another, accuracy on MathQA peaks at two loops (54.7%) and falls to 47.5% at sixteen [518]. Work on early exit in ALBERT, a weight-shared model, already observed error rising after ten of twelve shared layers [601]. A well-tuned fixed depth is still a strong baseline. In early comparisons with recurrent networks, fixed repetition "matches the performance of ACT" [156]. Despite this, adaptive methods

Table 6: Loop-count policies. "Savings type" records how the corpus measures the benefit: a proxy (average loops or nominal FLOPs) or measured wall-clock time.

Policy	Representative work	Training signal	Savings type
Fixed	most models; ACT comparison [156]	none	n/a
Randomized in training	Huginn [175]; [331, 518]	sampled depth per step or example	robustness (no savings)
Learned halting (penalty or prior)	ACT, PonderNet [37, 187]; AdaPonderLM [457]; Ouro-style gates [569, 604]	ponder cost; KL or entropy to a depth prior	mostly proxy; measured in [569]
Learned halting (oracle labels)	Think-at-Hard [162, 561]; RecurTrace [518]	imitation of loss-gain or mismatch labels	measured
Token-level routing	Mixture-of-Recursions [25]; ITT [88]; T-LoopFormer [565]	language-modeling loss, plus capacity or balance terms (MoR) or alignment terms (T-LoopFormer)	measured with engine support
Convergence rules	successive-output KL [331]; fixed-point residual [363]; patience [601]	none for the rule (patience needs trained per-layer classifiers)	proxy; measured in [601]
Budget-conditioned	LoopFormer [235]	shortcut trajectories with consistency loss	user-chosen budget
Reinforcement-learned depth	none for looped language models (related: [214])	reward minus depth cost	n/a

are often compared only with full depth and not with the best fixed depth at matched average compute.

Randomized depth. Sampling the iteration count during training (Table 6) makes the model robust to the inference depth, and useful depth then saturates at the mean training depth. This training gives one checkpoint that works across depths, which fixed-depth training does not give. For example, a model trained at a constant four loops has loss 1.97 at four loops but 6.01 at one [331]. With randomized training, loss falls monotonically up to the mean training depth and then stays flat. The authors summarize this as "trained at mean 4, saturates at 4; trained at mean 8, saturates at 8" [331]. Raising the maximum inference depth above the maximum training depth adds cost but does not improve accuracy [561]. Randomization can also apply to which layer runs. Random recursive models sample one layer from a learned pool at each step, which allows variable depth [357].

Learned halting. In pretrained looped models, learned halting gates are unreliable. Gates in the ACT and PonderNet tradition (Section 4), which are trained with a penalty or a prior, fail in a few recurring ways (Table 7). They collapse to the shallowest depth, ignore their cost and run to full depth, or freeze at a near-constant depth. A large coefficient on the Kullback–Leibler (KL) divergence to the prior "collapses the gate onto a near-fixed-depth operating point" [395]. The same study finds that the damage comes from the trajectory that joint training of the gate induces, and

Table 7: Failure modes of learned halting reported in the corpus.

Failure	Where reported
Collapse to the shallowest depth	[156, 343, 424, 518, 561]
Little or no response to the cost, for the ACT baseline or under weak penalties (runs to full depth)	[142, 457]
Collapse to a near-fixed operating point	[395]
Sampled exits leave far too early	[34]
Train–test mismatch when tokens actually exit	[561, 569]
Ponder-cost weight is hard to tune	[100, 370]; one graph model is robust to it on one of two datasets [460]

not from the capacity of the gate. Gates trained on full-depth trajectories also face a train–test mismatch when tokens actually exit early. In a looped state-space model, an Ouro-style gate that is not trained on skipped states saves no compute, and forcing skips increases perplexity [569]. The table also lists sampled exits that leave far too early and a ponder-cost weight that is hard to tune, although one graph model reports robustness to this weight. Graph networks with learned halting showed similar problems earlier: "The ACT algorithm provides adaptive but usually short iterations of layers" [478].

Conditions for reliable halting. Gates trained on supervised oracle labels avoid some of these problems. The labels come from a reference model or an offline sweep [162, 518], or they are recomputed online from the backbone during joint training [561]. The failures seem specific to gates trained jointly with weight-tied language models. Differentiable halting fine-tuned on an unshared BERT classifier is stable, in that "the same values of ponder penalties give rise to similar efficiency outputs" [143]. Learned halting has also been used in other settings. One design combines per-layer adaptive looping and learned halting with gated memory banks, and in it looping mainly helps mathematical reasoning [158]. A controlled study examines per-token depth allocation in a prelude–core–coda model [361]. Convolutional recurrent networks with learned halting scale their computation to harder instances of the same visual tasks [492]. Halting is useful for more than saving cost, because a learned stopping rule also improves accuracy when running all iterations goes past the best depth [39, 102].

Token-level routing. Routers that assign a depth to each token fix the compute budget as part of their design. Mixture-of-Recursions and its successors decide *which* tokens go deeper, and they do not decide *how much* total compute to spend. A capacity per recursion step, a target routing ratio or a balance loss sets the average depth in advance [25, 79, 88]. With an engine that batches tokens by depth, the savings appear in measured time and memory. Mixture-of-Recursions reports 25% fewer training FLOPs, 19% less training time and 25% lower peak memory at 20B tokens. It also reports up to 2.06× decode throughput [25]. Two caveats apply. First, its main comparisons are at matched training compute, and "this superior performance is achieved despite using nearly

50% fewer parameters" [25], so part of the gain comes from seeing more tokens. Second, its router scores become so separated that it is "challenging to adjust top-k values after training" [25]. Other routers decide more than the depth. In one design, routers send each token through shared modules in any order, so the model learns both its depth and its sharing pattern [105]. In another, per-layer routers on a frozen language model choose whether to skip, execute or repeat each layer [208].

Training-free convergence rules. Stopping rules that need no training and halt when the state converges are competitive with learned gates. In randomized-depth recurrent models, the state of each token converges, and different token types converge at different speeds [331]. Halting each token once its successive outputs stop changing reaches depth-8 quality with 4.94 loops on average. A linear router trained on convergence labels from the same model needs 7.99 loops [331]. Post-hoc confidence readouts are competitive with Ouro's pretrained gate on the released checkpoints [395]. A fixed-point-residual criterion also outperforms ACT-style halting on puzzle tasks [363]. An analysis of loop-step geometry motivates an early-exit rule based on second-order differences between steps. Its authors find that this rule beats Huginn's KL-divergence exit rule in performance, stability and time efficiency [388]. The largest model on which halting policies are compared under measured latency has 370M parameters [569]. No study compares a jointly trained gate with a convergence rule on the same pretrained language model at that scale or above, and this is the most important missing experiment.

Budget-conditioned depth. A loop that adapts its depth to a given compute budget is flexible, but it loses quality at low budgets. Conditioning each iteration on normalized time and step size lets one model serve any budget up to its maximum [235]. At small budgets, however, this elastic looped model has worse Pile perplexity than non-looped models with matched FLOPs, so its flexibility costs some quality.

Proxy and measured savings. Table 8 lists the cases in which the same paper reports both a proxy saving and a measured saving. A per-token iteration policy with FLOPs within 4–5% of the non-looped model was 43% slower in measured latency [162]. In another paper, an average of 2.76 loops implied a 1.45× speedup, but the measured speedup was 1.29× [569]. In a third, a 38% reduction in average depth gave "about 0 percent realized" speedup in the authors' harness [331]. Every positive measured result for looped language models in this subsection depends on batching or engine work that keeps the hardware from idling when per-token depths are ragged (Section 8).

Audits without weight sharing. Two audits of adaptive depth without weight sharing confirm the gap between proxy and measured savings, and they add a second confound, the choice of baseline. Trained layer-skipping routers on Qwen2.5 give measured speedups of 1.08–1.33× over the full model, and these speedups are "smaller than layer-count-implied theoretical savings would suggest" [450]. On a microcontroller, nominal multiply-accumulate operations (MACs) do not predict the order of measured latencies. For example, one model with 21% more MACs runs 15% faster. Per-frame routing saves latency inside one network, but compared with fixed-depth models retrained at the same realized compute, "routing brings little advantage" [286]. No paper on looped language models in Table 8 compares with retrained fixed-loop models at matched realized compute.

Table 8: Proxy savings versus measured savings reported in the same paper.

Work	Proxy	Measured
Think-at-Hard [162]	FLOPs within 4–5% of the non-looped model	latency 301.4 s vs. 210.6 s (batch 1)
CHASE [569]	2.76 average loops (implies 1.45×)	1.29×; no gain at batch 1
Per-token convergence [331]	38% fewer average loops	about 1.5× best case; about 0% realized
Adaptive depth without sharing (for comparison)		
Layer-skipping audit [450]	skipped-layer count (not reported)	1.08–1.33× for trained routers (batch 1)
Speech early exit [286]	analytic MACs	21% more MACs yet 15% faster; no gain over retrained static models

Takeaways

- The placement of unique layers, the block size and the choice between per-layer and whole-block repetition are not settled. The training horizon, normalization and sparsity are plausible moderators of the conflicting results.
- Experts inside the loop and relaxed sharing restore the feed-forward capacity that strict sharing removes. In looped language models, access to earlier iterations helps, but its memory cost is rarely reported.
- Training with randomized depth makes a looped model usable at several depths, which fixed-depth training does not. Useful depth then saturates near the training mean.
- Learned halting gates fail in a small number of recurring ways. Simple convergence rules match or beat jointly trained gates in the controlled comparisons so far, most of them at small scale, while gates trained on oracle labels avoid some of the failures.
- Most reported savings are proxies, and for looped language models the measured speedups so far require batching or engine support for ragged depths. None of these speedups is compared with retrained fixed-loop models at matched realized compute.

Open questions

- Does a jointly trained gate beat a convergence rule on a pretrained language model above 1B parameters when latency is measured?
- Why does useful depth saturate at the training mean, and can any training distribution move it?
- Can reinforcement learning learn loop counts for looped language models? No paper in the corpus uses reinforcement learning for this.

6 Training Looped Models

Section 5 reviewed the architectural choices of looped models, and this section reviews how such models are trained. Training a looped model differs from training a standard Transformer in three ways. First, the same parameters receive gradients from every iteration, so updates are correlated across depth. Second, the forward pass is a discrete dynamical system, and its stability is not guaranteed. Third, a loop-count policy must often be learned together with the model. This section groups the corpus by the stage at which a looped model is obtained. The stages are pretraining from scratch (Section 6.1), keeping the recurrence stable (Section 6.2), propagating gradients through

depth (Section 6.3), converting existing checkpoints (Section 6.4) and post-training (Section 6.5). We summarize the evidence as a training recipe in Table 10.

6.1 Pretraining from scratch

Recipes at scale. Large looped models have been trained with multi-stage schedules. A later study summarizes the schedule of Ouro as pretraining at eight recurrent steps, continued pretraining at four, annealing, context extension and supervised mid-training. In this schedule, the depth was cut from eight to four after loss spikes [343, 604]. The later study shows that the same 1.4B architecture can be trained with one 210B-token pretraining run followed by 100B tokens of response-only mid-training, which is 310B tokens in total [343]. Two of its findings apply to looped training more generally. First, a fixed four-step loop is enough, because in an eight-step run "loops five to eight change the loss by about 0.01 nats or less" [343]. Second, high-quality mid-training contributes more to reasoning than pretraining does. Compared with the checkpoint at the end of pretraining, mid-training "more than doubles GSM8K accuracy" [343].

Exit-gate collapse. When a learned exit distribution is trained jointly with the model, it can collapse onto the first iteration without any visible sign in the language-modeling loss. With a small entropy coefficient and no warmup, the probability of exiting at the first step "stays at p0 =1.00 for all 2,267 steps of the run" [343]. A learning-rate warmup removes this early collapse, and a larger KL coefficient lets the gate recover if it does collapse [343]. The practical lesson is to monitor the exit distribution directly.

Residual scaling. For looped models, rules for scaling the residual updates must take weight tying into account. The updates of a tied block are correlated across iterations, so the residual stream grows with the loop count. As a result, depth-based residual scaling rules that were designed for untied networks are miscalibrated. One analysis finds that scaling residual branches by $1/T$, the inverse of the loop count, restores learning-rate transfer across loop counts [506]. A second analysis raises the DeepNorm exponent for post-layer-norm models [308]. A third uses a fixed $1/T$ branch scale [448]. The rules disagree in detail, and they were derived for different normalization placements. None of them has been tested above 0.5B parameters in pretraining from scratch.

Auxiliary objectives. Later iterations of a looped model do not automatically become useful, and several objectives give them a target of their own. Latent multi-token prediction aligns each successive iteration with a more distant future token [448]. Another looped model is trained with local denoising objectives at decreasing noise levels [461]. For the Tiny Recursive Model (TRM), an analysis of recursion steps as policy improvement uses per-step supervision to avoid wasted compute [21]. In contrast, another TRM-style recursive model is trained with detached history and a loss only on the final step [197]. Another approach changes the target itself. Adversarial training for recurrent reasoning models replaces single-point targets with a local target distribution [40].

Curricula. Curricula are a second way to make later iterations useful, and most of them grow the depth during training. In retrofitted models, gradually increasing the mean of the sampled recurrence during training is efficient in data and compute [348]. A recursion-depth curriculum with decaying supervision weights speeds up TRM training [400]. On synthetic tasks, training in single-pass mode before switching to looped computation speeds up training [185]. An older curriculum acts on sharing instead of depth. It pretrains BERT with weights shared across all layers at first and then unties them, which cuts the number of training iterations [554].

Table 9: Failure modes of looped training and the fixes proposed in the corpus. Scales are the largest at which each is shown.

Failure mode	Evidence	Proposed fix
Exit-gate collapse, invisible in the loss	1.4B [343]	warmup and a larger entropy/KL coefficient
Vanishing gradients through the tied block under post-norm or sandwich norm	1.4B whole-stack loop [343]	pre-norm-family decoder with a final norm before the head
Residual growth from correlated tied updates	0.3–0.5B [161, 308, 506]	$1/T$ or DeepNorm-style residual scaling, injection in every layer
Hidden-state norm drift beyond the training horizon	44M–1.4B [440, 556]	spectral-radius penalty, scale-visible readout or norm penalty, inter-loop normalization
Accuracy collapse when running more loops than trained	1.4B [556, 608]	random loop counts plus stability penalty, loop dropout
Optimizer instability in retrofits	1B [348]	Muon instead of AdamW

Controlled comparisons with dense models. The most informative training study in the corpus compares a 1.4B looped model with dense models trained on the same data [343]. At matched parameters and data, the four-step looped model beats its dense counterpart on all 12 benchmarks (83.32 versus 69.14 on GSM8K). At matched inference compute and effective depth, a 72-layer dense model with 3.9B parameters is "ahead or level on every benchmark" [343] (82.79 on GSM8K). In this comparison, the only advantage left for the looped model is its parameter count, which is "36% of those of dense-ouro-3.9B" [343]. This one study shows the central lesson of Section 15, which is that the same pair of models gives a win or a tie depending on what is matched (Figure 9). An older analysis of parameter sharing in Transformers suggests one source of the matched-parameter gain. It finds that depth-wise parameter sharing helps mostly through better convergence and only to a small extent through added model complexity [321].

6.2 Keeping the recurrence stable

Table 9 lists the failure modes that the corpus isolates and the fixes proposed for each, and three patterns stand out.

Normalization. The normalization choices that make training reliable are not the ones that keep the model stable beyond the training horizon. In whole-stack loops, pre-norm-family decoders with a final normalization before the language-model head train reliably, while post-norm and sandwich norm fail [343]. But in pre-norm models, the hidden-state norm can grow without bound when the loop is run past the training horizon [556]. A per-iteration cross-entropy loss through scale-invariant readouts cannot see this growth. As a result, dense per-loop supervision "trains exits" but does not control recurrent scale [440]. With post-norm, the norm stays bounded, but the model performs poorly [556]. An analysis verified at CPU scale explains why pre-norm can stabilize the loop near its fixed points. It finds that pre-LayerNorm inside a looped block acts as an implicit gain control

that makes the recurrence contractive at every verified fixed point [55].

Depth extrapolation. More loops do not automatically help, and accuracy commonly peaks at the trained loop count and declines beyond it. Fine-tuning Ouro-1.4B with a spectral-radius penalty and random loop counts reduces the GSM8K drop from peak performance at eight loops from 20.47% to 8.26% [556]. Dropping per-loop adapter applications during fine-tuning also improves accuracy when the model is evaluated at twice the trained depth [608]. Stability beyond the horizon has to be trained into the model, and it does not appear by default. Inference-time control of the update is a complementary approach. An online scheduler that adapts the update scale of recurrent reasoning models improves their accuracy without retraining [498].

Injection. For an Ouro-style whole-stack model trained from scratch, concatenation-based injection slightly worsened the training loss [343]. Retrofitted models, in contrast, consistently need some form of anchoring to the input (Section 6.4). A fixed-point analysis of single-layer looped Transformers gives one reason for anchoring. It finds that recall of the input combined with outer normalization stabilizes looped Transformers [279]. Section 5.2 reviews the wider disagreement about injection, which may depend on the topology and on whether the model must extrapolate in depth.

6.3 Gradients through depth

Full backpropagation through T iterations stores the activations of every iteration, so memory grows linearly with depth. Studies in the corpus use four alternatives: truncated backpropagation, detached deep supervision, per-iteration losses and one-step gradients. *Truncated backpropagation* through the last iterations makes deep recurrence affordable. For example, a retrofitting study backpropagates through at most the last eight iterations, following the recipe of Huginn [348]. However, truncation also lowers the measured value of a recurrence (Section 7). It may also have a benefit beyond memory. One study finds that truncation alone shapes the fixed points that make cache sharing work [219] (Section 8.1). With fixed-depth training and full backpropagation, sharing "drops its GSM8K accuracy from 50.6 to 21.2" at 1.6B [219]. A retrofit study also reports that truncated backpropagation "incurs higher computational cost while plateauing at a substantially higher" loss than *detached deep supervision* with random depths [389]. *Per-iteration losses* are used to train adaptive exits [343] but are not sufficient for stability [440]. For fixed-depth accuracy, a post-loop readout can beat per-iteration readouts [147].

One-step or last-step gradients are typical of the recursive reasoning models in Section 11.2. They are derived from Jacobian-free backpropagation for implicit networks, which avoids solving the implicit-function-theorem system and trains with fixed memory [163]. Evidence from implicit fixed-point models, such as deep equilibrium (DEQ) models, suggests that one-step gradients do not scale. In an ImageNet diffusion model, training with the one-step gradient gives a Fréchet inception distance (FID) of 567.6, compared with 43.2 for stochastic multi-step truncation. The authors report that the "1-step gradient, the most common method for training DEQ [5] models, struggles to optimize models on the large-scale ImageNet dataset" [32]. In contrast, a 5.3M-parameter optical-flow model trains well with a one-step gradient plus a sparse fixed-point correction [31]. Other methods reduce the memory cost of backpropagation directly. One compresses the unrolled activations of a TRM to a low-rank form during backpropagation [61]. Another trains recurrent vision models over many processing steps with memory that does not grow with the number of steps [323]. No study in the corpus compares the memory and wall-clock cost of full, truncated and one-step gradients at matched quality on a language model.

6.4 Building looped models from existing checkpoints

Retrofitting with training. Converting a pretrained dense model into a looped one costs a small fraction of pretraining. A Qwen3-1.7B conversion that uses a single learned input-mixing scalar costs about 0.5% of the source model's pretraining FLOPs [343]. Another conversion loops four middle layers, selected by an angular-distance criterion, and costs a re-run of a 50B-token mid-training stage [267]. For the same recurrent architecture, a pretrained initialization is far more efficient than random initialization [348]. Per-iteration parameters are optional. Shared weights and per-step low-rank adaptation (LoRA) adapters perform within two points of each other [343], and a small LoRA can match a full-block retrofit [439]. Only one conversion above a billion parameters in the corpus is matched in both parameters and compute. It loops eight intermediate layers of a mixture-of-experts model during continual training and "shows substantial gains of 5.5 points overall" [226]. However, the compute for looping is freed by token-to-concept compression, so the two effects are not separated [226]. Other conversions differ in their supervision or in what they convert. One fine-tuning method makes a pretrained language model recursive and supervises each iteration with teachers [298]. Another distills pretrained language models into a compact recursive student that shares one layer across depth [381]. Conversion can also compress a model. One method prunes blocks of a large language model and replaces them with weight-shared unpruned blocks plus block-specific low-rank adapters [455]. In vision, a method deepens pretrained vision Transformers by inserting weight-sharing layers with small adapters [202].

Which layers to loop. It is unresolved which layers should be looped in a retrofit. Studies favor middle layers [76, 267, 339, 389], later layers with a prelude and coda [348], or nearly the whole stack [343]. Another study finds placement nearly irrelevant on synthetic tasks [91]. The same question is open for models trained from scratch (Section 5.1).

Training-free looping. Repeating layers of a frozen model is the cheapest experiment and the least reliable one. Naive repetition "degrades performance almost universally" [76], but damped sub-stepping recovers non-negative results in 87% of model–benchmark cells [76]. The result also depends on which unit is repeated. In frozen language models, "sharing weights among middle layers is catastrophic" [463] when a single center layer replaces the middle block, but looping a parallelized middle block helps [463]. A simple training-free heuristic is to re-apply the layer at which the hidden-state norm peaks. However, the norm "grows rapidly as the loop count increases" [586], and the authors conclude that "looping up to 3 times is relatively safe" [586]. Frozen models tolerate reuse that is shallow, local and damped, but deeper recurrence requires training.

Layer programs and selective reuse. A more flexible form of training-free looping chooses which layers of a frozen language model to skip or repeat. The choice can come from a per-sample search [313] or from a learned routing policy [41]. Oracle searches over such layer programs show large headroom, but deployable routers recover little of it. One reproduction of a learned program router [314] finds that the router's "top-1 decode is always the literal identity program" [526]. Other training-free methods restrict reuse to a few layers or a few tokens. One decoding procedure re-applies a frozen model's top layers at high-uncertainty tokens until convergence [45]. In vision, one method replays a short window of frozen vision Transformer blocks, gated by Gram consistency [86], and another repeats chosen decoder-layer blocks of a frozen vision-language model [452].

Table 10: A training recipe for looped language models, with the strength of the supporting evidence.

Choice	Setting supported by the corpus	Evidence
Stages	one pretraining run plus response-only mid-training	single controlled study at 1.4B [343]
Loop count from scratch	fixed T=4 with a trained exit gate	1.4B, consistent with Ouro [343]
Exit-gate training	use warmup and an entropy/KL coefficient ≥ 0.10, and monitor the exit distribution	clear sweep, one paper [343]
Normalization	pre-norm family plus final norm before the head	strong for trainability [343], unresolved for extrapolation [556]
Residual scaling	scale with loop count ($1/T$ or DeepNorm-style)	0.3–0.5B, rules differ [308, 506]
Random loop counts	sample during training (Poisson log-normal or log-normal)	helps retention but is not sufficient alone [348, 556]
Injection	optional from scratch (whole stack), standard in retrofits	contested [343, 348, 439]
Gradients	full at small T, truncated or detached deep supervision at large T	contested [348, 389]
Post-training RL	credit over all loops, mixed solve rates	$\leq$2.6B [343, 530]
Beyond the horizon	train stability in, and do not assume that more loops help	strong negative default [556, 608]

6.5 Post-training

Reinforcement learning (RL) that rewards only the output of the final iteration does worse than methods that expose intermediate iterations to the reward. On Ouro-1.4B and 2.6B, distributing the policy gradient over the log-probabilities of every loop improves math accuracy over group relative policy optimization (GRPO) applied to the terminal loop. The memory of this method "scales linearly with the number of loops" [530]. Per-loop rewards with a time penalty reduce the average number of iterations and improve results on hard tokens [477]. In two runs, whether RL helped depended on the rollout solve rate, together with a learning rate large enough to move the policy [343]. Parameter-efficient fine-tuning of a looped model is more effective at later loops, and dropping adapter applications per loop with inverse-survival rescaling narrows this bias [608]. All of this evidence is at or below 2.6B parameters. Rewards can also be used without RL training. A latent reward model trained on the latent thoughts of Huginn-3.5B selects better trajectories by rejection sampling [129].

Table 10 combines the findings of this section into a training recipe and records the strength of the evidence for each choice. Most rows rest on a single study or on models with at most a few billion parameters, and the choices of injection and gradient method remain contested.

Takeaways

- A 1.4B looped model can be trained competitively with 310B tokens. For reasoning, mid-training matters more than pretraining length.
- The main training risks are exit-gate collapse that does not show in the loss, residual growth from tied updates and norm drift beyond the training horizon. Each risk has a fix, but no fix has been validated above a few billion parameters.
- Converting a dense checkpoint can cost as little as about 0.5% of its pretraining compute, and it requires anchoring each iteration to the input.
- At matched parameters, a looped model beats its dense counterpart. At matched inference compute, it ties on math and trails on knowledge benchmarks (Figure 9).

7 Scaling

Section 6 described how looped models are trained. Scaling studies ask how much a recurrence is worth when resources are allocated optimally. Because looping trades parameters for compute (Section 3.4), the answer depends on which resource is held fixed. This section reviews the scaling laws proposed for looped models and what they imply about test-time depth. It then reviews the evidence for the hypothesis that loops help reasoning but not knowledge.

7.1 Scaling laws and what they hold fixed

Table 11 lists the scaling laws in the corpus and the resource that each one holds fixed. Each law was fitted under a different matching rule, and their conclusions differ accordingly.

Matched compute and depth. At matched compute and depth, a recurrence is worth less than a fresh block. The most careful dense study fixes effective depth and training compute across recurrence counts $r \in \{1, 2, 4, 8\}$, so unique parameters shrink as r grows. It fits the effective-parameter law of Eq. (9) over 116 runs. The fit gives $\varphi = 0.46$ with a 95% confidence interval of [0.41, 0.53], so "replacing unique blocks with shared recurrences increases validation loss at matched training compute". At r=4, "a 410M looped model performs on par with a 580M non-looped model, but incurs the training cost of a 1B non-looped one" [433]. The same study also uses φ as a diagnostic. Truncated backpropagation lowers it to 0.38, and loop-boundary hyperconnections raise it to 0.65. Two other studies agree in the dense setting. In one study, looping the same architecture "seems thus to strictly reduce expressivity" at matched compute and effective depth [288]. Ouro's own small-scale pretraining sweep also finds that, on general benchmarks, a standard model with the multiplied number of layers beats the looped model at every size [604]. On synthetic knowledge-manipulation tasks, however, looped models match or beat such a model [604].

Saturation. The loop scaling laws find that the gain from more recurrences saturates. They are fitted over recurrence counts up to eight, with sixteen held out. They also find that linear and power-law mappings over-predict and that the effective-parameter gain is bounded. For a dense model, the asymptote of this gain is about 0.37 of the shared block's parameters, and sparsity raises it [84]. An earlier scaling study of ten architectures is consistent with saturation. In it, the weight-shared Universal Transformer scales poorly, and adding recurrent operations raises FLOPs without improving performance [481]. Figure 6 plots the three mappings. They agree for one or two recurrences and diverge sharply beyond four. It is an open question which mapping holds at large r.

Table 11: Scaling laws for looped models and the resource each holds fixed.

Study	Held fixed	Finding
Iso-depth law [433]	training compute, effective depth (unique parameters shrink)	recurrence worth $r^{0.46}$ fresh blocks, and looping raises loss
Parcae [397]	unique parameters	looping and data should grow together, and the looped isoFLOP frontier beats fixed depth
Loop scaling laws [84]	active parameters and data (sweep)	gain is bounded and saturates, and sparsity raises the bound
SMELT [507]	per-token FLOPs, total parameters, KV cache, data (MoE)	looping saves 6.8–10.0% of training compute at 10^{20} FLOPs and 14.7–18.0% at 10^{21}
Sparse layers [288]	training compute, effective depth	dense looping worse than standard, and looped MoE better than dense only through sparsity
Growth and tying [93]	training compute, each arm compute-optimal	tying costs a constant 1.06–1.08× compute (1.14–1.16× with growth)
Ouro, small-scale pretraining sweep [604]	effective depth and data (hence training compute)	on general benchmarks, the standard model with the multiplied layers wins at every size from 53M to 1.36B, and the gap generally shrinks with size at low loop counts; on synthetic manipulation tasks, looped models match or beat it

Parameter-limited settings. Looping helps when parameters are the limiting resource. If unique parameters are fixed, for example by a memory budget, then spending extra compute on looping is better than spending it on more tokens. Parcae's looped isoFLOP frontier outperforms fixed-depth models trained on the extra data, and compute-optimal looping and data should grow together [397]. One study matches total parameters, per-token FLOPs and KV cache in mixture-of-experts models. In this setting, looping the middle half of the network twice saves 6.8–18.0% of training compute, depending on the budget [507]. When data are the limiting resource, looping beats adding parameters or epochs at BabyLM scale [193]. With repeated data, the optimal loop count rises with compute [93]. Section 15.3 combines these results with the matched-compute evidence above into one reading.

7.2 Test-time depth

Recurrent depth is meant to let a model think longer at inference by running more iterations. At language-model scale, the evidence shows that this works up to the training depth but not beyond it. Ouro has a trained depth of four recurrent steps (Section 6.1) and shows "a moderate degradation" on base benchmarks at five to eight steps [604]. Parcae is trained with randomized depth, and its performance plateaus without degrading. Its test-time law is a saturating exponential whose floor is set by the training depth [397]. A machine translation study on Ouro finds that quality improves over the first few iterations and then saturates quickly [80]. In tool calling, Ouro-2.6B is near saturation after three iterations, while Ouro-1.4B keeps gaining on multi-call tasks [396]. No paper in the corpus shows gains on language-model benchmarks from running more loops than were used in training. A recent looped decision model is consistent with this, since its accuracy falls from 72.0% at the trained eight loops to 70.1% at sixteen [164]. The strong extrapolation results come from synthetic algorithmic tasks (Section 9) and from randomized-depth models whose gains

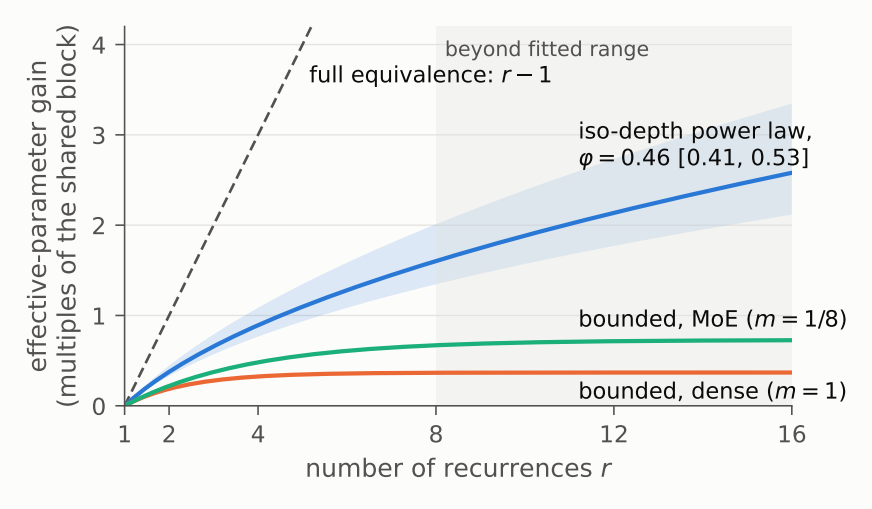


Figure 6: How much is a recurrence worth? Effective-parameter gain from looping (in multiples of the shared block's parameters) under three fitted mappings: the iso-depth power law (φ=0.46, shaded 95% interval) [433] and the bounded mapping for dense (m=1) and sparse (m=1/8) models [84]. The fits come from different experimental designs, and their data end at eight recurrences (the gray region is extrapolation).

saturate near the training mean (Section 5.5).

Figure 7 collects depth curves from one language model and five systems in other modalities. When one checkpoint is run at several loop counts, the curves rise steeply over the first few loops and then flatten at or before the training depth. The loss of a randomized-depth language model stops improving at its training mean of eight loops [331]. A robot policy trained with a mean of 32 iterations peaks at 24 [488]. A multimodal model trained at 32 does slightly worse at 64 [591]. A vision-language model trained with two high-level cycles collapses when run with four [401]. When a separate model is trained at each loop count, quality peaks at an intermediate loop count (four in an embedding model, five in a speech encoder) and declines beyond it [417, 508]. In contrast, one study finds that out-of-distribution performance can keep improving as the training-time loop count grows, even after in-distribution performance has saturated [548].

7.3 Knowledge versus reasoning

A widely repeated claim is that loops improve how a model manipulates knowledge but not how much knowledge it stores. The storage part of this claim is supported by controlled studies at small scale. On synthetic biographies, looped and non-looped models with the same parameter count both store about two bits per parameter, so "looping does not increase knowledge capacity" [604]. A second study finds the same result with a different capacity probe [605]. In general pretraining at matched compute and depth, the gap in parametric knowledge for looped models grows with the recurrence count [433]. However, knowledge does not suffer when total parameters are *not* reduced. With total parameters matched, looping also improves the knowledge validation category [507]. The lower knowledge storage is therefore explained by the smaller number of parameters, and looping itself does not cause it.

The reasoning part of the claim depends on the comparison regime. At matched parameters and on synthetic tasks trained directly, looped models beat their non-looped counterparts [426, 604, 605] and often match deeper ones [426, 604]. At matched training compute in general pretraining, looping lowers loss and improves downstream accuracy broadly in a mixture-of-experts setting [507], and

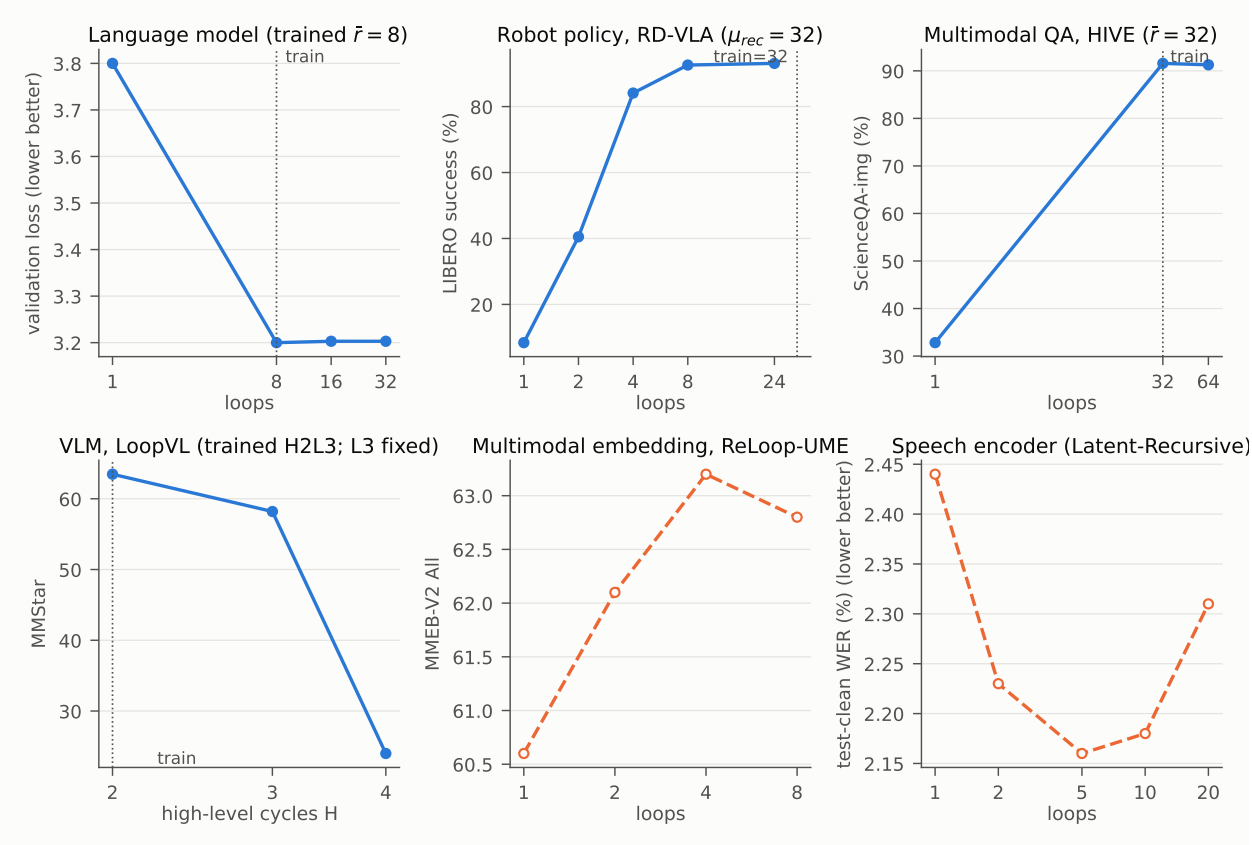


Figure 7: Quality as a function of loop count in one randomized-depth language model and five systems in other modalities. Blue filled markers: one checkpoint run at different loop counts. Dotted line: training loop count (for randomized depth, the mean-setting parameter of the sampling distribution). Orange open markers: a separate model trained at each loop count. Sources, left to right and top to bottom: Logan [331], Qian et al. [401], Rolland et al. [417], Tur et al. [488], Wang et al. [508], Zhang et al. [591]. The script `scripts/fig_testtime_loops.py` checks every plotted value against a passage copied from the source.

one looped mixture-of-experts model matches a non-looped one with about twice its parameters on reasoning benchmarks [84]. It does not appear in small dense models, where reasoning primitives favor the non-looped model "at nearly every budget" [433]. It is unresolved whether this difference is due to scale, sparsity or the choice of metric.

7.4 How many loops?

Reported optimal loop counts range from two to tens, and the comparison regime explains most of the spread. When per-token FLOPs are matched, more loops force a thinner model, and two loops are optimal [507]. Several large releases also choose two loops [169, 368, 550]. When only training compute is matched and active parameters are fixed, the compute-optimal count rises with compute and is five in the largest matched-compute experiment, a trillion-token run [84]. When unique parameters or data are fixed, the optimal count grows with compute [93, 397]. Large "depth" counts in the tens appear only on synthetic tasks.

Takeaways

- In dense models at matched compute and effective depth, a recurrence is worth about $r^{0.46}$ fresh blocks, and looping does worse than the non-looped model.
- Looping does better when parameters are the limiting resource: fixed memory, limited data, or sparse experts that keep the total parameter count.
- Gains from more recurrences saturate, and at language-model scale, running more loops than were trained does not help.
- Knowledge storage depends on the number of stored parameters and not on the number of loops. The reasoning advantage at matched compute is shown only for sparse models.

8 Inference and Systems

Section 7 asked what looping is worth at training time. This section turns to the cost of serving a looped model, which Eqs. (6) and (7) predict. Compute per token grows with the loop count, and so does the key–value (KV) cache unless the cache is shared. Both costs also appear in measurements of real systems. Ouro-2.6B with four loops needs 48 GiB of KV cache at a 32K prompt, while an 8B standard model needs 4 GiB [555]. Four loops over 24 layers carry "the KV cache of a 96-layer decoder" [369]. At batch size one, a looped decode step takes about 2.3, 3.4 and 5.7 times as long as a standard step at two, three and five recursions [360]. Work on systems addresses these costs in four areas: memory, latency, adaptive depth and numerics. Table 12 summarizes the main methods and how each result was measured.

8.1 The KV cache

Redundancy along the loop axis. Several observations suggest that the caches of the different iterations carry largely duplicated information. The KV cache has a low effective rank along the loop axis [369], and residuals between adjacent loops are small and quantize well [555]. Keys stabilize before values [301, 369], and attention routing stabilizes before hidden states [496].

Sharing caches. Five families of methods use this redundancy to reduce memory. The Looped Prediction Transformer (LPT) is trained from scratch and lets later recursions read only the cache of the first recursion plus a short local window. It uses 2.0–4.7× less KV than a full-cache looped model and also improves perplexity, so that "sharing memory does not cost quality and instead improves it" [360]. The Parallel Loop Transformer (PLT) shares the cache of the first loop with later loops and adds gated sliding-window attention (SWA). This halves the KV cache at two loops without a loss in accuracy [531]. A related architecture computes keys and values once from a prelude and reuses them in every iteration of the core [64]. In a retrofit with fine-tuning, the Memory-Efficient Looped Transformer (MELT) keeps a single gated KV entry per token and layer, which reduces Ouro's cache by a factor of 3.98 [493]. Post-training codecs such as Looped Latent Attention (LLA) compress the trajectory along the loop axis into low-rank latents and keep GSM8K accuracy at a 4× budget [369]. Training-free sparsification with 4-bit integer (INT4) cross-loop residuals, in FlashLoop, reduces peak KV memory by 5.18–6.06× [555].

Why sharing can improve quality. A result along the sequence axis suggests why sharing the cache of the first loop can improve quality instead of only preserving it. This is our inference, because the paper does not discuss looped models. State-Prediction Separation runs each token

through a shared network twice. Only the KV of the first pass persists, and the KV of the second pass is temporary and limited to a local window. At 1.6B parameters, it "matches the validation loss of a standard Transformer trained on 47B tokens" with 2.6× fewer tokens. Keeping the KV of both passes does worse, so "the gain is not capacity-based" [359]. LPT and PLT use the same structure, with a persistent cache from the first pass and a temporary local cache for later passes.

Huginn versus Ouro. The response of an existing checkpoint to *untrained* cache sharing depends on how it was trained, and Huginn and Ouro respond in opposite ways. Huginn was trained with randomized depth and a prelude–core–coda layout, and it tolerates sharing down to a single slot [176, 196, 434]. Ouro collapses under such sharing, and single-slot sharing drops its GSM8K accuracy from 78.09% to 0.23% [434]. Other studies report similar failures of Ouro under cache sharing [196, 369]. The Ouro paper itself reports that sharing during decoding loses little and cuts cache memory by 4×, while sharing during prefill degrades GSM8K by more than 10 points [604]. A study that could not reproduce near-lossless sharing for Ouro suggests that this split between prefill and decoding explains the difference [434]. Two papers posted at the snapshot date add further evidence. Training-free sharing of a single loop's cache drops the MATH500 accuracy of Ouro-1.4B from 75.9 to 45.9 [90]. Hybrid latent attention, which adds small trained projections to the frozen model, instead shrinks the cache by 10.7× per token and retains 97.5 to 99.6% of the original accuracy [90]. Sharing can also be trained in from scratch. At 1.6B, a learned depth prior with a 3× smaller cache "matches the downstream average of fixed-depth training with the full cache" [219]. In the same study, fixed-depth training with full backpropagation through time does not support sharing [219] (Section 6.3). The evidence therefore supports two conclusions. Untrained sharing works for Huginn but not for Ouro once the prompt cache is shared as well. Sharing works when it is part of training.

Memory versus speed. Smaller caches do not make decoding faster by themselves, and they help throughput only by allowing larger batches. LPT fits 2.0–4.8× more sequences, but it is 3–8% slower than a full-cache looped model at equal batch size [360]. The LLA codec raises the maximum batch from 32 to 768, but its latent path decodes far more slowly than the uncompressed path [369]. MELT, a retrofitted model with a shared cache, decodes at 8.5 tokens per second at batch size one, compared with 13.5 for the unmodified Ouro-1.4B-Thinking [493].

8.2 The latency of sequential loops

Cross-loop parallelism (trained). PLT offsets each loop by one token, so that loop l of token i runs in the same pass as loop $l{+}1$ of token $i{-}1$. With this shift, a two-loop model decodes in 4.9 ms at batch size four, against 4.8 ms for the non-looped baseline. It also cuts the latency of naive looping by 47% at batch sizes 32 and 64 [531]. Earlier work pipelined tokens through cyclically shared decoder layers in a similar way [212]. Work on staggered stacks reports only simulated latency [111].

Training-free parallel and speculative decoding. The early iterations of a looped model are well suited as draft models for its later iterations, because predictions agree strongly after a few iterations [98, 196]. Several training-free decoding methods build on this observation. A diffusion-forcing sampler accelerates Huginn by 4.36–4.81× in tokens per second with small changes in accuracy [176]. Lossless self-speculation (LoopSpec) reaches up to 6.83× decode speedup on models with 32 recurrences and up to 3.36× on Ouro [98]. Wavefront decoding (WFD) reaches end-to-end speedups of 2.42× on Ouro-2.6B and 3.54× on Huginn [196], and depth-asynchronous

drafting reports 4.00–6.96× [301]. None of these methods reduces FLOPs. Instead, all of them use compute that would otherwise be idle in memory-bound decoding to reduce the number of sequential steps.

Batch size. Every training-free latency result above is reported at batch size one. The one paper that varies batch size finds that the speedups "decrease monotonically with b" [196], where b is the batch size. A plain looped self-speculation scheme is also not automatically the best accelerator, since a generic token-recycling method outperforms it on Huginn [301]. Several speedups are measured against unoptimized eager baselines, so absolute throughputs are not comparable across papers. The same caveat appears in neighboring model families. Solving a diffusion sampling chain as one fixed point is faster for single images, but "on full-batch inference with larger batches, sequential sampling might outperform DEQ-DDIM" [394]. A diffusion language model reports "1.5 and 1.8 times higher throughput" than an autoregressive baseline, and this throughput is measured at batch size one [376].

8.3 Serving adaptive depth

Per-token adaptive depth (Section 5.5) produces ragged batches, in which tokens that have exited leave the hardware idle. The only end-to-end study of batched serving is continuous depth batching (CDB). It runs prefill, prelude, core and coda as separate queues and refills the slots of exited tokens. On one H100, it reaches 1.32–1.53× the throughput of full-depth continuous batching for Ouro and 1.30–1.37× for Huginn. These gains are close to the bound once prefill time is accounted for [434]. Two lessons from this study apply more generally. First, bounds based on decode FLOPs overestimate speedups, because early exit does not affect prefill. Second, "fully looped architectures are best suited" to depth batching, since unshared boundary layers fragment the schedule [434]. An earlier dissertation on early exit makes a related argument, that deep parameter sharing eases the batching problems of per-token early exit [23].

8.4 Quantization and numerics

Work on quantization agrees on several practical rules. Eight-bit quantization causes little or no degradation in the looped models tested [227, 240, 307]. Per-tensor four-bit quantization is catastrophic for some recursive models, and finer scale granularity fixes this [227, 240]. Calibration must also see later iterations. In Trajectory GPTQ, accumulating calibration statistics across the recurrence restores Huginn to bf16-level GSM8K accuracy, while one-step calibration fails [307]. With 4-bit weights and activations (W4A4), loop-aware calibration improves the quantization of looped models, including Ouro, by a large margin [149]. The strongest static baselines reach near-zero LAMBADA accuracy on LoopFormer and Parcae [149].

The literature disagrees on *why* low-bit looped models fail. Several papers attribute the failure to error that accumulates across loops [149, 227, 240]. A study of 33 models argues instead that a fixed rounding error only shifts the fixed point in loops that settle. It also argues that the accumulation explanation "holds only for loops that never settle", and that collapsed models can be recovered by finishing with a few 8-bit iterations [268]. A certified small-scale analogue supports the second view and also reconciles the two views. In a monotone equilibrium network, quantization that stays within the monotonicity margin keeps the solver convergent with a bounded shift of the fixed point (98.24% versus 98.22% accuracy at 8 bits). In contrast, 3-bit and 4-bit weights exceed the margin and diverge [303]. Accumulation describes what happens once the margin is lost. The best quantization configuration can also change with the loop count, so calibration should

Table 12: Inference methods for looped models and how their main results were measured. BS1: batch size one. SWA: sliding-window attention. PTQ: post-training quantization.

Method	Mechanism	Training	Reported result	Measured as
LPT [360]	later loops read first loop's KV + window	from scratch	2.0–4.7× less KV; lower perplexity	H100; 3–8% slower at equal batch
PLT [531]	one-token cross-loop shift; shared KV + gated SWA	from scratch	two loops at near non-looped latency	wall-clock, BS 4–64
MELT [493]	single gated KV per layer	retrofit	KV ×3.98 smaller	slower decode at BS1
LLA [369]	low-rank loop-axis latents	codec only	max batch 32→768	batch capacity only
FlashLoop [555]	sparse loops, INT4 KV residuals	training-free	1.52–1.64× end-to-end	whole-benchmark wall-clock
Diffusion-forcing [176]	wavefront refinement	training-free	4.36–4.81× (Huginn)	BS1; approximate
LoopSpec [98]	pipelined self-speculation	training-free	up to 6.83×; 3.36× Ouro	BS1; lossless
WFD [196]	mixed-depth wavefront + verification	training-free	2.42× Ouro; 3.54× Huginn	BS1; declines with batch
CDB [434]	depth batching with refill	training-free	1.30–1.53× vs. continuous batching	H100 serving
Trajectory GPTQ [307]	Hessians over recurrence	PTQ	bf16-level GSM8K for Huginn	real W4A16 kernel
LoopCD [327]	contrast with first iteration	training-free	AIME24 61.88→73.33	analytic FLOPs

happen at the depth used in deployment. On Ouro-1.4B under W4A4, depth-aware calibration reduces LAMBADA perplexity by 36.5% relative to an adapted SpinQuant baseline [310]. Wall-clock evidence for quantized looped models is still scarce, because most studies report accuracy and memory under simulated quantization.

8.5 Decoding with early iterations

Earlier iterations have done less computation, so a looped model contains its own weak and strong predictors. Two concurrent papers use this property for contrastive decoding against an early iteration. One improves Huginn's HumanEval pass@1 from 24.39 to 30.49 with almost no loss in throughput [562]. The other, LoopCD, raises the AIME 2024 pass@1 of Ouro-2.6B-Thinking from 61.88 to 73.33. It also reports that half the loops with guidance can match full depth without guidance, but the authors state that this saving is analytic and was not measured [327]. The two papers disagree on the best reference depth. A different way to save iterations treats the shared layers as steps of a neural ordinary differential equation. It runs fewer iterations of parameter-shared language models with larger steps [81].

Takeaways

- Looping increases KV memory and latency in proportion to the loop count. Current methods share or compress memory and run loops in parallel to reduce latency, but none removes the extra FLOPs.
- Memory savings increase throughput only through larger batches. Training-free latency gains are reported at batch size one and shrink as batch size grows.
- A checkpoint's tolerance of cache sharing depends on its training recipe (Huginn tolerates it, Ouro does not).
- Eight-bit quantization is safe, and four-bit quantization needs fine-grained scales and calibration across iterations.

9 Theory

The preceding sections review empirical results on design, training, scaling and inference. Theory asks three questions about looped models: what they can express, how they compare with chain of thought, and what training actually makes them compute. The answers are more precise than for most architectures, because looping corresponds closely to the notion of circuit depth. The main caveat, which we discuss at the end of the section, is that almost none of these results has been tested on looped language models.

9.1 Expressivity: loop count as circuit depth

Universality. Looped Transformers with hand-set weights are universal computers. A looped encoder with 9 layers and 2 heads executes programs in a Turing-complete one-instruction language, one instruction per iteration. Its depth does not depend on program length, because the loop carries the program counter [180]. Engineering extensions compile C programs into such machines [489]. Graph algorithms such as Dijkstra, BFS and DFS can be simulated with constant width on graphs of any size, up to the limits set by precision [119]. Constant-width looped Transformers also simulate hypergraph algorithms such as Helly's algorithm, with theoretical guarantees [224]. Universality does not require attention, since looped ReLU networks also emulate the same instruction set [316]. It also holds with one fixed set of weights for all targets, where only the order or the number of applications changes. Two frozen residual attention blocks, applied in a task-chosen order, can interpolate any sequences [344], and one fixed recurrent network approximates any continuous function by running longer [1]. Universality therefore cannot by itself explain why looping helps language models.

Depth efficiency. Looped models can reach near-optimal depth with few parameters. A one-layer Transformer looped $\lceil \log_2 n \rceil$ times composes n elements of a finite group, which matches the best known depth for non-looped models. Looping one layer $\lfloor \log_2 p \rfloor + 2$ times solves p-hop induction, which matches a depth lower bound for standard Transformers [426]. On this view, many reasoning problems need depth and not parameters. Looped Transformers with a logarithmic number of repetitions recognize regular languages and solve graph connectivity. For these problems, the analysis concludes that "depth scaling is more efficient than scaling width or chain-of-thought steps" [350], but its experiments use untied models. Approximation analyses reach a related conclusion. A statistical analysis shows that extra loop iterations improve approximation accuracy without adding parameters [485]. An approximation-rate analysis of single-layer looped Transformers motivates per-loop scaling conditioned on timestep encodings [541], which connects to the debate on step conditioning (Section 5.2).

Table 13: Looped models versus chain of thought (CoT). The stronger of the two depends on iterations, persistent memory and stochasticity.

Setting	Result	Source
Deterministic CoT, sequence-state loop with padding	m CoT steps simulated by m loops over the input plus m dummy positions	[426]
Equal polylog budgets, parallelizable problems	CoT[$\log^k n$] $\subseteq \mathsf{TC}^{k-1}$ while $\log^k n$ loops reach TC^k; strict under standard assumptions	[543]
Compressed loop (state independent of input length) vs. polynomial CoT	loops with polylog state cannot decide P-complete problems that polynomial CoT can (if P $\not\subseteq$ polylog space)	[582]
Randomized approximate tasks	CoT with sampling achieves approximate counting that no deterministic loop does (if FPTAS $\neq$ FPRAS)	[543]

Exact complexity classes. Padded looped Transformers have exact characterizations in which the loop count plays the role of circuit depth and padding plays the role of circuit width. With polynomial padding, $O(\log^d n)$ loops recognize exactly FO-uniform TC^d, and polylogarithmic loops give exactly NC [351]. These characterizations still hold when hard attention is replaced by softmax attention, provided that precision times width is logarithmic. Precision and depth decide the class, and attention type and width do not. Constant precision gives AC^d, and growing precision gives TC^d [469]. Related results extend these characterizations. Masked diffusion models are characterized as padded looped Transformers [468], and the padding and loops needed to recognize context-free languages are bounded [237]. A different kind of result links looped networks to description length. It proves that the minimal weight norm of a fixed-precision looped network equals the Kolmogorov complexity of its output, up to a logarithmic factor [366].

9.2 Looped models versus chain of thought

Statements in the literature about whether loops or chain of thought are more powerful appear to conflict. They become consistent once three resources are fixed: the number of iterations, the amount of persistent mutable memory, and whether decoding is stochastic (Table 13).

Circuit size and circuit depth. The part of this correspondence that concerns circuit size is older than looped models. With T steps of chain of thought, constant-depth Transformers with constant-bit precision and logarithmic embedding size "can solve any problem solvable by boolean circuits of size T" [311]. For parity, multiplication, median and graph reachability, "CoT lengths need to scale at least linearly in the size of the original problem" [17]. The same authors suggest that "Logarithmic growth of the depth may be one way to overcome these" [17]. Loops win at equal small step budgets on parallelizable problems, because chain-of-thought steps correspond to circuit *size* while loop iterations correspond to circuit *depth* [543]. An earlier version of the same analysis states this separation in terms of computation graphs, which looped Transformers evaluate in parallel [542]. With polynomially many steps, chain of thought wins against loops whose persistent state does not grow with the input. As the analysis puts it, "Running the loop longer adds computation but does not by itself create a growing scratchpad" [582]. Chain of thought also wins on randomized approximate tasks, which the paper describes as "the first formal separation in favor of CoT" [543].

Memory and padding. The positive result that loops can simulate chain of thought requires padding positions, one per simulated step [426]. Practical looped language models carry state at every position (a "sequence-state" loop) but do not append padding. They therefore lie between these regimes, as the memory-budget analysis states explicitly [582]. Any statement that looped models "can simulate chain of thought" should name the memory regime and the padding requirement.

9.3 Looped models as iterative algorithms

In linear and convex in-context settings, looping is both expressive and learnable, and the learned computation is a convergent iterative algorithm. For in-context linear regression with Gaussian data, every global minimizer of a linear looped Transformer with L loops implements L steps of preconditioned gradient descent. Gradient flow in this setting converges despite non-convexity [170]. Linear looped Transformers can also implement multi-step gradient descent in context with far fewer examples than earlier constructions required [71]. For in-context classification with softmax attention, L layers perform exactly L steps of normalized gradient descent. Looping a trained layer gives an out-of-distribution error bound that decreases with the number of loops [580]. Gradient descent on a looped linear-attention model with normalization learns the power method with one iteration per loop, and it cannot learn this method without normalization [533]. Looping also adds capabilities that a single layer lacks. Looped linear attention can use unlabeled in-context demonstrations, which one-layer attention cannot [309]. Looped minimizers are also more robust to distribution shift than multilayer ones. In one analysis, the in-context regression of multilayer minimizers is "not robust to even distributional shifts as small as $O(e^{-L})$" [171]. The study of preconditioned gradient descent above also observes that running more loops than were used in training keeps improving the test loss, and it leaves the explanation open [170]. It is not known whether pretrained looped language models implement such optimizers internally.

9.4 Length generalization and algorithmic extrapolation

Ingredients of extrapolation. On synthetic algorithmic tasks, looped models trained on small instances solve larger ones by running more iterations. Independent studies agree on a small set of ingredients that make this work. One is re-injecting the input at every iteration [38, 146]. Recurrent solvers outside Transformers use related mechanisms. A recurrent convolutional solver with input recall extrapolates from small to larger problems [139]. Recurrent graph networks trained on small graphs extrapolate to larger ones with the help of skip connections [189]. Another ingredient is coupling the loop count to problem size in training and adapting it at test time. With this ingredient, a looped model trained on parity with up to 20 digits generalizes near-perfectly beyond 40 digits [146]. A third ingredient is randomizing the loop count during training [38, 275]. Settling dynamics matter as well, and a terminal fixed-point objective has a causal role in depth safety, meaning stable behavior when the model runs more iterations than in training [495]. The remaining ingredients are discretizing and re-embedding between loops [16], and supervising only the final answer instead of the answer at every step [74]. Under final-answer supervision, two looped-Transformer training schemes learn different mechanisms for length generalization on composition tasks [315]. Per-iteration supervision is still used for other goals. One looped encoder is trained with supervision at each iteration aligned to chain-of-thought steps, and it is then used to fine-tune autoregressive models [567].

Shared versus unshared depth. At matched effective depth and training compute, an 8-layer block looped twice has half the out-of-distribution error of a 16-layer model on addition [347].

Looped models also reach systematic generalization on unseen compositions, which iso-compute standard models never reach [266]. Earlier work on weight-tied models points the same way. On a synthetic chain task, weight-tied ALBERT extrapolates to longer chains and BERT does not [589]. A Universal Transformer with local attention and adaptive halting learns multi-digit addition and extrapolates [103]. Hyper-UT combines hypernetworks with the adaptive depth of the Universal Transformer to generalize to more computation steps [2]. Among other configuration choices, Universal Transformer variants improve systematic generalization on CFQ and PCFG, although they do not help on all of the benchmarks studied [106]. The same goal of training on short inputs and testing on long ones appears in streaming tasks. There, a recurrent core applies several weight-shared latent updates per observation and carries its state across stream steps, which improves out-of-distribution generalization over recurrent, state-space and Transformer baselines [471].

Choice of test tasks. A formal framework for length generalization explains why copying and multi-digit addition are the standard test beds. Neither task can be expressed by the idealized fixed-depth Transformers of the framework. Any solution must therefore let depth, heads, parameter norms, MLP width or precision grow with the input length [222]. Looping is the cheapest way to let depth grow. On a cellular-automaton benchmark, extending effective depth through recurrence helps multi-step reasoning, but the gains remain bounded [414]. The framework bounds depth in its own regularizer, so it does not analyze input-dependent loop counts directly. No study tests whether a looped language model with input-dependent loops removes these fixed-depth barriers on the same tasks.

Robustness of the evidence. The evidence for extrapolation is unstable across seeds and tasks. Across seeds, looped Transformers show out-of-distribution standard deviations of 0.087–0.122, against at most 0.009 for fixed-depth models [275]. On Boolean, arithmetic and ListOps tasks, a standard looped Transformer trained with the length-generalization recipe "still collapses toward chance out of distribution" [228]. Maze-solving extrapolators fail when difficulty increases along new axes [264]. Looped Transformers trained end to end extrapolate to a few times the training length, which is far below the extrapolation factor of fixed-rule systems with step supervision [524, 588].

Expressivity versus learnability

Three findings separate what looped models *can* compute from what training makes them compute. End-to-end training picks the shallowest program that fits, instead of the length-invariant one [441]. Weight-tied loops learn a serial computation frontier whose speed is set by the training loop budget. They do not learn the logarithmic-depth scans that theory permits, and learning difficulty follows operator size and not complexity class [588]. Hand-constructed simulators lie in ill-conditioned basins that training rarely reaches [119].

9.5 The theory–practice gap

Two theoretical predictions have been tested on language models with a billion or more parameters. First, the claim that reasoning requires depth and not parameters was tested on 24-layer models trained on 250B tokens. There, a 12-layer block looped twice beats the 24-layer model on math word problems but closes only part of the perplexity gap [426]. Second, diagnostics of settling dynamics were applied to Huginn. They found that across thousands of per-token latent trajectories "not one settles", and that procedural accuracy collapses at large recurrence counts [495]. Every other result in this section is shown on synthetic tasks with at most a few hundred million parameters, and most results use far fewer. A structural mismatch also limits the transfer of these results to

practice. Every positive expressivity theorem lets the loop count grow with input length, but looped language models run a small fixed or sampled number of loops. The missing experiment is a test of an input-dependent loop schedule at scale.

Takeaways

- Loop count acts as circuit depth and padding acts as circuit width, while precision decides between AC and TC classes.
- Loops beat chain of thought at small step budgets on parallel problems. Chain of thought wins with many steps when loop state is compressed, and it also wins with sampling.
- In convex in-context settings, looped models provably learn convergent optimizers. Extrapolation recipes work on synthetic tasks, but their results are unstable across seeds and tasks.
- Almost no theoretical prediction has been tested on looped language models.

10 Mechanisms and Interpretability

Section 9 described what looped models can compute in principle. This section asks what a trained looped model computes in each iteration. The question matters for science, because it tests whether looping implements iterative algorithms or only reuses parameters. It also has practical importance, because any oversight of latent computation (Section 14) depends on reading it. The mechanistic literature on looped models is young. It focuses on three public checkpoints (Ouro, Huginn and retrofitted Llama) and on two puzzle models, the Hierarchical Reasoning Model (HRM) and the Tiny Recursive Model (TRM) of Section 11.2. Its results are tested at no more than 3.5B parameters.

10.1 Limiting dynamics: fixed points, cycles and attractors

Per-layer fixed points. In looped models, per-layer fixed points are common but not universal. In Huginn and in retrofitted Llama models, each layer of the recurrent block converges to its own fixed point, so the block traces a consistent cyclic trajectory through latent space. Ouro does not reach a fixed point, even though successive iterations differ only slightly [48]. Non-fixed-point behaviors such as orbits are rare and affect "approximately 0.02% of tokens" without a system prompt [48]. An independent analysis of five configurations agrees with this result. In its words, "Only Huginn-0125 functions as a contraction mapping" [117].

Role of architecture. The fixed-point property depends on architecture and not on training alone. Controlled experiments from random initialization attribute the property to input injection and the placement of normalization. They report that "input injection results in stable fixed point behavior for all norm types other than Ouro, whereas omitting input injection means that only pre-norm reaches a stable fixed point" [48]. Models without fixed points show unstable internal stages at unseen depths. This links the present discussion to the extrapolation failures of Section 6.2. For compact recursive reasoners, one design builds the fixed-point property in directly. It reformulates HRM and TRM recursion as a contraction mapping and adds stability losses [137].

Limits of fixed points. Fixed points are not sufficient for correct reasoning. The Hierarchical Reasoning Model violates the fixed-point property that its training assumes, and it shows false attractors [413]. In Ouro and Huginn, one more iteration can make predictions worse, and such failures occur in every model–task condition tested. Applying only a quarter of the proposed update recovers 72.2–83.2% of selected failures. This suggests that "overthinking" is often an overshoot

along a useful direction and not a movement in a wrong direction [195]. In toy models, the same architecture and objective can be trained to perfect accuracy and still end up in qualitatively different dynamical phases, depending on initialization. The phase determines how compute scales with difficulty [257]. Unrolled refinement outside language shows the same non-convergence. In optical-flow refinement, "activations usually oscillate at a relatively high level after about 15 update iterations" [31].

Convergence time. Even models that do converge can take very different times to do so. One study examines equilibrium, recursive reasoning and recurrent-depth models of at most 140M parameters. In these models, the set of initial latent states that lead to each outcome forms basins that "become self-similar fractals on harder tasks" [280]. Basin entropy "strongly correlates with the number of iterations", and slow runs are those in which "reasoning becomes trapped near answers that are nearly, but not quite, correct" [280]. For models that sample their initial state at random (Section 3.1), this makes compute budgets based on convergence noisy on hard inputs.

10.2 What each iteration does

Stages of inference. The iterations of a looped model repeat stages of inference that resemble those of a feedforward Transformer. Standard Transformers pass through recognizable stages of inference (broad mixing early, refinement in the middle, preparation for prediction late). Looped models trained from scratch repeat these stages within every iteration. Retrofitted models run the first and last stages once, in the prelude and coda, and repeat only the middle stage [48]. Huginn is an exception that "does not demonstrate clear stages of inference". The authors suggest that this is likely because Huginn normalizes its residual stream, which keeps massive activations from forming [48]. At 1.7B parameters and about 400B tokens, looped and depth-grown models both show attention contributions with a period equal to the shared block [251].

Iterations and algorithmic steps. One iteration does not correspond to one algorithmic step. On graph-walk tasks, decoded predictions across iterations "can advance by different numbers of graph steps or remain at a reached target" [532]. A learned linear map applied to the state that enters a frozen loop can select which transition the shared weights perform. In graph-walk models, patching attention patterns recovers much of this steering effect [532]. Useful computation concentrates in a band of layers that is repeated in each iteration [242]. Progress is also often abrupt and not gradual. In a 30M-parameter recurrent-depth model trained on synthetic facts, the answer resolves within a single iteration, and the index of this iteration grows with the number of reasoning hops [283]. Puzzle models show a different pattern. In HRM and TRM, activation patching shows that high-level states write information locally and that repeated updates accumulate these local writes into a broader solution structure [356].

Unused multi-step capacity. Pretrained loops do little multi-step work by default, but small edits change that. On a reference-chain task, frozen Ouro-1.4B follows "0, 1.6, 2.3, 2.2, and 2.2 lines after one through five loops", and Huginn follows "only 1.4–1.6 lines" even at 32 recurrences [242]. A rank-8 adapter on a single layer raises this to 60 lines after four loops and to at least 160 after eight [242]. The latent capacity is present but unused, and a tiny targeted edit makes it available. This finding has implications for both capability elicitation and oversight (Section 14.3). A result on standard Transformers shows why reusing layers can help. In two-hop queries, the second hop can start in late layers that may not encode the needed knowledge. Patching later hidden states back into earlier layers, which runs those layers again, often fixes the answer [47].

Table 14: Interpretability instruments applied to looped models, what they found, and known pitfalls.

Instrument	Finding on looped models	Known pitfall
Logit / coda lens	no latent chain-of-thought signature in Huginn [334]	readouts depend strongly on which block is read
Fitted Jacobian lens	verbalizable, steerable content in Huginn and Ouro [513]	short read/write horizon; cannot distinguish absent from idle content
Linear probes + knockout	causal multi-line relay across loops [242]	probe accuracy alone is not causal use [415]
Probes + sparse autoencoders (HRM)	high probe accuracy on dynamic features [415]	ablating probe directions matches random controls
Depth truncation	standard test of depth use	conflates block applications, distinct computation and readout miscalibration [117]
Step-decomposed influence	influence concentrates in late iterations [248]	shown at 328M; alignment uses untested

10.3 Reading intermediate iterations

Table 14 summarizes the instruments used to read looped models and their known failure modes. The main lesson is that the findings depend on the instrument.

Latent chain of thought. Fixed lenses applied to Huginn find no evidence that intermediate results appear before final ones [334]. Fitted Jacobian lenses find a readable and steerable workspace within a horizon of about two recurrences [513]. Trained probes with causal knockouts find a genuine relay across multiple iterations [242]. The disagreement comes mostly from the instruments and not from the data. None of these studies shows intermediate results that can be decoded step by step in a pretrained looped language model on natural tasks.

Decodability and use. Information that can be decoded from an iteration is not necessarily used by the model. In HRM, linear probes reach high accuracy, but ablating the probed directions has effects "comparable to random controls" [415]. The most active sparse-autoencoder features are also "no more impactful than 50 random features" [415]. Training also affects whether an intermediate iteration is decodable. Only Ouro, which is supervised at every iteration, verbalizes injected concepts, while Huginn never does [513]. Agreement between an intermediate answer and the final answer also needs care. A margin analysis of Huginn-3.5B and Ouro-1.4B explains why intermediate answers can agree with the endpoint while their scores continue to change [338]. A related gap between decoding and use appears in two-hop reasoning. The first loop of a looped Transformer often makes the bridge entity nearly perfectly decodable, but the hidden state remains poorly aligned with the embedding of the bridge token [160]. Carrying a discrete embedding channel alongside the hidden state improves two-hop reasoning [160].

Measurement artifacts. Truncating depth at inference is the standard way to ask whether a model uses its depth. However, it changes three things at once: the number of block applications,

the amount of distinct computation and the calibration of the readout (Table 14). In one analysis, readout miscalibration accounts for a quarter of the apparent depth effect [117]. One study of preference readouts from Ouro's loop states withdrew its main numbers in an erratum after it found an ordering prior and leakage between data splits. After the correction, its strict accuracy is 63.9% [260] (Section 14.2). Mechanistic claims about looped models should be read with these pitfalls in mind.

Takeaways

- Injection and normalization choices determine whether a looped model settles into fixed points. Huginn settles into fixed points, and Ouro does not.
- Iterations repeat feedforward-like stages, but one iteration does not correspond to one reasoning step. Pretrained loops leave most of their multi-step capacity unused.
- Findings about latent reasoning depend heavily on the instrument, and decodability without causal tests is weak evidence.

11 Capabilities and Empirical Evidence

This section reviews what looped models have been shown to do. Section 11.1 covers pretrained looped language models on standard benchmarks, code and tool use. Section 11.2 covers small recursive reasoners on puzzle benchmarks. Controlled evidence on reasoning versus knowledge is in Section 7.3, and algorithmic and length generalization is covered in Section 9.4.

11.1 Looped language models

Unmatched comparisons. Most of the capability claims that attract attention come from comparisons without matched baselines. They compare looped models with public checkpoints trained on different data. Ouro's 1.4B and 2.6B models "match 4B and 8B standard transformers on most benchmarks" [604]. Nanbeige4.2-3B reports strong agentic results against larger open models [368]. A two-loop 7B coding model reports results competitive with much larger open models on software-engineering benchmarks [550]. Each of these results is evidence that looped models can be competitive products. None of them isolates the effect of the architecture, because data, compute and post-training differ from those of the baselines. In addition, Ouro runs four times the effective depth of its parameter-matched comparators [288].

Controlled comparisons. When the baseline is trained on the same data, the results follow the regime pattern of Section 15. In a recent example, a looped decision model reaches 72.0% accuracy, "13.5 points above a non-looped model of the same shape trained with the same recipe", at 7.7 times the compute of a single pass [164]. At matched parameters and data, looping helps on a broad range of tasks. A 1.4B looped model beats its dense twin on all 12 benchmarks [343]. In a family of 7B parallel-loop coders trained on 18T tokens, two loops raise SWE-bench Verified from 43.0 to 64.4, while three loops fall to 27.6 [550]. This result is a reminder that more loops are not monotonically better. At matched inference compute, the 1.4B looped model ties on mathematics and trails on knowledge (Figure 9).

Tool use, code and other tasks. On function-calling benchmarks, the gains of looped models are largest in compositional categories (multiple and nested calls) and smaller and model-dependent on single calls. Adaptive exits recover most of the gain of deeper recurrence and in several settings

match or exceed the best fixed depth [396]. The advantage is inflated by baselines that were fine-tuned with the same recipe and that collapse on parallel calls. On machine translation, a LoRA-tuned Ouro-2.6B is reported to be comparable to translation models three to five times larger, and translation quality improves over the first few iterations and then saturates [80]. Across these tasks, gains saturate within a few iterations, although on multi-call tool use the smaller Ouro model keeps gaining for longer (Section 7.2).

11.2 Recursive reasoning models

Recursive reasoning models are small networks, typically with a few million parameters. They are trained from scratch on about a thousand examples per task and applied recursively to solve puzzles such as Sudoku-Extreme, Maze-Hard and ARC-AGI. They are the most visible success of looping outside language modeling, and they are also the most sensitive to the evaluation protocol.

From HRM to TRM and URM. The Hierarchical Reasoning Model (HRM) uses two modules that are updated at different frequencies. It computes a one-step gradient through only the last update of each module, applies deep supervision over detached outer steps, and uses a learned halting head [499]. The Tiny Recursive Model (TRM) replaces the two modules with one 2-layer network that carries two states (an answer and a latent), and it backpropagates through the full last recursion. With 7M parameters, it reports 45% on ARC-AGI-1 [245]. The Universal Reasoning Model (URM) adds a short convolution and truncated backpropagation, and it reports 53.8% pass@1 on ARC-AGI-1. Its authors attribute the gains of this line of work to "the recurrent inductive bias and strong nonlinear components of Transformer, rather than from elaborate architectural designs" [168]. Table 15 describes these models in terms of the template of Section 3.

Other recursive reasoners. Several other architectures also solve puzzles by iterating a shared update. Neural cellular automata apply a shared local update to solve mazes, Sudoku and ARC puzzles [140]. A model that iterates over internal ticks with neuron-level processing and synchronization solves two-dimensional mazes and adapts its compute on image classification [116]. A recurrent latent reasoning model learns from in-context demonstrations and iterates on ARC tasks [135]. A second group of papers changes how the recursion is formulated. One weight-tied map iterates on simplex belief states, so a fixed point always exists [118]. A framework casts Tiny Recursive Models as recursive inference and adds a reweighting component [269]. Two further designs make the refinement stochastic or flow-based. A recurrent refiner self-conditions a flow over discrete outputs on its past predictions [210], and stochastic recursive latent transitions trained by variational inference allow multiple hypotheses and sampling-based scaling [27].

Design ablations. Ablations and reproductions agree on several design choices. Within the HRM/TRM pipeline, weight-tied recursion beats unshared depth at matched depth or compute. A 4-layer block looped eight times reaches 40.0 pass@1 on ARC-AGI-1, against 23.75 for a 32-layer model with eight times the parameters [168]. Two tied blocks reach 32.6% on Sudoku-Extreme, against 2.6% for a 42-block feedforward model that memorizes its training set [218]. Separate high- and low-level modules are not needed, but two distinguishable states help. A re-examination of HRM finds that a plain stack performs similarly to the hierarchical model [173]. In the TRM ablations, one network beats two on Sudoku (87.4 versus 82.4), and two latent features beat one (87.4 versus 71.9) [245]. Another design reuses one shared Transformer for both recurrent states and injects the input into only one of the two updates [443]. An extension of HRM adds a persistent high-level subgoal to study when to re-plan [69]. The gradient horizon can be either too short

Table 15: Recursive reasoning models as instances of the looped template.

Model	Modules and sharing	Recursion	Gradient	Params
HRM [499]	two modules (high and low level), separate weights	low level every step, high level every T steps; outer supervised segments	one step (last update of each module)	27M
TRM [245]	one 2-layer network, two states	n latent updates then one answer update; T recursions; 16 supervised steps	full last recursion	5–7M
URM [168]	one shared 4-layer block with short convolution	8 inner loops, outer ACT steps	last 6 of 8 loops	not stated
EqR [218]	two weight-tied blocks	up to 1,024 outer steps at test	truncated, detached carry	5.03M
HRM-Text [500]	two 16-layer modules	2 high-level cycles of 3 low-level steps	truncated, horizon warmed from 2 to 5	1B

or too long. HRM's one-step gradient gives 56.5, against 87.4 for backpropagating through the full last recursion [245], while backpropagating too far is also suboptimal [168, 597]. Noise in the recursive trajectory improves generalization [128, 218], and strong nonlinearity in the shared block is important on ARC [168]. Structural priors can also be built into the shared block. For example, a symbol-equivariant recurrent reasoning model for Sudoku attends over both positions and symbols [157].

Protocol sensitivity. Scores on these benchmarks depend heavily on evaluation choices, which are often unreported or inconsistent across papers. On ARC-AGI, models of this family train on the demonstration pairs of the evaluation tasks. A learned per-task embedding is "the only channel connecting these demonstrations to the held-out test input" [438]. Replacing that embedding with a blank or random one gives 0%, and voting over 1000 augmentations adds 10.75 points of pass@1 [419]. With the same pipeline, a non-recurrent dense control reaches 33.50 pass@2, against 40.30 for HRM [438]. If the per-task embedding table is counted, these "tiny" models have about 450M learnable parameters on ARC-AGI-1 [597]. Independent reruns of the same baseline give widely different results. TRM's reported Sudoku-Extreme accuracy of 87.4 has been reproduced at values from 66.8 to 87.9 across papers, and its Maze-Hard accuracy of 85.3 at values from 60.8 to 92.0 [63, 168, 250, 512]. These spreads are as large as most claimed method gains (Figure 8). Exact-match metrics on Maze-Hard also penalize valid alternative paths [272]. Across pipelines, a strong from-scratch ARC-AGI-1 score does not require recursion. A feedforward 18M vision Transformer reaches 54.5 pass@2, against the reported 40.3 for HRM and 44.6 for TRM. Its authors write that "once test-time training is completed, our model performs fully feedforward inference, with no recurrence involved in reasoning" [215]. That comparison is not matched (per-task test-time training, voting over 510 views and extra generated training pairs). A looped version of the same vision model raises the score to 65.8 [449]. These scores therefore depend on the evaluation protocol and not on recurrence alone.

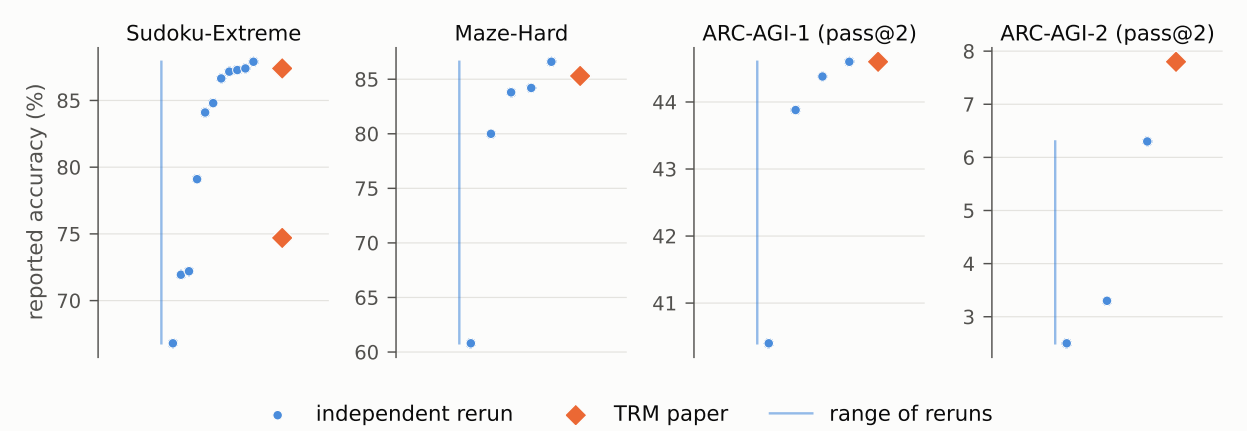


Figure 8: Replication spread of the Tiny Recursive Model. Diamonds show values reported in the TRM paper (two Sudoku values for its MLP and attention variants). Dots show values reported for TRM by independent papers, in sorted order, and the vertical line spans their range. Each panel has its own vertical scale. Variants, splits and aggregation protocols differ across sources, so the figure shows the spread of values and is not a ranking. Every value is checked against the text of the paper that reports it.

Test-time compute. Extra recursion at inference helps on Sudoku, where running TRM with 512 steps instead of 16 on the hardest puzzles raises accuracy from 59.2 to 87.5 [272]. It helps little on ARC-AGI-1, where extending recursion past the training depth left accuracy unchanged in one audit [419]. Parallel stochastic trajectories with a selector add further gains. With a halting head as the selector, the gains need no retraining [436]. A residual criterion helps only once the depth is sufficient and training has shaped the latent landscape [218]. This advantage holds only when the selector tracks correctness. On Maze-Hard, more exploration produces more correct rollouts, but the halting head cannot pick them out [436]. Proposed selectors differ in the signal that they use. Hopfield-type energy scores select among the trajectories of a recursive reasoner and also bound its useful depth [595]. A confidence-based rule votes over several latent initial states of recurrent reasoning models such as HRM [274]. A halt-first rule for Tiny Recursive Model ensembles treats halting speed as confidence [125]. Guided stochastic exploration over the latent trajectories of a frozen recursive reasoner reweights them with its early-stopping head [104].

Transfer to language. The strongest positive result is HRM-Text, a 1B hierarchical model trained on instruction data. It beats a 3B Transformer at matched training compute and tokens (MMLU 60.73 versus 56.67) [500]. However, inference compute is not matched in this comparison, and the training objective alone raises the MMLU of the Transformer baseline from 40.55 to 53.15 [500]. An independent 1B model reproduces the recipe on different data [428]. Small-scale autoregressive transfers give no reliable gains [410] or trade away raw benchmark accuracy [281]. In a from-scratch language-modeling test, HRM-style recursion has a higher training loss than GPT-2 at equal tokens [459]. A language model with fast and slow shared-weight modules has also been compared with a flat Universal Transformer [199]. Other work attaches recursion to a pretrained model. A trainable recursive block on a frozen small language model refines a latent thought [387]. A recursive reasoning framework built on a pretrained Qwen model repeatedly refines a candidate puzzle solution [342]. In code generation, a Tiny Recursive Model tracked across training beats a parameter-matched control, but a depth-matched Transformer with more parameters fits and generates better [453]. There is no compute-matched evidence that TRM-style recursion improves language modeling.

Takeaways

- On constraint-satisfaction puzzles and within the HRM/TRM pipeline, weight-tied recursion with supervised outer refinement clearly beats unshared depth at matched depth or compute.
- The hierarchy of HRM is not needed, and a single small network with two states, an intermediate gradient horizon and noise works better.
- ARC-AGI results of this family are transductive and depend strongly on the evaluation protocol. Benchmark tables should record training data, task-embedding parameters, aggregation and pass@k.
- Transfer to language modeling is promising at 1B, but the main comparison matches training compute and not inference compute. No study shows that TRM-style recursion improves language modeling at matched compute.

12 Beyond Language

Looping has spread to vision, speech, multimodal models, reinforcement learning, recommendation and scientific modeling. We find four patterns that recur across these domains. First, naive looping usually fails, and the fixes are the same as in language. The fixes are to supervise intermediate iterations, to condition on the iteration index, and to keep the first and last blocks unshared. Second, the result depends on which comparison regime is used. At matched parameters, looping wins almost everywhere, while at matched compute it wins on planning, constraint-satisfaction and generation tasks. At matched compute, it loses or ties on small-scale perception unless the shared block is wide enough. Third, the useful number of loops depends on the task. Reasoning-like settings keep improving up to tens of iterations, while perception and speech settings peak at two to five. Fourth, the evidence is small-scale, because most models outside language have fewer than 100M parameters. The largest looped models trained from scratch have about 1B. Table 16 summarizes representative results and the regime under which each one holds.

Tying versus untying. Outside language, controlled comparisons of tied and untied models at equal depth suggest an *algorithm-alignment* interpretation of when tying wins. When the shared block mirrors an iterative algorithm, tying wins. A tied recurrent update for optical flow reaches lower error than an untied one with far fewer parameters (1.63 versus 1.96 end-point error, 4.8M versus 32.5M parameters) [484]. A tied value-iteration module beats its untied version, especially with little data [472]. For generic perception, untied layers win at equal depth and width because they have more parameters, and they perform about the same as tied layers at equal parameters [131]. No study has tested whether the same criterion predicts when looping helps language models.

12.1 Diffusion and generative models

In diffusion denoisers, looping adds a compute axis that is distinct from the number of sampling steps. The authors of a looped diffusion Transformer (DiT) find that at matched per-image FLOPs, "allocating compute to loop depth consistently yields higher performance than allocating it to additional denoising steps" [96]. The effect holds even without training, in frozen 2.4B and 9.8B text-to-image models. Repeating their layers at 25 denoising steps outperforms 50 steps without repetition, and it also runs faster [549]. This is the only evidence in this section that loops the layers of a multi-billion-parameter model. In masked diffusion over discrete sequences, looped and fixed-point denoisers are strongest at small denoising budgets [65, 353]. One-step generators can be distilled into a single interval-conditioned shared block with substantial parameter compression and some loss in quality [22]. In class-conditional image generation, looped diffusion Transformers

Table 16: Looped models outside language: representative results and the regime under which they hold. T2I: text-to-image. ViT: vision Transformer. DiT: diffusion Transformer. FID: Fréchet inception distance. TTS: text-to-speech. WER: word error rate.

Domain	Work	Result	Regime
Diffusion (T2I)	Looped-DiT [96]	loop depth beats extra denoising steps at matched FLOPs; beats parameter-matched model by 4.0 points	several, incl. matched training FLOPs
Diffusion (frozen)	training-free looping [549]	25 steps with loops beat 50 steps without on a 2.4B model	matched parameters
Masked diffusion	R-MDM [65]	93.6% vs. 77.8% Sudoku validity against a 5× larger equal-depth model	matched effective depth
Vision	SoftMoR [289]	top-1 79.83 → 82.48 with recursion	nearly matched parameters (1.7M added)
Vision	weight-tied ViT study [190]	untied ViT preferable at matched compute	matched FLOPs
Vision	bViT [58]	one shared block ×12 at ViT-L width: 0.805 vs. 0.808 top-1 with 14.6M vs. 304.3M parameters; fails at small width	matched FLOPs and data
Image generation	ELT [186]	looped DiT (FID 2.83) beats a 32-layer DiT (3.43) at matched inference FLOPs	matched FLOPs; matched parameters
Masked diffusion LM	LoopMDM [291]	reaches baseline loss with up to 3.34× fewer training FLOPs	matched parameters and training FLOPs
Robotics	LoopVLA [442]	96.0 vs. 94.9 LIBERO success, 1.2B looped vs. 2.2B full depth	matched inference FLOPs
Tabular	LoopICL [35]	competitive with a strong tabular model at matched FLOPs with 88.2% fewer parameters	matched FLOPs
Multimodal	LoopVL [401]	beats a same-layer 1B baseline (MMStar 55.33 → 63.47); collapses beyond trained cycles	matched data
Speech (TTS)	looped flow-matching TTS [6]	47.1% fewer parameters at matched effective depth, WER not significantly different	matched depth and FLOPs
RL / planning	Looped Actor [422]	about 2× Boxoban success vs. a 16× larger untied model	matched FLOPs
Robotics	RD-VLA [488]	LIBERO success 8.4% → 92.6% from 1 to 8 iterations; KL stopping saves 34% compute	within model
Recommendation	LoopCTR [480]	beats compute-matched stacked model on all four datasets; zero-loop inference nearly as good	matched FLOPs

beat a deeper unshared model at matched inference FLOPs. Intra-loop self-distillation makes them usable at any loop count, which is not true of vanilla looped generators [186]. In masked diffusion language models, looping two early-middle layers reaches the baseline's final loss with 3.34×, 2.95× and 2.34× fewer training FLOPs on three corpora [291]. The outcome can depend on the sampling budget. One fixed-point diffusion Transformer has one unshared layer on each side of an implicit core with input injection, which is the prelude–core–coda layout, used a year before Huginn. It has 85M parameters against 674M for DiT and wins at 140 block passes (FID 33.9 versus 110.0 with the DDIM sampler), but "for more than 560 it is outperformed by DiT" [32]. A next-scale visual autoregressive generator uses a similar layout, with an implicit equilibrium layer between shallow pre and post blocks [239]. In multimodal diffusion, a recursive sparse mixture of experts in the attention refines visual tokens over latent steps [467].

12.2 Vision and multimodal models

With few added parameters, recursion improves vision Transformers (ViTs). A recursive ViT improves ImageNet top-1 from 79.83 to 82.48 between one and four recursions while adding only 1.7M parameters [289]. Earlier recursive designs report similar gains [444]. At matched compute, the result reverses on small-scale classification. One study reports that "Under matched computational budgets, the classical ViT remains preferable" [190]. Width also matters more than tied depth in implicit vision models [456]. Strict sharing at equal depth reduces accuracy, and top-1 "decreases by 2%" when sharing halves the parameters of a DeiT. This happens partly because layer normalization "should not be identical during parameter sharing". Per-layer normalization and small per-layer transformations recover part of the loss, and adding weight distillation recovers the rest [584]. This approach is the vision precursor of relaxed sharing in language models (Section 3.3). Tying helps when the block mirrors an iterative algorithm. Solving for the fixed point of an optical-flow update operator, instead of unrolling it, is measured to be "about 20% faster than the RAFT models with the same structure and size" [31]. The result at matched compute also depends on width. At ViT-L width, a single shared block applied twelve times nearly matches ViT-L at matched compute (0.805 versus 0.808 top-1) with 14.6M instead of 304.3M parameters. At ViT-S width, the same design falls far behind [58]. In multi-view 3D reconstruction, shared weights beat an untied stack at matched data and compute [56]. On ARC-AGI from pixels, a looped vision model reaches 65.8, against 54.5 for a non-looped model with the same parameters [449].

Some mechanisms from language models transfer poorly to vision. In ViTs, hard per-token recursion routing in the style of Mixture-of-Recursions underperforms soft mixing of all steps, because it keeps only one step's output per token [289]. A hierarchical reasoning model used as an image classifier overfits relative to a small convolutional neural network (CNN) [341].

Weight reuse across depth also appears in many task-specific vision systems. Early convolutional examples reapply one layer many times for image super-resolution [258], or use the iteration count of a recurrent segmentation network to set a speed and accuracy trade-off at deployment [346]. Another network feeds late features back into earlier layers to refine medical image segmentation [265]. In video restoration, sharing the recurrent block cuts parameters by 71.6% with a difference in peak signal-to-noise ratio (PSNR) of 0.08 dB [539]. More recent systems reuse a single Transformer layer for several iterations to refine poses in two-view reconstruction [581]. Others repeatedly apply a few shared Transformer blocks in a visual tracker [390], or reuse one scale- and loop-conditioned core across resolutions in a visual tokenizer with far fewer parameters [337]. A small recurrent Transformer trained on procedural data can also be expanded to initialize larger vision and language Transformers [447].

In multimodal modeling, a looped vision–language model trained from scratch beats a baseline with the same unique layers at matched data. However, it collapses when run with more cycles than it was trained with, and its MMStar score drops from 63.47 to 24.00 [401]. Huginn's test-time scaling still holds when vision is added. A Huginn-based multimodal model improves on image ScienceQA from 32.82% at one recurrence to 91.57% at thirty-two [591]. Retrofitting recursion onto a pretrained multimodal model fails when done naively, and it needs a connector back to input space and per-step supervision [544]. Another design re-applies every layer of a vision–language model once through a token-wise gate, which adds few parameters [287]. Looping a localized block beats looping the whole decoder [508]. Parameter sharing in multimodal Transformers is older than these designs. Audio-visual Transformers share parameters across layers and modalities through a low-rank scheme [290].

12.3 Speech

In flow-matching text-to-speech, a looped model uses 47.1% fewer parameters than its unshared counterpart at the same effective depth. Its word error rates are not significantly different, and it has lower peak memory but unchanged sampling speed [6]. Too much sharing hurts, and adjacent-layer sharing beats cyclic sharing, as in vision [319]. In speech recognition, word error rate improves across supervised loop checkpoints, which gives a way to adjust test-time compute. The looped model clearly beats a parameter-matched non-looped encoder, and it matches or slightly beats a 16-block unshared encoder with 28.9M parameters while using 7.7M [249]. Unshared prelude and coda layers recover most of the gap left by sharing in recognition encoders [417]. Loop-specific low-rank adapters are another way to relax sharing. Re-running a frozen audio deepfake detector with such adapters lowers cross-domain error [153]. The choice of which tokens to loop also matters. In audio tagging, looping all tokens degrades with more passes, whereas looping only the class token over cached keys and values improves at every pass [11]. In a spoken-dialogue model, looping a block of middle layers moves reasoning from generated tokens into depth, and it produces 64.5% fewer generated tokens than chain-of-thought fine-tuning [59]. In audio language models, an adaptive-depth loop in the connector raises average accuracy by 2.9 to 3.8 points on three 7B backbones. Replacing its adaptive halting with a fixed depth of four causes the largest single drop in its ablation [435].

12.4 Reinforcement learning, robotics and control

Planning-heavy reinforcement learning benefits from looping even against untied models with matched compute. A looped actor with 0.5M parameters roughly doubles the success rate on Boxoban and ties on goal-conditioned control. It "substantially outperforms the Iso-FLOPs and DRC baselines despite having 2 to 16 times less" parameters [422]. The DRC (Deep Repeated ConvLSTM) baseline is itself a tied design. It is a stack of ConvLSTM modules "repeated N times within each time-step", and it learns planning-like behavior with model-free reinforcement learning [192]. A recurrent-depth action head for vision–language–action models raises manipulation success from 8.4% with one iteration to 92.6% with eight. Halting on a KL criterion saves 34% of compute at equal success [488]. Depth elasticity has to be trained, because fixed-depth looped policies are reliable only near their trained depth [515]. At matched inference FLOPs, a looped vision–language–action policy with 1.2B parameters slightly outperforms its 2.2B full-depth counterpart [442]. Another vision–language–action policy refines parallel latent hypotheses with weight-tied steps before it decodes actions [578]. Recurrent policies trained with reinforcement learning improve up to tenfold from one to ten recurrent steps on some tasks [179]. A Hierarchical Reasoning Model trained with reinforcement learning on dynamic mazes reuses its recurrent computation across time-steps [115]. In a world-model planner, a fixed four-iteration transition never beats a single iteration, and only learned sparse allocation helps [243]. Another looped world model refines latent states with a parameter-shared Transformer block and adaptive depth [333].

12.5 Recommendation

In click-through-rate prediction, a looped model beats a compute-matched stacked model on all four datasets. With per-loop supervision, even zero-loop inference nearly matches looped inference on one dataset. There, the area under the receiver operating characteristic curve (AUC) is 0.8715 at zero loops, against 0.8728 at one or two loops and 0.8726 at three [480]. In this domain, looping acts largely as a regularizer during training. In sequential recommendation, gains peak at a moderate depth and then decline [326, 437]. Another sequential recommender refines compressed latent interests with a shared recursive reasoner [122].

12.6 Science and other domains

Protein-structure models that recycle their trunk are, in effect, weight-tied loops that are already deployed. Skipping early blocks in later recycles reduces block evaluations from 212 to 92 while preserving agreement for medium and long proteins [292]. Tabular in-context learning is well suited to looping. An interpretability study of tabular foundation models motivates a looped single-layer model with comparable performance [36]. A looped tabular model is competitive with a strong baseline at matched FLOPs with 88.2% fewer parameters, and it works at about twice its mean training loop count [35]. Another tabular model finds the opposite after two-loop training: "Additional inference-time repetition therefore does not reliably reproduce the benefit of training with greater loop depth" [312]. In molecular string generation, a 53K-parameter block applied eight times beats a 527K-parameter unshared model in validity [244]. In surrogate models for partial differential equations, a neural operator repeatedly applies one backbone block and grows its depth during training [206]. Domain gains are not always due to looping itself. In an omics classifier, plain recursion is slightly worse than a larger vanilla Transformer, and the improvement comes from biology-guided routing [213]. Recursive reasoning models have been transferred to optimal control and astrodynamics with simulators in the loop [234, 529], but these studies have no matched non-recursive baselines. Tiny Recursive Models have also been adapted to insurance pricing on tabular data, where the model refines an answer token iteratively [385]. An adaptation to combinatorial optimization iterates on a latent state with adaptive halting [593]. In a Pareto comparison of engineering surrogates, the two-latent recursion of TRM is "parameter- and FLOP-expensive without a commensurate accuracy return" [293].

Takeaways

- Across domains, the fixes for naive looping are the same as in language: intermediate supervision, iteration conditioning and unshared boundary blocks.
- At matched compute, looping wins on planning, constraint satisfaction and generation, and loses or ties on small-scale perception.
- In diffusion, loop depth is a second test-time axis that can beat additional denoising steps.
- Reasoning-like tasks keep improving with more loops, while perception and speech peak at two to five. Loops beyond the training depth rarely help without randomized-depth training, and the savings come from adaptive halting.

13 Relation to Neighboring Paradigms

Looped models are one of several ways to spend extra serial computation without adding parameters. Table 3 compared the three axes along which a Transformer can do so. This section places depth recurrence among its closest neighbors, describes how each neighbor relates to looping, and names the comparisons that have not been made.

13.1 Sequence-axis latent reasoning

Continuous-thought methods feed the last hidden state back as the next input embedding, so each latent "thought" is one more pass through the full network along the sequence axis. The Coconut authors link this directly to depth and write that "the effective depth of the transformer increases because the generated outputs are looped back to the input" [200]. With the same weights at GPT-2 scale, continuous thoughts beat no chain of thought on GSM8K (34.1 versus 16.5) but trail language chain of thought (42.9) [200]. On a search-heavy synthetic task, they beat language chain

of thought [200]. Continuous thoughts also depend on supervision, and without its curriculum Coconut scores 14.4 on GSM8K [200]. In addition, latent steps collapse as their number grows unless they are supervised step by step [523]. Internalizing chain of thought into a fixed-depth model is the baseline that uses no extra compute. A 12-layer GPT-2 learns 9×9 multiplication with no visible steps, although its layers are "far fewer than the number of reasoning steps in the CoT" [124]. The method has a cost, because "models trained using our approach lose interpretable intermediate steps" [124].

The minimal sequence-axis variant feeds nothing back and uses meaningless filler tokens instead. These tokens let a 34M model solve 3SUM instances on which it is otherwise near chance (100% versus 66%), but they add only parallel computation [391]. Like looped models, several sequence-axis methods claim to add computation per token without adding parameters. A model with one latent thought per token beats a model twice its size that is trained on the same data, at equal inference cost [574]. Routing a thought token through the network recovers 67% and 81% of the gain from doubling depth with about half the parameters [225]. A third line of work feeds the deep state of one token back into the shallow layers of the next token [60, 514]. In this way it adds depth across time steps instead of within a single token. One large-scale study argues against looping for systems reasons, because "looped computation does not fit naturally with the pipeline parallelism used to train the largest models" [325].

The missing head-to-head

No study in the corpus compares one latent thought per token with a two-loop looped model at matched parameters, data and per-token FLOPs while reporting KV memory for both. Both methods spend the same extra compute per token. They differ in whether the state travels through the KV cache (sequence axis) or through the residual stream (depth axis). This comparison would settle whether the choice of axis matters.

13.2 Diffusion language models

Diffusion language models refine a whole output sequence over many denoising passes of the same network. Standard diffusion language models therefore loop over the *output* and not over the hidden state of a token. The two can also be combined, for example in an adaptively looped diffusion language model that iterates a shared core inside each denoising step. Raising its exit threshold increases the average number of loops per token from 1.6 to 2.34 and its average score from 77.9% to 79.1% [148]. The two families become more similar when a latent channel carries state across passes. Examples are diffusion models with a recurrent relay trained by truncated backpropagation [421] and diffusion models with gradients only through the final refinement pass [462]. Gradients only through the final pass are the diffusion analogue of the one-step gradients of recursive reasoning models. Theoretical work connects the two families, but its conclusions do not all point the same way. Masked diffusion models are characterized as padded looped Transformers [468]. Continuous diffusion is argued to be more expressive than discrete diffusion and at least as expressive as looped Transformers, because it can simulate any looped Transformer [599]. In contrast, diffusion with an exact score is confined to TC^0, because it converges in a number of steps that does not grow with the input [329]. Extra denoising steps therefore "should yield minimal improvements" on such tasks [329]. This result and the previous one differ in whether the score is exact. Finally, parallel samplers for recurrent-depth models use the connection to diffusion directly [176].

13.3 Adaptive depth without sharing

Early exit, layer skipping and Mixture-of-Depths let each token use a subset of the unique layers of a model. These methods save compute, but they cannot go beyond the depth of the stack. The mechanisms of these methods transfer directly to looped models. The expert-choice routing of Mixture-of-Depths [409] reappears as per-token recursion depth in Mixture-of-Recursions [25]. Early-exit decoders face the same missing-KV problem as looped early exit [430]. Self-speculative decoding with early layers [132], or with adaptively skipped intermediate layers, which needs no extra training [534], is the unshared precursor of looped self-speculation (Section 8.2). The Mixture-of-Depths authors also observe that "it is better to add depth than to add width when adding FLOPs to the model" [409].

13.4 Deep equilibrium and implicit models

Deep equilibrium models are the infinite-iteration limit of weight-tied looping. Their output is the fixed point of one layer. A root solver finds this fixed point, and the model is trained by implicit differentiation with constant memory [30]. Their stability tools (Jacobian regularization, monotone parameterizations, reversible gradients) transfer to looped language models (Section 6.2). They also show a limit for overparametrized networks, in which "increasing the number of iterations at test time cannot improve performance" [408]. This result is directly relevant to claims of test-time depth extrapolation (Section 7.2). Equilibrium formulations have also been applied to linear recurrences and to generation. Dense linear recurrent networks can be defined as fixed points of iterated diagonal recurrences, which trades iterations for expressivity [362]. A deep equilibrium Transformer distilled from a diffusion model generates images in one step [177].

13.5 How standard Transformers use depth

Analyses of standard models supply much of the motivation for looping. In Llama 3.1 70B, layers in the second half "contribute much less than those in the first half", and processing depth does not grow with problem difficulty [109]. Long chain-of-thought models show no increase in effective depth over their base models [216]. Implicit k-hop reasoning requires a number of layers that grows linearly in k [559]. If the layers of a stack are not used compositionally, reusing a block may be a better use of parameters. One study of vision Transformers makes this hypothesis concrete, and proposes that trained models can be rewritten as a few distinct blocks applied recurrently [231]. Pruning studies that remove the layers whose input and output are most similar point in the same direction [349], but their results need careful reading. Deep layers can be removed with little loss on knowledge question answering, but "both GSM8K and HellaSwag, our two reasoning tasks, exhibit immediate degradation in performance with any amount of pruning" [188]. Deep-layer redundancy is therefore task-specific, and it is not evidence that deep layers are unused for reasoning. The causes of these patterns are contested, with explanations ranging from the pretraining objective [109] to pre-norm variance growth [464]. The same diagnostics have barely been applied to looped models.

13.6 Prior surveys and the scaling-axis claim

Prior surveys treat looped models as one branch of latent reasoning. One survey describes them as "vertical recurrence for expanding computational depth", as opposed to horizontal recurrence over the sequence [603]. Others place them at the "layer-wise vertical" level of latent chain of thought [83] or call them "layer-recurrent execution" [305]. A position paper similarly organizes recurrent Transformers into a taxonomy by recurrence axis, depth versus step [365]. The first

Table 17: Prior surveys and lists that cover looped models. "Core papers cited" is a lower bound from matching the titles of our core papers against the text of each work (547 of our 560 core papers have titles long enough to match). "Regime audit" means an analysis of which quantities each comparison holds fixed.

Work	Version used	How looped models are covered	Core papers cited	Mentions Ouro / TRM	Regime audit
Latent chain-of-thought survey [83]	v3, Aug. 2026	one level ("layer-wise vertical" [83]) of latent chain of thought	12	no / no	no
Latent reasoning survey [603]	v2, Jul. 2025	vertical recurrence, one branch of latent reasoning	14	no / no	no
Implicit reasoning survey [305]	v1, Sep. 2025	layer-recurrent execution, one form of implicit reasoning	6	no / no	no
Community list [373]	README, Oct. 2026	topology taxonomy and paper list, no analysis	73	yes / no	no
This survey	snapshot Oct. 2026	dedicated survey with a unified template, claims ledger and safety evidence map	560	yes / yes	yes

of these surveys notes that "most studies compare their models to non-reasoning LLM baselines rather than to each other" and that it "does not offer a direct empirical comparison" [603]. The comparison-regime audit in Section 15 is designed to fill that gap. Table 17 compares the existing surveys and a community paper list with this survey.

Several lines of work claim to be a new scaling axis. Work on parallel computation calls it "the third and more inference-efficient scaling paradigm" [77], and work on per-token latent reasoning calls it "an alternative axis" [576]. We use "recurrent depth as a scaling axis" in a specific sense, to mean serial reuse of parameters within a token. This is distinct from parallel reuse across streams and from serial computation across sequence positions.

Takeaways

Like looped models, several sequence-axis methods claim to add computation per token without adding parameters. They carry state through the KV cache instead of the residual stream, and no study compares the two axes at matched parameters, data and per-token FLOPs. Diffusion language models refine the output and not the hidden state of a token, but the two families can be combined. Theory also connects them, with conclusions that depend on whether the score is exact. Adaptive depth without sharing cannot go beyond the depth of the stack, but its routing, early-exit and self-speculation mechanisms transfer directly to looped models. Deep equilibrium models are the infinite-iteration limit of looping. Their stability tools transfer to looped models, and for overparametrized networks they show that extra test-time iterations cannot improve performance. Analyses of standard Transformers motivate looping, but deep-layer redundancy is task-specific, and the same diagnostics have barely been applied to looped models.

14 Safety, Oversight, and Trustworthiness

Chain-of-thought reasoning gives overseers a written trace that they can read. A widely endorsed position paper argues that monitoring chains of thought for "the intent to misbehave" is a unique opportunity [271]. Because monitorability "may be fragile", the paper recommends that developers "consider the impact of development decisions on CoT monitorability" [271]. Looped models are

Table 18: Evidence map for the safety-relevant properties of looped models at the October 2026 snapshot. "None" means no study in our corpus.

Property	Looped language models (≥1B)	Adjacent evidence (non-looped latent reasoning)
Chain-of-thought monitorability	one study: Ouro loop-depth sweep, Nanbeige4.2-3B cross-model comparison [502]	[85, 133, 134, 154, 191, 271, 611]
Probe-based latent monitors	signals on Ouro that are weak, also readable without recurrence, or corrected downward by an erratum [72, 260, 261]	[154, 406]
Causal faithfulness of latent steps	partial (causal interventions, no faithfulness metric) [242, 513]	[52, 241]
Adversarial robustness, jailbreaks	**none**	[78]
Refusal and alignment under variable depth	**none**	non-shared layer skipping [306]
Knowledge editing	**none** (toy scale only [380])	none
Unlearning, memorization, privacy	**none**	none

a direct example of such a change, because they move serial computation from generated tokens into repeated passes over hidden states. In September 2026, press coverage reported that a new OpenAI model uses "recurrent depth" [483]. In the same month, safety researchers argued that latent reasoning architectures would "likely undermine" chain-of-thought oversight unless shown otherwise [155]. This section reviews what is known, and Table 18 summarizes it. The cells that report no study are the main result of the table, because they show which safety properties of looped models have not been examined.

14.1 Chain-of-thought monitorability

The role of depth. The argument for chain-of-thought monitoring depends on a depth limit. "The number of serial operations a transformer can perform within a single forward pass is constrained by its number of layers" [133]. Sufficiently serial tasks must therefore pass through the visible trace. The same authors call reasoning entirely in a continuous latent space the "most extreme failure mode" [133], and they cite the recurrent-depth model among their examples. A second study states the link explicitly, writing that "Increasing depth increases the maximum serial chain of cognition a model can perform without needing to use the CoT" [191]. In this view, generated tokens carry the state between forward passes, and each pass "can perform only as much work as the LLM's depth allows in a single cycle" [295]. Looping multiplies the depth of each pass without adding parameters, so it should shrink the set of tasks for which a visible chain of thought is necessary. This is our inference, and no study measures chain-of-thought necessity as a function of loop count.

Direct evidence. The only study of chain-of-thought monitorability in looped language models runs two experiments [502]. Within one model family, raising the loop count of Ouro from four to eight lowers monitorability on three of eight benchmark tasks, and only under stress-test prompts.

For Ouro-1.4B-Thinking under direct concealment, monitorability falls "from 77.9% to 41.8% on Logic" [502]. Across architectures, a looped 3B model is not less monitorable than non-looped models matched on size, layer count or effective depth. The authors conclude that there is "no evidence that LoopLMs are systematically less monitorable" [502]. The models in the cross-architecture comparison were not trained on the same data.

Two caveats determine how these results should be read. First, Ouro was trained with four recurrent steps (Section 6.1), so the deeper settings are *extrapolation* depths. In this regime, Ouro has no fixed point and its internal stages drift (Section 10.1). The measured drop may therefore come from behavior beyond the training horizon instead of from additional hidden reasoning. Second, both experiments evaluate existing checkpoints, and neither tests what happens when training *rewards* moving decisive computation out of the visible trace. This is the scenario that oversight researchers worry about (Section 16). For non-looped models, such pressure is known to work. Reinforcement learning against a chain-of-thought monitor raised accuracy from 69.8% to 94.4%, while monitor triggers fell from 81.8% to 19.5% [133]. Under a generic obfuscation penalty, a model was seen moving "much of its reasoning into the final response" [191]. The success of prompted obfuscation depends on the model. The effectiveness of a chain-of-thought monitor collapses to about 10% for one attacker model and stays at 86–88% for another [611]. Loop iterations would give such pressure one more place to move computation, and no study has tested this. For non-looped models with latent tokens, "monitorability depends more on properties of the task" and on the level of access to model internals than on the reasoning mode [154]. This finding is similar to the cross-architecture null result.

The baseline against which looped models are compared is itself imperfect. In one study, reasoning models acknowledge the prompt hints they use with overall faithfulness scores of "25% for Claude 3.7 Sonnet and 39% for DeepSeek R1" [85]. The study focuses on "reasoning that models can perform in a single forward pass" [85], which is the kind of computation that looping enlarges. A comparison between looped and standard models therefore measures a change relative to a baseline that is partly unfaithful. This unfaithfulness is concentrated on tasks that do not need the trace. When following a hint requires reasoning, "the rate of providing the incorrect answer without attribution drops to the no-hint baseline" [133]. Looping can move a task out of the regime in which the trace is necessary. Comparisons across loop counts should therefore report chain-of-thought necessity (accuracy with and without the trace) at each loop count.

Inexpensive tools for tracking such changes exist, and one of them is a prompted autorater that scores the legibility and coverage of chains of thought [134]. Its authors offer it as a way to track how design decisions affect monitorability, and they name "architectures that rely on latent reasoning" among the threats [134]. However, they present it as "a complement, not a replacement" for adversarial stress testing [134]. No study reports such metrics as a function of loop count. A broader evaluation suite already tracks monitorability across reasoning effort, reinforcement-learning steps and model size [191]. It finds that "for a given capability level, it is often possible to achieve a higher monitorability by instead using a smaller model at a higher reasoning effort" [191]. The authors write that in many cases "it seems advantageous to scale solution length rather than compute per token" [191]. Recurrent depth is one way to scale compute per token. For looped models, the corresponding open question is whether reaching a capability level with more loops lowers monitorability compared with reaching it with more generated tokens.

One useful way to quantify the concern is *opaque serial depth*. It is the length of the longest computation that a model can perform without interpretable intermediate steps such as chain of thought [54]. For a diffusion language model, opaque serial depth "seems at first 28.6X higher" than for the corresponding autoregressive model [136]. However, mapping intermediate states back to tokens recovers much of the transparency [136]. The framework does not analyze depth-looped

models, although its authors note that "adding recurrence should enable long serial cognition within the activations" [54]. Applying its per-layer accounting ourselves, we find that a block of L layers run r times has the same opaque serial depth as an unshared model with rL layers. A fixed loop count therefore raises opaque serial depth by a constant factor, as added layers would. The case that is qualitatively different is a loop count that grows with the input or is unbounded at test time. Such a loop count behaves like the continuous latent chain of thought, which the framework treats separately. For looped models, no study has shown a recovery of transparency comparable to the diffusion case.

Invisible reasoning is not unique to latent architectures. Frontier models gain up to 13 percentage points from semantically irrelevant filler tokens, and they can use these tokens to satisfy a constraint that is invisible to chain-of-thought monitoring [28]. In masked diffusion language models, patching the hidden states of end-of-sequence padding "increases the models' relative likelihood of outputting the counterfactual answer" [52]. Filler-token computation is qualitatively different from the computation in a loop. Transformers with polynomially many filler tokens remain in TC^0, and small models learn to use filler tokens only from parallelizable demonstrations, with "no transfer from serial, instance-adaptive demonstrations" [391]. Filler tokens hide parallel computation from the trace, while loops add serial depth.

14.2 Reading latent computation directly

If looped models think in hidden states, monitors could read those states, but the current evidence is discouraging. A safety-judgment probe on the hidden states of Ouro degrades slowly as the number of loops increases [72]. Pre-answer success probes on the loop states of an Ouro model, combined with length and log-probability features, reach an area under the receiver operating characteristic curve (AUROC) of 0.797 on GSM8K, against 0.731 for those features alone [261]. However, "a non-looped SFT transformer reads candidate quality at 0.568 task-disjoint", so this class of readout does not depend on recurrence [261]. One study reported high preference-readout accuracy from the loop states of Ouro [260]. As noted in Section 10.3, its authors then issued an erratum that corrects its main numbers to at most 63.9% and withdraws parts of its interpretation. The erratum also says that its feasibility claims for alignment monitoring should be read against the corrected values [260]. A fitted lens on Huginn "cannot distinguish 'not held' from 'held but idle"', so the absence of a readout is not evidence that content is absent [513].

Causal interventions give only partial evidence on whether latent steps are faithful. The same lens finds steerable content in Huginn and Ouro [513], and probes with causal knockouts find a relay across loops [242], but neither study measures faithfulness in the sense used for chain of thought, that is, whether the decoded intermediate content is what drives the output. For non-looped latent reasoning, faithfulness has been analyzed along training trajectories [241]. No study builds a latent monitor for misbehavior such as deception or reward hacking on a looped model and compares it with a chain-of-thought monitor. The closest evidence comes from sequence-axis continuous thought. In a 124M Coconut-style model with an implanted backdoor, a linear probe detects the armed state [406]. The transfer accuracy of this probe "peaks at z1 (89.4%) and degrades monotonically to near-chance at z6 (58.3%)" [406]. No study has tested whether misaligned intent can be read in the same way from early loop iterations.

14.3 Capability elicitation

Section 10.2 described a rank-8 single-layer adapter that raises the multi-step reach of frozen Ouro-1.4B from about two lines to at least 160 at eight loops [242]. For oversight, this means

that a tiny edit can greatly increase the amount of unverbalized serial computation that a looped model performs per token. Evaluations of the latent capabilities of a looped model should therefore include elicitation in addition to the behavior of the released weights. Elicitation must also deal with strategic underperformance by the evaluated model. When models are prompted to sandbag and are aware of the monitor, they evade a chain-of-thought monitor on 16–36% of their successful sandbagging attempts [297]. Open models with as few as 32B parameters can do this [297]. In a looped model with adaptive halting, the chosen depth is one more channel for underperformance that leaves no trace in the text (our inference). Capability evaluations should therefore sweep over loop counts instead of reporting only the default exit.

14.4 What has not been studied

No paper in the corpus measures adversarial robustness or jailbreak susceptibility for a looped language model, and none measures refusal behavior, unlearning, membership inference or privacy (Table 18). Knowledge editing has been studied only in a controlled synthetic knowledge-graph setting. There, a single factual edit propagates only weakly through two-hop compositions in a looped Transformer, and strongly in models with an addressable shared memory, whether or not they loop [380]. For non-looped latent reasoning models, robustness is lower than for explicit chain of thought, "with particularly severe degradation under white-box attacks" [78]. For non-shared token-level layer skipping, existing adaptive models "exhibit higher harmful-response rates than their reference backbones" [306]. Looped models add a test-time control that neither setting has, namely the loop count, which is often chosen by a learned exit gate. The most urgent open question in this section is whether safety behavior is invariant to that control, and Section 16 lists it with the other open questions.

Takeaways

- Chain-of-thought monitoring relies on the depth limit of a single forward pass, and looping relaxes this limit. No study measures chain-of-thought necessity as a function of loop count.
- The one direct study finds no systematic monitorability penalty for looped models. Its within-model drop occurs at depths beyond training, and it does not test optimization pressure, which is known to move computation elsewhere in non-looped models.
- Probe-based readouts of loop states are weak, do not depend on recurrence, or (in one case) were corrected downward by an erratum.
- Robustness, refusal, unlearning and privacy have not been studied for looped language models, and knowledge editing has been studied only at toy scale. Every safety result on looped models so far is at or below 3.5B parameters.

15 Meta-Analysis: What the Evidence Supports

The preceding sections report what individual papers find. This section asks what the corpus as a whole supports. For each core paper we extracted its main claims, with a median of 4 claims per paper and at most 11. We classified each claim by the comparison regime under which it was established (Section 3.5) and by the largest model scale at which it was shown. We kept only claims whose supporting quotation passed automated verification against the text of the paper. The resulting claims ledger is released with the survey, and Section C prints an excerpt. The individual results behind the main question are described in Section 7.1, and this section combines them.

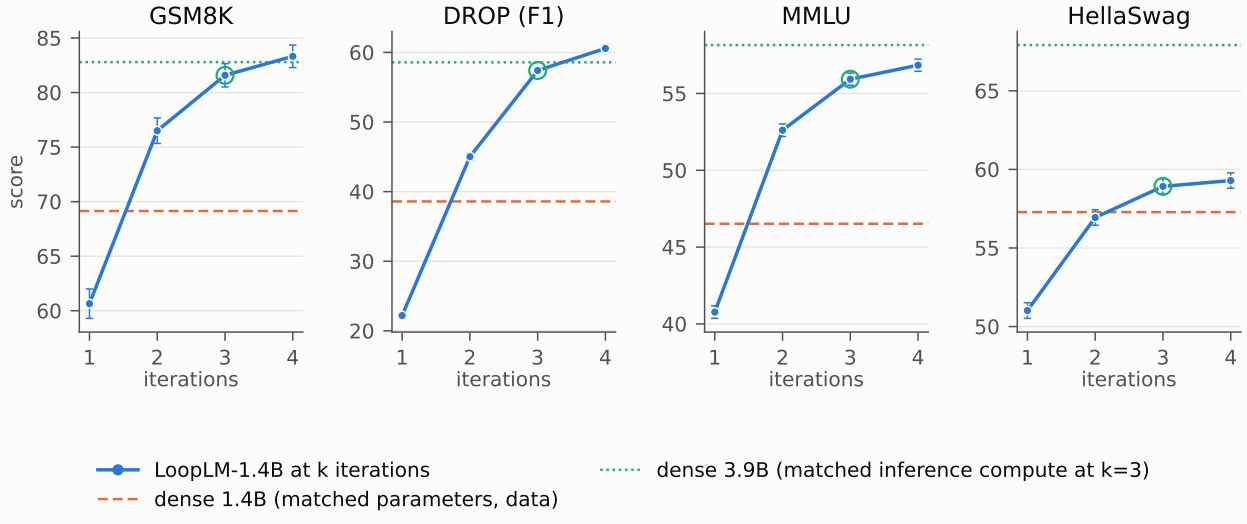


Figure 9: The regime flip in one study (data: Table 2 of [343], mean ± standard error). From three iterations on, the looped model beats its parameter-matched dense counterpart (dashed) on all four benchmarks. It ties the compute-matched dense model (dotted, with the matching point at three iterations ringed) on GSM8K and DROP, and it trails that model on MMLU and HellaSwag.

15.1 One pair of models, two verdicts

Figure 9 shows why the regime of a comparison must be stated. One study evaluates a 1.4B looped model and two dense models trained on the same data [343]. Against the dense model with the same parameters, the looped model wins on all four benchmarks shown when it runs three or more iterations. On HellaSwag, two iterations are not yet enough. Against the dense model that matches its inference compute at three iterations, the looped model ties on mathematics and reading comprehension and trails on knowledge-heavy benchmarks. Both comparisons are valid, but they answer different questions. Two readers who each see only one of them will reach opposite conclusions about the same architecture.

15.2 The shape of the evidence

The ledger contains 1211 verified claims from 334 core papers. Of these claims, 822 are empirical comparisons with an identifiable regime. A further 88 claims compare settings of a single model, for example one checkpoint run at different loop counts. These claims describe how a model behaves, but they do not say whether looping beats an alternative architecture, so we exclude them from the regime statistics below. Parameter-matched claims (459) and data-matched claims (368) are the most common. There are 69 claims matched on training compute and 96 claims matched on inference compute. Only 19% of empirical claims are matched on any notion of compute, and 31% are shown at a billion parameters or more. Figure 10 breaks the claims down by regime and scale. Claims matched on training compute, which is the regime of standard scaling-law comparisons, are the rarest kind overall and nearly absent below 100M parameters. Very few compute-matched claims of any kind are shown at ten billion parameters or more. The central question of the field is therefore being answered mostly with parameter-matched evidence, and this kind of evidence cannot answer it.

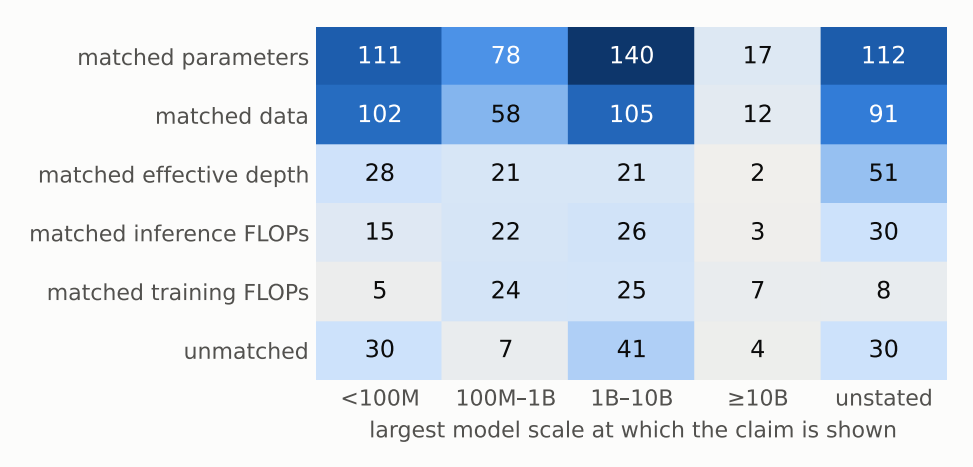


Figure 10: Verified empirical claims from core papers, by the comparison regime under which each was established and the largest model scale at which it was shown. A claim established under several regimes is counted once in each row. Source: the released claims ledger.

15.3 Is a recurrence worth a layer?

Table 19 sorts the controlled evidence on the central question of the field by what each comparison holds fixed. The table explains the apparent contradictions described in Section 1. Every negative result compares a looped model with a non-looped model that has more unique parameters at the same compute. Most positive results either keep the number of parameters (unique or total) from shrinking or come from a setting where parameters are the limiting resource. Three rows rest on results that the earlier sections do not describe in detail, and two of them are partial exceptions to this pattern. A gated recurrent Transformer with fewer unique parameters reaches lower loss than a dense baseline at matched FLOPs at small scale. The dense baseline leads at medium and large scale when both use the standard token budget [209]. A looped mixture-of-experts model with fewer total parameters overtakes a larger baseline after about 600B tokens, but the two models are matched on measured training step time and not on FLOPs [169]. The third study matches a looped and a standard mixture-of-experts model on total parameters, per-token FLOPs and training tokens, and the gain from looping grows with model scale in its experiments [82].

One interpretation is consistent with this pattern, namely that **a recurrence adds compute per parameter that is worth less than a fresh block of parameters.** Looping therefore loses whenever it is used to *replace* unique parameters at fixed compute. It wins whenever parameters are the limiting resource, whether because memory is capped, because data are limited, or because sparse experts keep total parameters constant. The SMELT authors make the same argument from the other side, writing that the iso-depth deficit "may reflect parameter loss rather than a flaw in looping itself" [507]. It is not settled how large this gap in value is beyond a few recurrences. At four recurrences, the fitted mappings in Figure 6 disagree by nearly a factor of three (0.89 versus 0.32 extra blocks, computed from the fitted constants).

15.4 Graded claims

Table 20 grades the recurring claims of the field with the five grades defined here. "Strong" means that controlled evidence supports the claim in more than one study at a billion parameters or more. A grade of "moderate" means controlled evidence at smaller scale or in one study. A grade of "mixed" means that controlled studies disagree. A grade of "weak" means that the evidence is

Table 19: When does looping help? Controlled evidence sorted by what the comparison holds fixed.

Held fixed	Effect of looping	Evidence
Unique parameters (compute grows)	positive, not always monotone in the loop count	[343, 397, 550, 604, 605]
Unique parameters and training compute (fewer tokens)	positive, saturating	[84, 397]
Training compute and effective depth, dense (unique parameters shrink)	negative	[288, 433, 604]
Training compute, each arm compute-optimal, tied vs. untied core	small constant penalty	[93]
Inference compute, effective depth and data, dense	tie on math, loss on knowledge	[343]
Per-token FLOPs, total parameters and data (MoE), plus KV cache in one study	positive	[82, 507]
Training compute and data, dense, same-recipe retraining	positive at small scale, negative at larger	[209]
FLOPs, looped vs. parallel wide sublayers	pure looping worst, mixture best	[159]
Measured training step time and data (MoE)	positive after about 600B tokens	[169]
Training compute with total parameters halved (MoE)	parity on reasoning, loss overall	[84]
Data fixed and limiting	positive	[93, 193]
Nothing (public checkpoints, different data)	positive main results, but uninformative about the architecture	[368, 604]

unmatched, proxy-based or confounded. A grade of "untested" means that no study in the corpus tests the claim.

15.5 Limitations of the ledger

The ledger has four limitations. First, the quotations behind each claim were verified automatically, but the regime labels were not coded twice by independent annotators. Second, the scale of a claim is the largest model size at which its paper shows it, so a claim shown at several sizes is counted once, at the largest. Third, papers contribute different numbers of claims, so the counts weight papers unequally. Fourth, a regime label records what the paper reports or what its setup implies. When a paper does not report enough to decide, the claim is marked unclear and excluded, which applies to 21 claims. Matched-compute evidence may therefore be slightly undercounted, because papers that match compute without saying so cannot be identified.

15.6 Recommendations

The analysis suggests three changes to research practice. First, state the regime of every comparison and report the quantities that let readers work out the comparison under the other regimes. These

Table 20: Recurring claims about looped models, graded against the corpus.

Claim	Grade	Basis
Looping does not add knowledge storage	moderate	equal bits per parameter at matched parameters in small models [604, 605]; knowledge gap grows with r when parameters shrink [433]; no deficit when total parameters are matched [507]
Looping improves reasoning at matched parameters	moderate	synthetic tasks and 1–3B models [343, 426, 604]
Looping improves reasoning at matched compute	mixed	positive for MoE [84, 507]; no signal for small dense models [433]
A looped model of size N matches a standard model of size kN	weak as stated	true at matched parameters with more compute; false at matched compute for dense models; the prominent versions are unmatched [433, 604]
Running more loops than trained improves results	weak at language-model scale	degradation or plateau beyond the training depth [397, 604] (Figure 7); positive mainly on synthetic tasks with specific recipes (Section 9.4); one tabular model works at about twice its mean training loop count (Section 12)
Recursion explains the puzzle results of small recursive models	mixed	tied recursion beats unshared depth within the HRM/TRM pipeline [168, 218]; a feedforward model with test-time training scores higher on ARC-AGI-1 [215]; reruns vary widely (Figure 8)
Looped models length-generalize	moderate (synthetic only)	recipes work on algorithmic tasks [38, 146]; high seed variance [275]; the standard recipe collapses out of distribution on some tasks [228]
Adaptive depth saves compute	weak as measured	mostly proxy savings; measured gains need engine support (1.3–1.5× in batched serving, up to 2.06× decode throughput for Mixture-of-Recursions [25]), and without it there is no gain or a slowdown relative to a non-looped model [162, 331, 434]; no comparison with fixed-loop models retrained at the same compute (Section 5.5)
Learned halting beats simple rules	mixed	convergence and confidence rules match or beat learned gates at small scale and are competitive on Ouro checkpoints [331, 395]; supervised deciders help [561]
Looped models are less monitorable	weak	one study, ≤3.5B; within-model drop at extrapolation depths; no cross-architecture penalty [502]
Looping reduces the need for a visible chain of thought	untested	follows from the depth limit of a forward pass (Section 14.1); not measured
Looped models are less robust or less safely aligned	untested	no study (Table 18)

quantities are parameters, effective depth, training and inference FLOPs, and data (Section B). Second, report quality over a range of loop counts that includes counts beyond the training horizon, and do not report quality only at a single best depth. Third, prefer controlled baselines trained on the same data over public checkpoints. A comparison against a model trained on different data measures the training pipeline as much as the architecture.

Takeaways

The same pair of models can win at matched parameters and tie or lose at matched inference compute, so a result can only be read once its comparison regime is known. Only 19% of the verified empirical claims are matched on any notion of compute, and 31% are shown at a billion parameters or more. Sorted by regime, the controlled evidence supports the reading that a recurrence adds compute per parameter that is worth less than a fresh block of parameters. Looping therefore loses when it replaces unique parameters at fixed compute, and it wins when parameters are the limiting resource. Most recurring claims in Table 20 have moderate, mixed or weak support, and the claims about safety are weak or untested.

16 Open Problems and Research Agenda

This section turns the gaps found in the meta-analysis and in the preceding sections into a research agenda. A survey of a field this young should state which topics the field has *not* yet studied. In Table 21, we count the core papers whose title or abstract addresses topics that matter for deploying looped models. We count them first by keyword match and then after a manual audit of every hit, and the audit notes are released with the corpus. Several topics with large literatures for standard Transformers have no dedicated study for looped models at the time of our snapshot. We then list the problems that we consider most important, each with the measurement that would settle it.

16.1 Measurement and accounting

P1. A reporting standard for comparisons. Section 15 shows that the comparison regime of a claim is often implicit and that compute-matched evidence is a minority of the empirical record. We propose that every empirical paper on looped models report the quantities in the checklist of Section B, which expands the recommendations at the end of that section. The checklist asks for unique parameters, active parameters per iteration and effective depth. It asks for training FLOPs and tokens, and for inference FLOPs per token at each evaluated loop count. It also asks for the KV-cache memory per token, and for wall-clock latency and throughput on stated hardware in place of FLOP proxies or average depth alone. Finally, it asks for the regime of every baseline (Section 3.5) and for the full quality curve over loop counts below, at and beyond the training horizon. For adaptive depth, the baselines should include a fixed-loop model retrained at the same realized compute.

P2. Value of a recurrence at frontier scale. Compute-matched evidence for dense models comes from runs of at most 47B training tokens [433]. For mixture-of-experts models, it comes from models with at most 1.6B active parameters [507]. It is not known whether dense loops close the gap with scale or whether sparse experts inside the loop are the decisive ingredient. It is also open how the recurrence-equivalence exponent φ of Eq. (9) changes with model size, data and training method. A shared benchmark suite that is compute-matched and has public baselines would settle more of these questions than another architecture variant would.

16.2 Reliability of depth

P3. Depth extrapolation at language-model scale. The ability to run more iterations than were seen in training is the property that distinguishes looped models from deep ones. However, models do not reliably stay stable beyond the training horizon (Section 6.2), and gains at language-model scale saturate at or before the training depth (Section 7.2). The strong extrapolation results

Table 21: Coverage of deployment-relevant topics among the 560 core papers. "Keyword hits": papers whose title or abstract matches the topic's patterns. "Audited": hits whose main subject is the topic *in looped models*, after manual review. The audited adversarial-robustness paper trains recursive reasoning models on puzzles with an adversarial objective and tests robustness to distribution shift (Section 6.1). It does not study looped language models.

Topic	Keyword hits	Audited
Knowledge editing / unlearning	0	0
Fairness / bias	0	0
Energy / carbon accounting	1	0
Continual / test-time training	2	0
Safety alignment / jailbreaks / refusal	3	0
Long context / retrieval	5	0
Memorization / privacy	6	0
Hallucination / calibration / uncertainty	12	0
Adversarial robustness	4	1
Monitorability / faithfulness	6	1
Multilingual	5	2
Data curricula for recurrence	10	3
Agents / tool use	13	3

come from synthetic tasks, and even there they vary a lot across seeds (Section 9.4). The field has no standard protocol (tasks, horizons, metrics) and no predictive account of when extrapolation helps, saturates or degrades.

P4. Depth as a calibrated signal. Per-token convergence of the recurrent state is directly measurable and is already used to halt computation (Section 5). The loop trajectory (convergence speed, successive-iteration divergence, per-iteration entropy) is available at no extra cost. However, no paper in the corpus uses it to detect hallucinations or to calibrate confidence and abstention (Table 21).

16.3 Safety and governance

P5. Alignment under variable depth. Looped models expose a test-time control that standard models lack, namely the number of iterations, which is often chosen by a learned exit gate. For looped models, no study tests whether refusal and other safety behaviors are invariant to loop count, whether reduced depth weakens them, or whether inputs can manipulate exit gates (Table 21). For non-shared token-level layer skipping, existing token-level adaptive models "exhibit higher harmful-response rates than their reference backbones" [306], which makes the question urgent.

P6. Monitorability under optimization pressure. The only study of chain-of-thought monitorability in looped language models evaluates existing checkpoints. It finds task-dependent reductions under stress tests at loop counts beyond the training depth, but "no evidence that LoopLMs are systematically less monitorable" than matched non-looped models [502]. For oversight, the relevant question is what happens when training *rewards* moving decisive computation out of the visible trace and into latent iterations [155, 271], and this question is open. The design of monitors that read the loop trajectory itself is also open (Section 14). Three inexpensive experiments would already give useful evidence. The first measures chain-of-thought legibility and coverage [134] as a function of loop count at fixed accuracy. The second measures hint-verbalization faithfulness [85]

below, at and beyond the training loop count. The third uses probes for implanted misalignment in early loop iterations, which are the depth-axis analogue of early-latent-step probes in continuous-thought models [406]. A fourth experiment would measure chain-of-thought necessity itself as a function of loop count. A task makes chain of thought necessary when it cannot be solved without a visible trace [133], and the measurement would be the share of a task suite that has this property. For optimization pressure, an existing protocol rewards solving a task without stating a required intermediate value [133]. On a looped model with a free loop count, the protocol would show whether the computation moves into other parts of the trace, into the final answer [191], or into the iterations.

P7. Editing, unlearning and privacy. Locate-and-edit methods assume that a fact is stored in a layer that is applied once. In a looped model, the edited block is applied at every iteration, so the effect of an edit compounds with depth. Apart from one toy-scale study of knowledge editing (Table 18), we found no study of knowledge editing, unlearning, memorization or privacy attacks on looped models (Table 21).

16.4 Capabilities

P8. Long context. Looping multiplies KV memory per token unless caches are shared (Eq. (7)), and KV memory is exactly the resource that long contexts consume. No study has directly examined how looping interacts with long-context retrieval and reasoning quality (Table 21).

P9. Agents, tools and long horizons. Evidence that looping helps compositional tool use is recent and limited [396]. It is open whether latent depth substitutes for, complements or competes with explicit multi-step interaction in agentic settings.

P10. Multilinguality. Latent computation does not depend on the language of a written trace, which suggests that looped models could narrow reasoning gaps for low-resource languages. We found no study that tests this idea (Table 21).

P11. Training data for loops. Recurrence helps some tasks and hurts others (Section 11). Data mixtures and synthetic algorithmic data for looped pretraining are barely explored. Only curricula over depth have dedicated studies [348].

16.5 Systems and theory

P12. Serving variable-depth models. Depth-adaptive batching, cache sharing and self-speculation exist as separate techniques (Section 8). An end-to-end serving stack is missing, and so is energy accounting at matched quality.

P13. From separations to scale. Theory establishes what loops can express and where they provably differ from chain of thought (Section 9). It is largely untested which of these separations appear in trained language models, and at what scale.

A minimal agenda

1. Report the comparison regime, train and inference FLOPs, KV memory and wall-clock time for every result (P1).
2. Measure φ at $\geq$10B parameters, for dense and sparse models (P2).
3. Test safety behavior across loop counts and under exit-gate manipulation (P5).
4. Apply optimization pressure against a monitor to a looped model, and measure where the computation goes (P6).
5. Evaluate editing, unlearning and extraction attacks on weight-tied models (P7).

17 Resources

This section lists the released models, code and data that support further work on looped models, including the data behind this survey.

Open checkpoints. Table 22 lists looped language models with publicly released weights, as linked from the papers themselves. Most mechanistic, systems and safety studies in this survey use the models in the first two rows. This concentration of evidence on Huginn and Ouro is itself a limitation, because architecture and training recipe are confounded across the two models (Sections 8.1 and 10.1).

Code. Among the extraction records, 196 link to released code and 22 link to released weights. The full list is in the released corpus, and each link in it has been checked against the paper's text.

This survey's data. For every paper in the corpus, we release the screening decision and its reason. For every core paper, we release the extraction record with taxonomy labels, comparison regimes, claims and numbers. Each of these items comes with a verbatim quotation and its location. We also release the claims ledger of Section 15 and the scripts that regenerate every count, figure and table in this paper from those files. We intend to keep maintaining and updating the corpus, and we welcome additions.

Table 22: Looped language models with released weights (links as given in each paper).

Model	Design	Scale and data	Weights
Huginn [175]	prelude–core–coda (2,4,2), random initial state, concatenation injection	3.5B; 800B tokens	huggingface.co/tomg-group-umd/huginn
Ouro [604]	whole-stack loop, four steps, learned exit gate	1.4B and 2.6B; 7.7T tokens	ouro-llm.github.io
Nanbeige4.2-3B [368]	two passes over the full stack	3B non-embedding; 28T tokens	huggingface.co/Nanbeige/Nanbeige4.2-3B
LoopCoder-v2 [550]	parallel loop Transformer, shared first-loop KV	7B; 18T tokens	huggingface.co/Multilingual-Multimodal-NLP/LoopCoder-V2
Intern-S2-Mobius [75]	shared feed-forward memory queried by attention reasoners	7B (from scratch); 35B (continual)	huggingface.co/internlm/Intern-S2-Mobius
Retrofitted recurrence [348]	dense 1B models converted to prelude–core–coda	parent models of about 1B	huggingface.co/collections/tomg-group-umd/retrofitting-recurrence
PonderLM [575]	probability-weighted embedding fed back per token	up to 2.8B; 300B tokens	huggingface.co/zeng123/PonderingPythia-2.8B
LT2 Ouro-hybrid [121]	Ouro converted to linear-time looped attention	1.4B	huggingface.co/chili-lab/Ouro-hybrid-1.4B
DFM Mimir v1 [428]	hierarchical reasoning model for text	1.3B non-embedding	huggingface.co/danish-foundation-models/DFM-Mimir
ModernALBERT [382]	recursive encoder with mixture of LoRAs	50M–120M	huggingface.co/collections/nlpie/modern-recursive-transformers

18 Conclusion

Looped language models reuse a block of parameters for several consecutive passes over the same tokens. As a result, effective depth can be chosen separately from the parameter count. Often it can also be chosen separately at training time and at inference time. In two years, the idea has moved from synthetic tasks to trillion-token pretraining and to released model families. According to press reports, it is also used in a frontier-lab model. This survey organized 560 papers on the topic around a single template and an explicit account of how parameters, compute, memory and depth are counted. The survey then asked what the evidence in these papers supports.

Our main answer is that a recurrence is worth less than a fresh block of parameters, although it still has some value. Looping hurts when it replaces unique parameters at fixed compute. It helps when parameters are the limiting resource, which is the case under a memory budget, with limited data, or with sparse experts that keep total parameters constant (Section 15.3). Most apparent contradictions in the field are resolved once each comparison is labeled by what it holds fixed. Beyond this answer, the evidence comes with more qualifications than the most visible claims in the field suggest. Gains from extra iterations saturate near the training depth at language-model scale (Section 7.2). Extrapolation in depth does occur on synthetic algorithmic tasks, but it varies a lot across seeds and tasks (Section 9.4). Adaptive depth saves compute mostly in proxy measures, unless the serving stack is built for it (Sections 5.5 and 8.3). Theory explains what loops can express much better than it explains what training makes them compute (Section 9.5). Almost nothing is known about the safety of looped models. There is one study of monitorability and none of robustness, alignment or privacy. Knowledge editing has been studied only at toy scale (Table 18). Every safety result is at or below 3.5B parameters.

We therefore think that, in the near term, better measurements would contribute more than new architectures. These measurements are compute-matched comparisons at scale, quality curves over loop counts, savings measured in wall-clock time instead of proxies, and the safety evaluations

listed in Section 16. We hope the formalism, the claims ledger and the released corpus make those measurements easier to design and to compare.

References

[1] Valentin Abadie, Clemens Hutter, and Helmut Bölcskei. Recurrent neural networks approximate continuous functions. arXiv preprint arXiv:2606.20325, 2026. URL https://arxiv.org/abs/2606.20325.

[2] Samira Abnar, Omid Saremi, Laurent Dinh, Shantel Wilson, Miguel Angel Bautista, Chen Huang, Vimal Thilak, Etai Littwin, Jiatao Gu, Josh Susskind, and Samy Bengio. Adaptivity and Modularity for Efficient Generalization Over Task Complexity. arXiv preprint arXiv:2310.08866, 2023. URL https://arxiv.org/abs/2310.08866.

[3] Umar Abubacar, Roman Bauer, and Diptesh Kanojia. Parameter-Efficient Quality Estimation via Frozen Recursive Models. arXiv preprint arXiv:2603.14593, 2026. URL https://arxiv.org/abs/2603.14593.

[4] Veeti Ahvonen, Damian Heiman, Antti Kuusisto, and Carsten Lutz. Logical Characterizations of Recurrent Graph Neural Networks with Reals and Floats. In *Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.14606.

[5] Hao Ai, Ying Liu, Jie Fang, and Sheikh Faisal Rashid. A Method with Universal Transformer for Multimodal Sentiment Analysis. International Conference on Artificial Intelligence and Pattern Recognition, 2022.

[6] Jiabao Ai, Peng Han, Yuchen Song, and Zhengjun Yue. Depth through recurrence: Looped transformers for flow-matching TTS. arXiv preprint arXiv:2609.29768, 2026. URL https://arxiv.org/abs/2609.29768.

[7] Xi Ai and Bin Fang. Leveraging Relaxed Equilibrium by Lazy Transition for Sequence Modeling. Annual Meeting of the Association for Computational Linguistics (ACL), 2022.

[8] Yongbao Ai and Tianxiang Gao. MoR-ST: content-adaptive recursive computation for efficient vision transformers. The Visual Computer, 2026.

[9] Yongbao Ai, Tianxiang Gao, Zhipeng Lin, Longqi Yang, and Qingyu Chang. MoR–Swin: Efficient Vision Transformer Using Mixture of Recursions. Journal of Imaging, 2026.

[10] Ange-Clément Akazan, Abdoulaye Koroko, Verlon Roel Mbingui, Choukouriyah Arinloye, Hassan Fifen, and Rose Bandolo. Vision Tiny Recursion Model (ViTRM): Parameter-Efficient Image Classification via Recursive State Refinement. arXiv preprint arXiv:2603.19503, 2026. URL https://arxiv.org/abs/2603.19503.

[11] Haider Al-Tahan, Sean O'Brien, Anastasia Razdaibiedina, and N. Apurva Ratan Murty. LAST: Looped Audio Spectrogram Transformer. arXiv preprint arXiv:2610.01926, 2026. URL https://arxiv.org/abs/2610.01926.

[12] Ibrahim Alabdulmohsin and Xiaohua Zhai. Recursive Inference Scaling: A Winning Path to Scalable Inference in Language and Multimodal Systems. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.07503.

[13] Preslav Aleksandrov, Meghdad Kurmanji, Fernando Garcia Redondo, David O'Shea, William Shen, Alex Iacob, Lorenzo Sani, Xinchi Qiu, Nicola Cancedda, and Nicholas D. Lane. AbbIE: Autoregressive Block-Based Iterative Encoder for Efficient Sequence Modeling. arXiv preprint arXiv:2507.08567, 2025. URL https://arxiv.org/abs/2507.08567.

[14] Howaida Allam and Inam Ullah Khan. Weight-Tied Adaptive Recursive Vision–Language–Action Transformer for Efficient Multimodal Robotic Control. Journal of Smart Algorithms and Applications (JSAA), 2026.

[15] Md Zahangir Alom, Mahmudul Hasan, Chris Yakopcic, Tarek M. Taha, and Vijayan K. Asari. Recurrent Residual Convolutional Neural Network based on U-Net (R2U-Net) for Medical Image Segmentation. arXiv preprint arXiv:1802.06955, 2018. URL https://arxiv.org/abs/1802.06955.

[16] Awni Altabaa, Siyu Chen, John Lafferty, and Zhuoran Yang. Unlocking Out-of-Distribution Generalization in Transformers via Recursive Latent Space Reasoning. arXiv preprint arXiv:2510.14095, 2025. URL https://arxiv.org/abs/2510.14095.

[17] Alireza Amiri, Xinting Huang, Mark Rofin, and Michael Hahn. Lower Bounds for Chain-of-Thought Reasoning in Hard-Attention Transformers. In *International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2502.02393.

[18] Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, Zico Kolter, and Roger Grosse. Path Independent Equilibrium Models Can Better Exploit Test-Time Computation. In *Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2211.09961.

[19] Arwansyah, Suryani, Hasyrif SY, Nurdiansah, and Ahyuna. Implementasi Hierarchical Reasoning Model (HRM) Pada Kasus Klasifikasi. SISITI : Seminar Ilmiah Sistem Informasi dan Teknologi Informasi, 2026.

[20] Arwansyah Arwansyah, Suryani Suryani, Hasyrif Sy, and Nurdiansah Nurdiansah. Implementasi Konsep Tiny Recursion Model Pada Kasus Klasifikasi. e-Jurnal JUSITI (Jurnal Sistem Informasi dan Teknologi Informasi), 2025.

[21] Arip Asadulaev, Rayan Banerjee, Fakhri Karray, and Martin Takac. Latent Reasoning in TRMs is Secretly a Policy Improvement Operator. arXiv preprint arXiv:2511.16886, 2025. URL https://arxiv.org/abs/2511.16886.

[22] Arnold Caleb Asiimwe, William Yang, Sanghyuk Chun, Esin Tureci, and Olga Russakovsky. Depth as Time in One-Step Generative Models. arXiv preprint arXiv:2610.03626, 2026. URL https://arxiv.org/abs/2610.03626.

[23] Sangmin Bae. Accelerating Large Language Model Inference via Early-Exiting Algorithms. arXiv preprint arXiv:2509.05915, 2025. URL https://arxiv.org/abs/2509.05915.

[24] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2410.20672.

[25] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2507.10524.

[26] Euijin Baek, Housam Babiker, Mi-Young Kim, and Randy G. Goebel. TRM-Planner: Offline Target Planning and Distillation for Tiny Recursive Models. Annual Meeting of the Association for Computational Linguistics, 2026.

[27] Junyeob Baek, Mingyu Jo, Minsu Kim, Mengye Ren, Yoshua Bengio, and Sungjin Ahn. Generative Recursive Reasoning. arXiv preprint arXiv:2605.19376, 2026. URL https://arxiv.org/abs/2605.19376.

[28] Vatsal Baherwani, Tom Goldstein, and Ashwinee Panda. Not All LLM Reasoning is Visible in the Chain-of-Thought. arXiv preprint arXiv:2607.22925, 2026. URL https://arxiv.org/abs/2607.22925.

[29] Runci Bai, Yucheng Xin, Pu Wang, Yongcong Wang, Chen Wu, Dianjie Lu, Guijuan Zhang, Pengwen Dai, Guangwei Gao, Siyuan Yao, and Zhuoran Zheng. Loop-Mamba: A Loop Mamba with Degradation-Aware and Shared Memory for Old Photo Restoration. arXiv preprint arXiv:2608.02346, 2026. URL https://arxiv.org/abs/2608.02346.

[30] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep Equilibrium Models. In *Neural Information Processing Systems*, 2019. URL https://arxiv.org/abs/1909.01377.

[31] Shaojie Bai, Zhengyang Geng, Yash Savani, and J. Zico Kolter. Deep Equilibrium Optical Flow Estimation. In *Computer Vision and Pattern Recognition*, 2022. URL https://arxiv.org/abs/2204.08442.

[32] Xingjian Bai and Luke Melas-Kyriazi. Fixed Point Diffusion Models. In *Computer Vision and Pattern Recognition*, 2024. URL https://arxiv.org/abs/2401.08741.

[33] Aaron Baier-Reinio and Hans De Sterck. N-ODE Transformer: A Depth-Adaptive Variant of the Transformer Using Neural Ordinary Differential Equations. arXiv preprint arXiv:2010.11358, 2020. URL https://arxiv.org/abs/2010.11358.

[34] Nikita Balagansky and Daniil Gavrilov. PALBERT: Teaching ALBERT to Ponder. arXiv preprint arXiv:2204.03276, 2022. URL https://arxiv.org/abs/2204.03276.

[35] Amir Rezaei Balef and Katharina Eggensperger. LoopICL: Looping a single transformer block to solve tabular tasks. arXiv preprint arXiv:2609.36108, 2026. URL https://arxiv.org/abs/2609.36108.

[36] Amir Rezaei Balef, Mykhailo Koshil, and Katharina Eggensperger. Is One Layer Enough? Understanding Inference Dynamics in Tabular Foundation Models. arXiv preprint arXiv:2605.06510, 2026. URL https://arxiv.org/abs/2605.06510.

[37] Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to Ponder. arXiv preprint arXiv:2107.05407, 2021. URL https://arxiv.org/abs/2107.05407.

[38] Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end Algorithm Synthesis with Recurrent Networks: Logical

Extrapolation Without Overthinking. arXiv preprint arXiv:2202.05826, 2022. URL https://arxiv.org/abs/2202.05826.

[39] Hieu Tran Bao, Nguyen Cong Dat, Nguyen Duc Anh, and Hoang Thanh-Tung. Learning to Stop Overthinking at Test Time. arXiv preprint arXiv:2502.10954, 2025. URL https://arxiv.org/abs/2502.10954.

[40] Hieu Tran Bao, Phung Thanh Dang, Pham Quang Nhat Minh, and Hoang Thanh Tung. DART: Distributional Adversarial Recurrent Training for Algorithm Learning. arXiv preprint arXiv:2609.05988, 2026. URL https://arxiv.org/abs/2609.05988.

[41] Paweł Batorski, Abtin Pourhadi, Akylgali Aitaza, Przemysław Spurek, and Paul Swoboda. MACRO: Markov Chain Routing of Transformer Layers. arXiv preprint arXiv:2608.05872, 2026. URL https://arxiv.org/abs/2608.05872.

[42] Jay Bear, Adam Prügel-Bennett, and Jonathon Hare. Rethinking Deep Thinking: Stable Learning of Algorithms using Lipschitz Constraints. In *Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2410.23451.

[43] Befekadu Bekuretsion, Wolfgang Menzel, and Solomon Teferra. Optimization-driven parameter sharing via neural architecture search for transformer models in machine translation. Discover Computing, 2026.

[44] Timo Bertram, Sidhant Bhavnani, Richard Freinschlag, Erich Kobler, Andreas Mayr, and Günter Klambauer. G-RRM: Guiding Symbolic Solvers with Recurrent Reasoning Models. arXiv preprint arXiv:2607.02491, 2026. URL https://arxiv.org/abs/2607.02491.

[45] Soham Bhattacharjee, Dushyant Singh Chauhan, Salem Lahlou, Martin Takac, and Nils Lukas. Entropy-Gated Latent Recursion. arXiv preprint arXiv:2606.16620, 2026. URL https://arxiv.org/abs/2606.16620.

[46] Manjot Bilkhu, Siyang Wang, and Tushar Dobhal. Attention is all you need for Videos: Self-attention based Video Summarization using Universal Transformers. arXiv preprint arXiv:1906.02792, 2019. URL https://arxiv.org/abs/1906.02792.

[47] Eden Biran, Daniela Gottesman, Sohee Yang, Mor Geva, and Amir Globerson. Hopping Too Late: Exploring the Limitations of Large Language Models on Multi-Hop Queries. In *Conference on Empirical Methods in Natural Language Processing*, 2024. URL https://arxiv.org/abs/2406.12775.

[48] Hugh Blayney, Álvaro Arroyo, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, Michael M. Bronstein, and Xiaowen Dong. A Mechanistic Analysis of Looped Reasoning Language Models. arXiv preprint arXiv:2604.11791, 2026. URL https://arxiv.org/abs/2604.11791.

[49] Giacomo Bocchese, Nicola Giacobbo, Etienne Guichard, James Wiles, and Akshaj Devireddy. Emergent Models: Intelligence from Tiny Substrates. arXiv preprint arXiv:2608.14019, 2026. URL https://arxiv.org/abs/2608.14019.

[50] Robert-Mihai Botarleanu, Mihai Dascălu, Scott A. Crossley, and Danielle S. McNamara. Sequence-to-Sequence Models for Automated Text Simplification. Lecture notes in computer science, 2020.

[51] Dimitrios Bralios, Efthymios Tzinis, Gordon Wichern, Paris Smaragdis, and Jonathan Le Roux. Latent Iterative Refinement for Modular Source Separation. In *IEEE International Conference on Acoustics, Speech, and Signal Processing*, 2022. URL https://arxiv.org/abs/2211.11917.

[52] Sarah Breckner and Sebastian Schuster. Masked diffusion LLMs can use EoS tokens for hidden reasoning. arXiv preprint arXiv:2603.05197, 2026. URL https://arxiv.org/abs/2603.05197.

[53] Nathan Breslow. Channel-Wise MLPs Improve the Generalization of Recurrent Convolutional Networks. arXiv preprint arXiv:2508.08298, 2025. URL https://arxiv.org/abs/2508.08298.

[54] Jonah Brown-Cohen, David Lindner, and Rohin Shah. Quantifying the Necessity of Chain of Thought through Opaque Serial Depth. arXiv preprint arXiv:2603.09786, 2026. URL https://arxiv.org/abs/2603.09786.

[55] Matthias M. M. Buehlmaier. LayerNorm as Implicit Gain Control in Looped Transformers. arXiv preprint arXiv:2607.10681, 2026. URL https://arxiv.org/abs/2607.10681.

[56] Alessandro Burzio, Tobias Fischer, Sven Elflein, Qunjie Zhou, Riccardo de Lutio, Jiawei Ren, Jiahui Huang, Shengyu Huang, Marc Pollefeys, Laura Leal-Taixé, Zan Gojcic, and Haithem Turki. Déjà View: Looping Transformers for Multi-View 3D Reconstruction. arXiv preprint arXiv:2605.30215, 2026. URL https://arxiv.org/abs/2605.30215.

[57] Eivinas Butkus, Kedar Garzón Gupta, and Nikolaus Kriegeskorte. Growing a Neural Network in Breadth, Depth, and Time. arXiv preprint arXiv:2605.25174, 2026. URL https://arxiv.org/abs/2605.25174.

[58] Michal Byra, Pawel Olszowiec, Grzegorz Stefanski, Grzegorz Gruszczynski, and Alberto Presta. Investigating Single-Block Recurrence in Vision Transformers for Image Recognition. arXiv preprint arXiv:2605.10661, 2026. URL https://arxiv.org/abs/2605.10661.

[59] Shengbo Cai, Yuxiang Wang, Jingran Xie, Zhisheng Zhang, Shun Lei, Di Cao, Teddy Sun, and Zhiyong Wu. Thinking in Depth, Speaking Directly: Recurrent Latent Reasoning for Paralinguistically Grounded Spoken Dialogue. arXiv preprint arXiv:2609.37818, 2026. URL https://arxiv.org/abs/2609.37818.

[60] Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T^2MLR: Transformer with Temporal Middle-Layer Recurrence. arXiv preprint arXiv:2607.15178, 2026. URL https://arxiv.org/abs/2607.15178.

[61] Ege Çakar, Ketan Ali Raghu, and Lia Zheng. LASER: Low-Rank Activation SVD for Efficient Recursion. arXiv preprint arXiv:2604.17224, 2026. URL https://arxiv.org/abs/2604.17224.

[62] David Calhas and Arlindo L. Oliveira. Deep Recurrence for Dynamical Segmentation Models. arXiv preprint arXiv:2507.10143, 2025. URL https://arxiv.org/abs/2507.10143.

[63] Chris Cameron, Wangzheng Wang, Nikita Ivanov, Ashmita Bhattacharyya, Didier Chételat, and Yingxue Zhang. One Step Forward and K Steps Back: Better Reasoning with Denoising Recursion Models. arXiv preprint arXiv:2604.18839, 2026. URL https://arxiv.org/abs/2604.18839.

[64] Chad A. Capps. CART: Context-Anchored Recurrent Transformer – A Parameter-Efficient Architecture with Learned Stability. arXiv preprint arXiv:2606.01495, 2026. URL https://arxiv.org/abs/2606.01495.

[65] Alba Carballo-Castro, Julianna Piskorz, Paulius Rauba, Mihaela van der Schaar, and Pascal Frossard. Recursive Scaling in Masked Diffusion Models. arXiv preprint arXiv:2606.18022, 2026. URL https://arxiv.org/abs/2606.18022.

[66] Simone Carnemolla, Salvatore Calcagno, Daniela Giordano, Concetto Spampinato, and Matteo Pennisi. PERL: Parameter Efficient Reasoning in CLIP Latent Space. arXiv preprint arXiv:2605.18464, 2026. URL https://arxiv.org/abs/2605.18464.

[67] Stefano Carotti, Marco Pacini, Alessio Gravina, Davide Bacciu, Bruno Lepri, and Sebastiano Bontorin. Graph Hierarchical Recurrence for Long-Range Generalization. arXiv preprint arXiv:2605.18387, 2026. URL https://arxiv.org/abs/2605.18387.

[68] Matheus Lima Castro, Cesar A. M. da Silva, Davi De Oliveira Costa, Lucas V. Souza, Felix M. Escalante, and Leopoldo Lusquino Filho. A Tiny Recursive Multi-Head Model for Semantic Traffic Sign Recognition in Autonomous Robotics. 2026 Brazilian Conference on Robotics (CROS), 2026.

[69] Ayushi Chadha. When to Re-Plan: Subgoal Persistence in Hierarchical Latent Reasoning. arXiv preprint arXiv:2606.03741, 2026. URL https://arxiv.org/abs/2606.03741.

[70] Mostafa Haghir Chehreghani. Persistent Gaussian Perturbations Prevent Oversmoothing in Recurrent Graph Neural Networks. arXiv preprint arXiv:2607.28185, 2026. URL https://arxiv.org/abs/2607.28185.

[71] Bo Chen, Xiaoyu Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. Bypassing the Exponential Dependency: Looped Transformers Efficiently Learn In-context by Multi-step Gradient Descent. In *International Conference on Artificial Intelligence and Statistics*, 2024. URL https://arxiv.org/abs/2410.11268.

[72] Guanxu Chen, Dongrui Liu, and Jing Shao. Loop as a Bridge: Can Looped Transformers Truly Link Representation Space and Natural Language Outputs? arXiv preprint arXiv:2601.10242, 2026. URL https://arxiv.org/abs/2601.10242.

[73] Hongbo Chen, Guohua Lu, Ting Dang, and Hong Jia. Not Too Hard, Not Too Easy: Learning from Intermediate States for LLM Structured Reasoning. arXiv preprint arXiv:2609.33149, 2026. URL https://arxiv.org/abs/2609.33149.

[74] Hung-Hsuan Chen. Thinking Deeper, Not Longer: Memory-Efficient Test-Time Reasoning with Depth-Recurrent Transformers for Compositional Generalization. arXiv preprint arXiv:2603.21676, 2026. URL https://arxiv.org/abs/2603.21676.

[75] Kai Chen, Jifeng Ding, Ning Ding, Jiaye Ge, Lixin Gu, Yicheng Gu, Qipeng Guo, Ermo Hua, Haian Huang, Haozheng Hou, Jie Hou, Xiangyu Hong, Che Jiang, Minxi Jin, Cheng Liang, Dahua Lin, Dawei Liu, Kuikun Liu, Chengqi Lv, Haijun Lv, Han Lv, Ningsheng Ma, Biqing Qi, Jianmin Qian, Shiya Su, Youbang Sun, Huanze Tang, Zhongbo Tian, Hanjing Wang, Rui Wang, Ting Wang, Yi Wang, Baiting Wu, Jun Xu, Bowen Yang, Hui Wang, Weida Wang, Haochen Ye, Jiashuo Yu, Shan Yu, Xiaoyi Yu, Qirui Zeng, Qi Zhang, Ming Zhang, Wenwei Zhang, Bowen Zhou, and Xinyu Zhou. Intern-S2-Mobius: Foundation Model

with Decoupled Knowledge and Reasoning. arXiv preprint arXiv:2608.14290, 2026. URL https://arxiv.org/abs/2608.14290.

[76] Lizhang Chen, Jonathan Li, Chen Liang, Ni Lao, and Qiang Liu. Training-Free Looped Transformers. arXiv preprint arXiv:2605.23872, 2026. URL https://arxiv.org/abs/2605.23872.

[77] Mouxiang Chen, Binyuan Hui, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Jianling Sun, Junyang Lin, and Zhongxin Liu. Parallel Scaling Law for Language Models. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.10475.

[78] Shaolong Chen, Ang Li, Mingjie Li, and Yisen Wang. Assessing Adversarial Robustness of Latent Reasoning Models. arXiv preprint arXiv:2609.22228, 2026. URL https://arxiv.org/abs/2609.22228.

[79] Tieyuan Chen, Xiaodong Chen, Haoxing Chen, Zhenzhong Lan, Weiyao Lin, and Jianguo Li. DND: Boosting Large Language Models with Dynamic Nested Depth. arXiv preprint arXiv:2510.11001, 2025. URL https://arxiv.org/abs/2510.11001.

[80] Wei-Rui Chen, Samar M. Magdy, Chiyu Zhang, Wenhui Zhu, Zhipeng Wang, and Muhammad Abdul-Mageed. LatentMT: Machine Translation with Latent Reasoning. arXiv preprint arXiv:2607.18618, 2026. URL https://arxiv.org/abs/2607.18618.

[81] Weize Chen, Xiaoyue Xu, Xu Han, Yankai Lin, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Boosting Inference Efficiency: Unleashing the Power of Parameter-Shared Pre-trained Language Models. In *Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2310.12818.

[82] Wenkai Chen, Tianshu Li, Wenyong Huang, Yichun Yin, Lifeng Shang, and Chengwei Qin. LoopMoE: Unifying Iterative Computation with Mixture-of-Experts for Language Modeling. arXiv preprint arXiv:2606.04438, 2026. URL https://arxiv.org/abs/2606.04438.

[83] Xinghao Chen, Anhao Zhao, Heming Xia, Xuan Lu, Hanlin Wang, Yanjun Chen, Wei Zhang, Jian Wang, Wenjie Li, and Xiaoyu Shen. Reasoning Beyond Language: A Comprehensive Survey on Latent Chain-of-Thought Reasoning. arXiv preprint arXiv:2505.16782, 2025. URL https://arxiv.org/abs/2505.16782.

[84] Yanbei Chen, Anirudh Goyal, and Raghuraman Krishnamoorthi. Scaling Laws for Looped Mixture of Experts. arXiv preprint arXiv:2609.40316, 2026. URL https://arxiv.org/abs/2609.40316.

[85] Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, Vlad Mikulik, Samuel R. Bowman, Jan Leike, Jared Kaplan, and Ethan Perez. Reasoning Models Don't Always Say What They Think. arXiv preprint arXiv:2505.05410, 2025. URL https://arxiv.org/abs/2505.05410.

[86] Yang Chen, Canyu Shen, Xinzhe Rao, Yuanyi Yan, Yunlu Chen, Meng Tang, Teng Long, and Vincent Tao Hu. GramLoop: Training-Free Gram-Gated Replay for Robust Dense Prediction. arXiv preprint arXiv:2608.29113, 2026. URL https://arxiv.org/abs/2608.29113.

[87] Yao Chen, Yilong Chen, Yinqi Yang, Junyuan Shang, Zhenyu Zhang, Zefeng Zhang, Shuaiyi Nie, Shuohuan Wang, Yu Sun, Hua Wu, HaiFeng Wang, and Tingwen Liu. Sparse Growing Transformer: Training-Time Sparse Depth Allocation via Progressive Attention Looping. In *Annual Meeting of the Association for Computational Linguistics*, 2026. URL https://arxiv.org/abs/2603.23998.

[88] Yilong Chen, Junyuan Shang, Zhenyu Zhang, Yanxi Xie, Jiawei Sheng, Tingwen Liu, Shuohuan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Inner Thinking Transformer: Leveraging Dynamic Depth Scaling to Foster Adaptive Internal Thinking. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL https://arxiv.org/abs/2502.13842.

[89] Yilong Chen, Naibin Gu, Junyuan Shang, Zhenyu Zhang, Yuchen Feng, Jiawei Sheng, Tingwen Liu, Shuohuan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Mixture of Universal Experts: Scaling Virtual Width via Depth-Width Transformation. arXiv preprint arXiv:2603.04971, 2026. URL https://arxiv.org/abs/2603.04971.

[90] Yuhan Chen, Siyuan Zhang, Nan Wang, Feiyang Kang, and Ruoxi Jia. Hybrid Latent Attention for Looped Language Models. arXiv preprint arXiv:2610.07940, 2026. URL https://arxiv.org/abs/2610.07940.

[91] Yutong Chen, Shouqian Shi, Xinran Liu, Haochen Wang, Jiaying Wang, Tianxing Xu, Yuanxi Wang, and Zirui Ding. Penelope: Localized Latent Recurrence for Efficient Structured Reasoning. arXiv preprint arXiv:2607.25915, 2026. URL https://arxiv.org/abs/2607.25915.

[92] Zhaoliang Chen and Jie Fu. Latent Recurrent Thoughts: Recurrent Refinement of Proposed Latents for Reasoning with Frozen LLMs. arXiv preprint arXiv:2609.01117, 2026. URL https://arxiv.org/abs/2609.01117.

[93] Zixi Chen, Akshay Vegesna, Samip Dahal, and Andrew Gordon Wilson. How Model Growth, Recursion, and Boundary Operators Influence Scaling Exponents. arXiv preprint arXiv:2609.19107, 2026. URL https://arxiv.org/abs/2609.19107.

[94] Po-Han Chiang. Parallelized Hierarchical Connectome: A Spatiotemporal Recurrent Framework for Spiking State-Space Models. arXiv preprint arXiv:2604.01295, 2026. URL https://arxiv.org/abs/2604.01295.

[95] Khaled Chikh, Roberto Cavicchioli, and Alessandro Capotondi. ShareDAB-DETR: Memory-Efficient Object Detection through CentralBank Parameter Sharing. International Conference on Computer Vision Theory and Applications, 2026.

[96] Yong Xien Chng, Tianyi Chen, Wenwen Tong, Haiwen Diao, Zhongang Cai, Lei Yang, Ziwei Liu, Lewei Lu, Dahua Lin, and Gao Huang. Looped Diffusion Transformer. arXiv preprint arXiv:2609.40305, 2026. URL https://arxiv.org/abs/2609.40305.

[97] Hyunmin Cho, Jaejun Yoo, and Kyong Hwan Jin. Recurrent Sinusoidal INRs for Efficient High-Fidelity Representation. arXiv preprint arXiv:2607.21485, 2026. URL https://arxiv.org/abs/2607.21485.

[98] SangLyul Cho, Langqing Cui, Sehoon Kim, Dongsu Han, and Insu Han. LoopSpec: Pipelined Self-Speculative Decoding for Looped Transformers. arXiv preprint arXiv:2609.17184, 2026. URL https://arxiv.org/abs/2609.17184.

[99] Hyeong Kyu Choi, Chong Keun Paik, Hyun Woo Ko, Min-Chul Park, and Hyunwoo J. Kim. Recurrent DETR: Transformer-Based Object Detection for Crowded Scenes. IEEE Access, 2023.

[100] Jishnu Ray Chowdhury and Cornelia Caragea. Investigating Recurrent Transformers with Dynamic Halt. arXiv preprint arXiv:2402.00976, 2024. URL https://arxiv.org/abs/2402.00976.

[101] Jishnu Ray Chowdhury and Cornelia Caragea. On the Design Space Between Transformers and Recursive Neural Nets. arXiv preprint arXiv:2409.01531, 2024. URL https://arxiv.org/abs/2409.01531.

[102] Ho Lam Chung, Yiming Chen, Dau-Cheng Lyu, Hsiao-Tsung Hung, and Hung yi Lee. Listen, Think, Transcribe: Continuous Latent Test-Time Scaling for ASR. arXiv preprint arXiv:2607.05051, 2026. URL https://arxiv.org/abs/2607.05051.

[103] Samuel Cognolato and Alberto Testolin. Transformers discover an elementary calculation system exploiting local attention and grid-like problem representation. In *IEEE International Joint Conference on Neural Network*, 2022. URL https://arxiv.org/abs/2207.02536.

[104] Andrew Corbett, Archit Sood, Anna Tzatzopoulou, Sai-Aakash Ramesh, and Tim Dodwell. Boosting Inference with Guided Reasoning: Stochastic Exploration for Recursive Models. arXiv preprint arXiv:2605.25230, 2026. URL https://arxiv.org/abs/2605.25230.

[105] Aditya Cowsik, Tianyu He, and Andrey Gromov. Towards Distributed Neural Architectures. arXiv preprint arXiv:2506.22389, 2025. URL https://arxiv.org/abs/2506.22389.

[106] Róbert Csordás, Kazuki Irie, and Jürgen Schmidhuber. The Devil is in the Detail: Simple Tricks Improve Systematic Generalization of Transformers. In *Conference on Empirical Methods in Natural Language Processing*, 2021. URL https://arxiv.org/abs/2108.12284.

[107] Róbert Csordás, Kazuki Irie, and Jürgen Schmidhuber. The Neural Data Router: Adaptive Control Flow in Transformers Improves Systematic Generalization. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2110.07732.

[108] Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. MoEUT: Mixture-of-Experts Universal Transformers. In *Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.16039.

[109] Róbert Csordás, Christopher D. Manning, and Christopher Potts. Do Language Models Use Their Depth Efficiently? In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.13898.

[110] Garett Cunningham, Razvan C. Bunescu, and David Juedes. Towards Autoformalization of Mathematics and Code Correctness: Experiments with Elementary Proofs. *MATHNLP*, 2023. URL https://arxiv.org/abs/2301.02195.

[111] Dylan Cutler, Arun Kandoor, Nishanth Dikkala, Nikunj Saunshi, Xin Wang, and Rina Panigrahy. StagFormer: Time Staggering Transformer Decoding for RunningLayers In Parallel. arXiv preprint arXiv:2501.15665, 2025. URL https://arxiv.org/abs/2501.15665.

[112] Raj Dabre and Atsushi Fujita. Recurrent Stacking of Layers for Compact Neural Machine Translation Models. In *AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1807.05353.

[113] Raj Dabre and Atsushi Fujita. Recurrent Stacking of Layers in Neural Networks: An Application to Neural Machine Translation. arXiv preprint arXiv:2106.10002, 2021. URL https://arxiv.org/abs/2106.10002.

[114] Jiawen Dai and Yue Song. Winfree Oscillatory Neural Network. arXiv preprint arXiv:2605.20922, 2026. URL https://arxiv.org/abs/2605.20922.

[115] Long H Dang and David Rawlinson. HRM-Agent: Training a recurrent reasoning model in dynamic environments using reinforcement learning. arXiv preprint arXiv:2510.22832, 2025. URL https://arxiv.org/abs/2510.22832.

[116] Luke Darlow, Ciaran Regan, Sebastian Risi, Jeffrey Seely, and Llion Jones. Continuous Thought Machines. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.05522.

[117] Ha Van Dau, Thanh Tung Khuat, and Nguyen Thanh Dung. Beyond Depth Truncation: Controlled Evaluation of Depth Utilization in Recursive Language Models. arXiv preprint arXiv:2609.19934, 2026. URL https://arxiv.org/abs/2609.19934.

[118] Talgat Daulbaev, Ilya Glazkov, Maxim Rakhuba, and Ivan Oseledets. Reasoning on the Simplex: Geometric Fixed-Point Models. arXiv preprint arXiv:2609.33540, 2026. URL https://arxiv.org/abs/2609.33540.

[119] Artur Back de Luca and Kimon Fountoulakis. Simulation of Graph Algorithms with Looped Transformers. In *International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2402.01107.

[120] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1807.03819.

[121] Chunyuan Deng, Yizhe Zhang, Rui-Jie Zhu, Yuanyuan Xu, Jiarui Liu, T. S. Eugene Ng, and Hanjie Chen. LT2: Linear-Time Looped Transformers. arXiv preprint arXiv:2605.20670, 2026. URL https://arxiv.org/abs/2605.20670.

[122] Wenhao Deng, Junchen Fu, Hanwen Du, Alexandros Karatzoglou, Ioannis Arapakis, Hangjun Guo, Kaiwen Zheng, Yongxin Ni, and Joemon M. Jose. RecRec: Latent Interests Recursive Reasoning for Sequential Recommendation. In *Proceedings of the 20th ACM Conference on Recommender Systems*, 2026. URL https://arxiv.org/abs/2607.12945.

[123] Yunlong Deng, Boyang Sun, Yan Li, Lingjing Kong, Zeyu Tang, Kun Zhang, and Guangyi Chen. Selection, Reflection and Self-Refinement: Revisit Reasoning Tasks via a Causal Lens. arXiv preprint arXiv:2510.08222, 2025. URL https://arxiv.org/abs/2510.08222.

[124] Yuntian Deng, Yejin Choi, and Stuart Shieber. From Explicit CoT to Implicit CoT: Learning to Internalize CoT Step by Step. arXiv preprint arXiv:2405.14838, 2024. URL https://arxiv.org/abs/2405.14838.

[125] Joshua V. Dillon. Speed is Confidence. arXiv preprint arXiv:2601.19085, 2026. URL https://arxiv.org/abs/2601.19085.

[126] Danimir T. Doncevic, Alexander Mitsos, Yue Guo, Qianxiao Li, Felix Dietrich, Manuel Dahmen, and Ioannis G. Kevrekidis. A Recursively Recurrent Neural Network (R2N2) Architecture for Learning Iterative Algorithms. *SIAM Journal on Scientific Computing*, 2022. URL https://arxiv.org/abs/2211.12386.

[127] Jordan Dotzel, Yash Akhauri, Ahmed S. AbouElhamayed, Carly Jiang, Mohamed Abdelfattah, and Zhiru Zhang. Radial Networks: Dynamic Layer Routing for High-Performance Large Language Models. arXiv preprint arXiv:2404.04900, 2024. URL https://arxiv.org/abs/2404.04900.

[128] Mariia Drozdova, Aidan Sirbu, Pietro Miotti, Robert Obryk, Mayalen Etcheverry, Eyvind Niklasson, and Blake Richards. Diffusion as a Training Curriculum for Timestep-Free Iterative Reasoning. arXiv preprint arXiv:2609.01449, 2026. URL https://arxiv.org/abs/2609.01449.

[129] Hanwen Du, Yuxin Dong, and Xia Ning. Latent Thinking Optimization: Your Latent Reasoning Language Model Secretly Encodes Reward Signals in Its Latent Thoughts. arXiv preprint arXiv:2509.26314, 2025. URL https://arxiv.org/abs/2509.26314.

[130] Shivam Duggal, Phillip Isola, Antonio Torralba, and William T. Freeman. Adaptive Length Image Tokenization via Recurrent Allocation. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2411.02393.

[131] David Eigen, Jason Rolfe, Rob Fergus, and Yann LeCun. Understanding Deep Architectures using a Recursive Convolutional Network. In *International Conference on Learning Representations*, 2013. URL https://arxiv.org/abs/1312.1847.

[132] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2404.16710.

[133] Scott Emmons, Erik Jenner, David K. Elson, Rif A. Saurous, Senthooran Rajamanoharan, Heng Chen, Irhum Shafkat, and Rohin Shah. When Chain of Thought is Necessary, Language Models Struggle to Evade Monitors. arXiv preprint arXiv:2507.05246, 2025. URL https://arxiv.org/abs/2507.05246.

[134] Scott Emmons, Roland S. Zimmermann, David K. Elson, and Rohin Shah. A Pragmatic Way to Measure Chain-of-Thought Monitorability. arXiv preprint arXiv:2510.23966, 2025. URL https://arxiv.org/abs/2510.23966.

[135] Björn Engdahl, Adrian Kosowski, Jan Chorowski, Zuzanna Stamirowska, Przemysław Uznański, Junlin Jiang, Rohan Phadke, Remigiusz Kinas, and Richard Zhong. BDH-CQ: In-Context Learning with Recurrent Latent Reasoning. arXiv preprint arXiv:2608.09888, 2026. URL https://arxiv.org/abs/2608.09888.

[136] Joshua Engels, Callum McDougall, Bilal Chughtai, Janos Kramar, Senthooran Rajamanoharan, Cindy Wu, Arthur Conmy, Asic Q Chen, Jean Tarbouriech, Min Ma, Brendan O'Donoghue, João Gabriel Lopes de Oliveira, Rohin Shah, and Neel Nanda. How Transparent is DiffusionGemma? arXiv preprint arXiv:2606.20560, 2026. URL https://arxiv.org/abs/2606.20560.

[137] Vasiliy A. Es'kin and Mikhail E. Smorkalov. Dynamical Systems Theory Behind a Hierarchical Reasoning Model. arXiv preprint arXiv:2603.22871, 2026. URL https://arxiv.org/abs/2603.22871.

[138] Mehdi Esmaeilzadeh, Alexia Jolicoeur-Martineau, Chirag Vashist, and Ke Li. One Pass Is Not Enough: Recursive Latent Refinement for Generative Models. arXiv preprint arXiv:2605.15309, 2026. URL https://arxiv.org/abs/2605.15309.

[139] Bernardo Esteves, Miguel Vasco, and Francisco S. Melo. NeuralSolver: Learning Algorithms For Consistent and Efficient Extrapolation Across General Tasks. In *Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2402.15393.

[140] Mayalen Etcheverry, Pietro Miotti, Aidan Sirbu, Konstantin Schürholt, Mariia Drozdova, Arna Ghosh, Blaise Agüera y Arcas, James Manyika, Blake Richards, and Eyvind Niklasson. Reasoning with Neural Cellular Automata. arXiv preprint arXiv:2609.36126, 2026. URL https://arxiv.org/abs/2609.36126.

[141] Neo Eyal, Nachum Dershowitz, and Kfir Bar. Layer Duplication in LLMs. Conference on Empirical Methods in Natural Language Processing, 2025.

[142] Cristobal Eyzaguirre and Alvaro Soto. Differentiable Adaptive Computation Time for Visual Reasoning. In *Computer Vision and Pattern Recognition*, 2020. URL https://arxiv.org/abs/2004.12770.

[143] Cristóbal Eyzaguirre, Felipe del Río, Vladimir Araujo, and Álvaro Soto. DACT-BERT: Differentiable Adaptive Computation Time for an Efficient BERT Inference. *NLPPOWER*, 2021. URL https://arxiv.org/abs/2109.11745.

[144] Md Fahim, Amin Ahsan Ali, M. Ashraful Amin, and A K M Mahbubur Rahman. Contextual Bangla Neural Stemmer: Finding Contextualized Root Word Representations for Bangla Words. Workshop on Bangla Language Processing (BLP), 2023.

[145] Tingshuo Fan, Hongtao Mu, Tianyu Zhou, Hansen Liu, and Tao Ji. Looped GPT-BERT: Trading Parameters for Computation in Small Language Modeling. arXiv preprint arXiv:2609.09691, 2026. URL https://arxiv.org/abs/2609.09691.

[146] Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped Transformers for Length Generalization. arXiv preprint arXiv:2409.15647, 2024. URL https://arxiv.org/abs/2409.15647.

[147] Ying Fan, Anej Svete, and Kangwook Lee. Bridging the Gap Between Latent and Explicit Reasoning with Looped Transformers. arXiv preprint arXiv:2606.31779, 2026. URL https://arxiv.org/abs/2606.31779.

[148] Liancheng Fang, Zhuowei Li, Youngeun Kim, Tianchen Zhao, Rajat Koner, Jiaye Wu, Linghan Xu, Xuanbai Chen, Xiang Xu, Zheng Zhang, Jakub Zablocki, Nishant Sankaran, and

Yifan Xing. ALoDLM: Adaptively Looped Diffusion Language Models. arXiv preprint arXiv:2610.04198, 2026. URL https://arxiv.org/abs/2610.04198.

[149] Rui Fang, Hsi-Wen Chen, and Ming-Syan Chen. LoopQ: Quantization for Recursive Transformers. arXiv preprint arXiv:2605.16343, 2026. URL https://arxiv.org/abs/2605.16343.

[150] Yang Fang, Xiang Zhao, Yifan Chen, Weidong Xiao, and Maarten de Rijke. Pre-Trained Models for Heterogeneous Information Networks. arXiv preprint arXiv:2007.03184, 2020. URL https://arxiv.org/abs/2007.03184.

[151] Mónika Farsang, Ramin Hasani, Daniela Rus, and Radu Grosu. Reshape and Recur: Improving SSMs with Input Reshaping and Depth Recurrence. arXiv preprint arXiv:2605.16048, 2026. URL https://arxiv.org/abs/2605.16048.

[152] Jacob Fein-Ashley and Paria Rashidinejad. Solve the Loop: Attractor Models for Language and Reasoning. arXiv preprint arXiv:2605.12466, 2026. URL https://arxiv.org/abs/2605.12466.

[153] Kunyu Feng, Yuxiang Wang, Li Wang, Wan Lin, and Zhizheng Wu. CoReLoop: Parameter-Efficient Controlled Recurrent Refinement for Audio Deepfake Detection. arXiv preprint arXiv:2609.19818, 2026. URL https://arxiv.org/abs/2609.19818.

[154] Pedro Ferreira, Wilker Aziz, and Ivan Titov. Does Out-of-Sight Equal Out-of-Mind in CoT Monitorability? arXiv preprint arXiv:2608.04928, 2026. URL https://arxiv.org/abs/2608.04928.

[155] Lukas Finnveden, Alexa Pan, Alek Westover, Girish Gupta, et al. Latent reasoning architectures would likely undermine CoT, our strongest oversight tool. Redwood Research blog, September 2026. URL https://blog.redwoodresearch.org/p/latent-reasoning-architectures-would. Accessed 2026-10-05.

[156] Daniel Fojo, Víctor Campos, and Xavier Giro i Nieto. Comparing Fixed and Adaptive Computation Time for Recurrent Neural Networks. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1803.08165.

[157] Richard Freinschlag, Timo Bertram, Erich Kobler, Andreas Mayr, and Günter Klambauer. Symbol-Equivariant Recurrent Reasoning Models. arXiv preprint arXiv:2603.02193, 2026. URL https://arxiv.org/abs/2603.02193.

[158] Markus Frey, Behzad Shomali, Ali Hamza Bashir, David Berghaus, Joachim Koehler, and Mehdi Ali. Adaptive Loops and Memory in Transformers: Think Harder or Know More? arXiv preprint arXiv:2603.08391, 2026. URL https://arxiv.org/abs/2603.08391.

[159] Markus Frey, Behzad Shomali, Joachim Koehler, and Mehdi Ali. A Dual-Path Architecture for Scaling Compute and Capacity in LLMs. arXiv preprint arXiv:2605.30202, 2026. URL https://arxiv.org/abs/2605.30202.

[160] Hengyu Fu, Tianyu Guo, Zixuan Wang, Hanlin Zhu, Jason D. Lee, Jiantao Jiao, Stuart Russell, and Song Mei. DiscoLoop: Looping Discrete Embeddings and Continuous Hidden States for Multi-hop Reasoning. arXiv preprint arXiv:2607.00341, 2026. URL https://arxiv.org/abs/2607.00341.

[161] Rao Fu, Zixuan Yang, Jiankun Zhang, Jing Ma, Hechang Chen, Yu Li, and Yi Chang. Simply Stabilizing the Loop via Fully Looped Transformer. arXiv preprint arXiv:2605.18797, 2026. URL https://arxiv.org/abs/2605.18797.

[162] Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, and Yu Wang. Think-at-Hard: Dynamic Looped Transformers for Improved Reasoning. arXiv preprint arXiv:2511.08577, 2025. URL https://arxiv.org/abs/2511.08577.

[163] Samy Wu Fung, Howard Heaton, Qiuwei Li, Daniel McKenzie, Stanley Osher, and Wotao Yin. JFB: Jacobian-Free Backpropagation for Implicit Networks. In *AAAI Conference on Artificial Intelligence*, 2021. URL https://arxiv.org/abs/2103.12803.

[164] Shuyu Gan, Young-Jun Lee, and Dongyeop Kang. SanSi: A Looped Typed Decision Model for System 1.5 Thinking. arXiv preprint arXiv:2610.07730, 2026. URL https://arxiv.org/abs/2610.07730.

[165] Gege Gao, Bernhard Schölkopf, and Andreas Geiger. Slots, Transitions, Loops: Learning Composable World Models for ARC. arXiv preprint arXiv:2606.12316, 2026. URL https://arxiv.org/abs/2606.12316.

[166] Guangwei Gao, Zhengxue Wang, Juncheng Li, Wenjie Li, Yi Yu, and Tieyong Zeng. Lightweight Bimodal Network for Single-Image Super-Resolution via Symmetric CNN and Recursive Transformer. In *International Joint Conference on Artificial Intelligence*, 2022. URL https://arxiv.org/abs/2204.13286.

[167] Yihang Gao, Chuanyang Zheng, Enze Xie, Han Shi, Tianyang Hu, Yu Li, Michael K. Ng, Zhenguo Li, and Zhaoqiang Liu. AlgoFormer: An Efficient Transformer Framework with Algorithmic Structures. *Trans. Mach. Learn. Res.*, 2024. URL https://arxiv.org/abs/2402.13572.

[168] Zitian Gao, Lynx Chen, Yihao Xiao, He Xing, Ran Tao, Haoming Luo, Joey Zhou, and Bryan Dai. Universal Reasoning Model. arXiv preprint arXiv:2512.14693, 2025. URL https://arxiv.org/abs/2512.14693.

[169] Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, and Bryan Dai. Loop the Loopies! arXiv preprint arXiv:2607.16051, 2026. URL https://arxiv.org/abs/2607.16051.

[170] Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. Can Looped Transformers Learn to Implement Multi-step Gradient Descent for In-context Learning? In *International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2410.08292.

[171] Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. On the Role of Depth and Looping for In-Context Learning with Task Diversity. arXiv preprint arXiv:2410.21698, 2024. URL https://arxiv.org/abs/2410.21698.

[172] Daniil Gavrilov, Pavel Kalaidin, and Valentin Malykh. Self-Attentive Model for Headline Generation. In *European Conference on Information Retrieval*, 2019. URL https://arxiv.org/abs/1901.07786.

[173] Renee Ge, Qianli Liao, and Tomaso Poggio. Hierarchical Reasoning Models: Perspectives and Misconceptions. arXiv preprint arXiv:2510.00355, 2025. URL https://arxiv.org/abs/2510.00355.

[174] Tao Ge, Si-Qing Chen, and Furu Wei. EdgeFormer: A Parameter-Efficient Transformer for On-Device Seq2seq Generation. In *Conference on Empirical Methods in Natural Language Processing*, 2022. URL https://arxiv.org/abs/2202.07959.

[175] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.05171.

[176] Jonas Geiping, Xinyu Yang, and Guinan Su. Efficient Parallel Samplers for Recurrent-Depth Models and Their Connection to Diffusion Language Models. arXiv preprint arXiv:2510.14961, 2025. URL https://arxiv.org/abs/2510.14961.

[177] Zhengyang Geng, Ashwini Pokle, and J. Zico Kolter. One-Step Diffusion Distillation via Deep Equilibrium Models. In *Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2401.08639.

[178] Sia Gholami and Marwan Omar. Do Generative Large Language Models need billions of parameters? arXiv preprint arXiv:2309.06589, 2023. URL https://arxiv.org/abs/2309.06589.

[179] Raj Ghugare, Michał Bortkiewicz, Alicja Ziarko, and Benjamin Eysenbach. On the Role of Computation in Reinforcement Learning. arXiv preprint arXiv:2602.05999, 2026. URL https://arxiv.org/abs/2602.05999.

[180] Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped Transformers as Programmable Computers. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2301.13196.

[181] Guillaume Godin. SCORE: Replacing Layer Stacking with Contractive Recurrent Depth. arXiv preprint arXiv:2603.10544, 2026. URL https://arxiv.org/abs/2603.10544.

[182] Surbhi Goel, Sham Kakade, Adam Tauman Kalai, and Cyril Zhang. Recurrent Convolutional Neural Networks Learn Succinct Learning Algorithms. In *Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2209.00735.

[183] Lore Goetschalckx, Lakshmi Narasimhan Govindarajan, Alekh Karkada Ashok, Aarit Ahuja, David L. Sheinberg, and Thomas Serre. Computing a human-like reaction time metric from stable recurrent vision models. In *Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2306.11582.

[184] Melika Golestani, Seyedeh Zahra Razavi, Zeinab Borhanifard, Farnaz Tahmasebian, and Hesham Faili. Using BERT Encoding and Sentence-Level Language Model for Sentence Ordering. arXiv preprint arXiv:2108.10986, 2021. URL https://arxiv.org/abs/2108.10986.

[185] Zixuan Gong, Yong Liu, and Jiaye Teng. What Makes Looped Transformers Perform Better Than Non-Recursive Ones. arXiv preprint arXiv:2510.10089, 2025. URL https://arxiv.org/abs/2510.10089.

[186] Sahil Goyal, Swayam Agrawal, Gautham Govind Anil, Prateek Jain, Sujoy Paul, and Aditya Kusupati. ELT: Elastic Looped Transformers for Visual Generation. arXiv preprint arXiv:2604.09168, 2026. URL https://arxiv.org/abs/2604.09168.

[187] Alex Graves. Adaptive Computation Time for Recurrent Neural Networks. arXiv preprint arXiv:1603.08983, 2016. URL https://arxiv.org/abs/1603.08983.

[188] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The Unreasonable Ineffectiveness of the Deeper Layers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2403.17887.

[189] Florian Grötschla, Joël Mathys, and Roger Wattenhofer. Learning Graph Algorithms With Recurrent Graph Neural Networks. arXiv preprint arXiv:2212.04934, 2022. URL https://arxiv.org/abs/2212.04934.

[190] Grzegorz Gruszczynski, Pawel Olszowiec, Michal Byra, Grzegorz Stefanski, and Alberto Presta. Training Crossroads for Recurrent Vision Transformers: Recurrence, Neural ODEs, and Deep Supervision. arXiv preprint arXiv:2608.04879, 2026. URL https://arxiv.org/abs/2608.04879.

[191] Melody Y. Guan, Miles Wang, Micah Carroll, Zehao Dou, Annie Y. Wei, Marcus Williams, Benjamin Arnav, Joost Huizinga, Ian Kivlichan, Mia Glaese, Jakub Pachocki, and Bowen Baker. Monitoring Monitorability. arXiv preprint arXiv:2512.18311, 2025. URL https://arxiv.org/abs/2512.18311.

[192] Arthur Guez, Mehdi Mirza, Karol Gregor, Rishabh Kabra, Sébastien Racanière, Théophane Weber, David Raposo, Adam Santoro, Laurent Orseau, Tom Eccles, Greg Wayne, David Silver, and Timothy Lillicrap. An investigation of model-free planning. In *International Conference on Machine Learning*, 2019. URL https://arxiv.org/abs/1901.03559.

[193] Serdar Gülbahar, Lukas Edman, and Alexander Fraser. Squeezing More from Limited Data with Recursive Transformers. arXiv preprint arXiv:2608.26973, 2026. URL https://arxiv.org/abs/2608.26973.

[194] Qiushan Guo, Zhipeng Yu, Yichao Wu, Ding Liang, Haoyu Qin, and Junjie Yan. Dynamic Recursive Neural Network. IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2019.

[195] Zhihao Guo, Zonghan Wu, Haizhou Du, Huan Huo, Yilei Shao, Athanasios V. Vasilakos, and Qingsong Wen. Right Direction, Wrong Step: Geometric Analysis of Finite-Step Failure in Looped Transformers. arXiv preprint arXiv:2609.16665, 2026. URL https://arxiv.org/abs/2609.16665.

[196] Hyeongju Ha and Jae-Joon Kim. WaveFront Decoding: Parallelized Self-Speculative Decoding for Looped Language Models. arXiv preprint arXiv:2609.23033, 2026. URL https://arxiv.org/abs/2609.23033.

[197] Navid Hakimi. Form Follows Function: Recursive Stem Model. arXiv preprint arXiv:2603.15641, 2026. URL https://arxiv.org/abs/2603.15641.

[198] John T. Halloran. Nanbeige4.2-3B on Apple Silicon: Fixing Deployment Bugs and Decreasing Looped Transformer Memory Overhead. arXiv preprint arXiv:2608.13987, 2026. URL https://arxiv.org/abs/2608.13987.

[199] Sang-Il Han. Hierarchical vs. Flat Iteration in Shared-Weight Transformers. arXiv preprint arXiv:2604.14442, 2026. URL https://arxiv.org/abs/2604.14442.

[200] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. arXiv preprint arXiv:2412.06769, 2024. URL https://arxiv.org/abs/2412.06769.

[201] Shijie Hao and Weining Zhang. Iterate or Widen? When Test-Time Refinement Helps LiDAR Scene Completion: A Controlled Study of Evidence Geometry, Training Coverage, and Compute. arXiv preprint arXiv:2608.06014, 2026. URL https://arxiv.org/abs/2608.06014.

[202] Zhiwei Hao, Jianyuan Guo, Li Shen, Kai Han, Yehui Tang, Han Hu, and Yunhe Wang. ScaleNet: Scaling up Pretrained Neural Networks with Incremental Parameters. *IEEE Transactions on Image Processing*, 2025. URL https://arxiv.org/abs/2510.18431.

[203] Kareem Hassani, Chaymaa Abbas, Hadi Al Mubasher, and Mariette Awad. RRFC: Recursive Refinement via Feedback Conditioning for Iterative Image-to-Image Generation. arXiv preprint arXiv:2608.15694, 2026. URL https://arxiv.org/abs/2608.15694.

[204] Tamir David Hay and Lior Wolf. Dynamic Layer Tying for Parameter-Efficient Transformers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2401.12819.

[205] Di He, Pengxiang Li, Da Chang, Qingyan Meng, Lu Yin, and Shiwei Liu. Looping Beyond Twice: A Scalable Recipe for Looped Mixture-of-Experts. arXiv preprint arXiv:2610.01153, 2026. URL https://arxiv.org/abs/2610.01153.

[206] Juncai He, Xinliang Liu, and Jinchao Xu. Self-composing neural operators for high-frequency and multiscale PDE surrogates. *Journal of Computational Physics*, 2025. URL https://arxiv.org/abs/2508.20650.

[207] Renjie He. RD-ViT: Recurrent-Depth Vision Transformer for Semantic Segmentation with Reduced Data Dependence Extending the Recurrent-Depth Transformer Architecture to Dense Prediction. arXiv preprint arXiv:2605.03999, 2026. URL https://arxiv.org/abs/2605.03999.

[208] Ahmed Heakl, Martin Gubri, Salman Khan, Sangdoo Yun, and Seong Joon Oh. Dr.LLM: Dynamic Layer Routing in LLMs. arXiv preprint arXiv:2510.12773, 2025. URL https://arxiv.org/abs/2510.12773.

[209] Amr Hegazy, Amr Alanwar, and Mostafa Elhoushi. Gated Recurrent Transformers: Expressive Depth through Recurrent Modulation. arXiv preprint arXiv:2608.15062, 2026. URL https://arxiv.org/abs/2608.15062.

[210] Alec Helbling, Andrey Bryutkin, Mauro Martino, Duen Horng Chau, Nima Dehmamy, and Hendrik Strobelt. Flow Reasoning Models: Turning Flows Into Efficient Recurrent Reasoners. arXiv preprint arXiv:2606.29150, 2026. URL https://arxiv.org/abs/2606.29150.

[211] Jaemu Heo, Eldor Fozilov, Hyunmin Song, and Taehwan Kim. RingFormer: Rethinking Recurrent Transformer with Adaptive Level Signals. In *Conference on Empirical Methods in Natural Language Processing*, 2025. URL https://arxiv.org/abs/2502.13181.

[212] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Sophia Shao. SPEED: Speculative Pipelined Execution for Efficient Decoding. arXiv preprint arXiv:2310.12072, 2023. URL https://arxiv.org/abs/2310.12072.

[213] Koushik Howlader, Tirtho Roy, Md Tauhidul Islam, and Wei Le. bioMoR: Biology-Guided Mixture-of-Recursions for Effective Genomic Learning. arXiv preprint arXiv:2608.06727, 2026. URL https://arxiv.org/abs/2608.06727.

[214] Chia-Hsuan Hsu and Jui-Ming Yao. Learning to Refine Hidden States for Reliable LLM Reasoning. arXiv preprint arXiv:2606.17524, 2026. URL https://arxiv.org/abs/2606.17524.

[215] Keya Hu, Ali Cy, Linlu Qiu, Xiaoman Delores Ding, Runqian Wang, Yeyin Eva Zhu, Jacob Andreas, and Kaiming He. ARC Is a Vision Problem! arXiv preprint arXiv:2511.14761, 2025. URL https://arxiv.org/abs/2511.14761.

[216] Yi Hu, Cai Zhou, and Muhan Zhang. What Affects the Effective Depth of Large Language Models? arXiv preprint arXiv:2512.14064, 2025. URL https://arxiv.org/abs/2512.14064.

[217] Yunzhe Hu, Difan Zou, and Dong Xu. Hyper-SET: Designing Transformers via Hyperspherical Energy Minimization. arXiv preprint arXiv:2502.11646, 2025. URL https://arxiv.org/abs/2502.11646.

[218] Benhao Huang, Zhengyang Geng, and Zico Kolter. Equilibrium Reasoners: Learning Attractors Enables Scalable Reasoning. arXiv preprint arXiv:2605.21488, 2026. URL https://arxiv.org/abs/2605.21488.

[219] Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. arXiv preprint arXiv:2610.06833, 2026. URL https://arxiv.org/abs/2610.06833.

[220] Minbin Huang, Han Shi, Chuanyang Zheng, Yimeng Wu, Guoxuan Chen, Xintong Yu, Yichun Yin, and Hong Cheng. UniPool: Learning Expert-to-Layer Ownership from Brief Global Access. arXiv preprint arXiv:2605.06665, 2026. URL https://arxiv.org/abs/2605.06665.

[221] Seth H. Huang, Xu Lingjie, and Jiang Congwei. Residual Attention Net for Superior Cross-Domain Time Sequence Modeling. arXiv preprint arXiv:2001.04077, 2020. URL https://arxiv.org/abs/2001.04077.

[222] Xinting Huang, Andy Yang, Satwik Bhattamishra, Yash Sarrof, Andreas Krebs, Hattie Zhou, Preetum Nakkiran, and Michael Hahn. A Formal Framework for Understanding Length Generalization in Transformers. arXiv preprint arXiv:2410.02140, 2024. URL https://arxiv.org/abs/2410.02140.

[223] Yulong Huang, Chen Jiang, Zhanpeng Zhou, Hongtao Zhang, Tianyu Li, Tianyu He, Xiangyu Zhang, and Bojun Cheng. Looped Transformers as Optimizers. arXiv preprint arXiv:2609.37379, 2026. URL https://arxiv.org/abs/2609.37379.

[224] Zekai Huang, Yingyu Liang, Zhenmei Shi, Zhao Song, and Zhen Zhuang. Neural Algorithmic Reasoning for Hypergraphs with Looped Transformers. arXiv preprint arXiv:2501.10688, 2025. URL https://arxiv.org/abs/2501.10688.

[225] Zeyi Huang, Xuehai He, Yong Jae Lee, and Yelong Shen. Trading Depth for Time in Recurrent Transformers. arXiv preprint arXiv:2609.21605, 2026. URL https://arxiv.org/abs/2609.21605.

[226] Zihao Huang, Jundong Zhou, Xingwei Qu, Qiyang Min, and Ge Zhang. ConceptMoE: Adaptive Token-to-Concept Compression for Implicit Compute Allocation. arXiv preprint arXiv:2601.21420, 2026. URL https://arxiv.org/abs/2601.21420.

[227] Thorir Mar Ingolfsson, Wajeeha Tahir, Anna Tegon, Lionnus Kesting, Gamze İslamoğlu, and Luca Benini. Quantizing Recursive Reasoning Models. arXiv preprint arXiv:2607.16237, 2026. URL https://arxiv.org/abs/2607.16237.

[228] Takuya Ito, Ruchir Puri, Murray Campbell, and Parikshit Ram. Universal Transformers for Circuit Computations: Perfect Length Generalization in Tiny Transformers. arXiv preprint arXiv:2608.31067, 2026. URL https://arxiv.org/abs/2608.31067.

[229] Jaber Jaber and Osama Jaber. Ouroboros: Dynamic Weight Generation for Recursive Transformers via Input-Conditioned LoRA Modulation. arXiv preprint arXiv:2604.02051, 2026. URL https://arxiv.org/abs/2604.02051.

[230] Jaber Jaber and Osama Jaber. Memoir: Should a Model Write to Its Memory While It Thinks? arXiv preprint arXiv:2607.20792, 2026. URL https://arxiv.org/abs/2607.20792.

[231] Mozes Jacobs, Thomas Fel, Richard Hakim, Alessandra Brondetta, Demba Ba, and T. Andy Keller. Block-Recurrent Dynamics in Vision Transformers. arXiv preprint arXiv:2512.19941, 2025. URL https://arxiv.org/abs/2512.19941.

[232] Akbar Anbar Jafari and Gholamreza Anbarjafari. Closed-Loop Transformers: Autoregressive Modeling as Iterative Latent Equilibrium. arXiv preprint arXiv:2511.21882, 2025. URL https://arxiv.org/abs/2511.21882.

[233] Martin Jaggi. Tying the Loop – Tied Expert Layers in Mixture-of-Experts Language Models. arXiv preprint arXiv:2606.16825, 2026. URL https://arxiv.org/abs/2606.16825.

[234] Amit Jain and Richard Linares. Tiny Recursive Control: Iterative Reasoning for Efficient Optimal Control. arXiv preprint arXiv:2512.16824, 2025. URL https://arxiv.org/abs/2512.16824.

[235] Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. LoopFormer: Elastic-Depth Looped Transformers for Latent Reasoning via Shortcut Modulation. arXiv preprint arXiv:2602.11451, 2026. URL https://arxiv.org/abs/2602.11451.

[236] Hyojin Jeon, Seungcheol Park, Jin-Gee Kim, and U Kang. PET: Parameter-efficient Knowledge Distillation on Transformer. PLoS ONE, 2023.

[237] Selim Jerad, Anej Svete, Sophie Hao, Ryan Cotterell, and William Merrill. Context-Free Recognition with Transformers. arXiv preprint arXiv:2601.01754, 2026. URL https://arxiv.org/abs/2601.01754.

[238] Zeyu Jia. Port-Hamiltonian Latent Deliberation: Mitigating the Deliberation Drift Cliff in Test-Time Compute Scaling. arXiv preprint arXiv:2609.37351, 2026. URL https://arxiv.org/abs/2609.37351.

[239] Pengfei Jiang, Jixiang Luo, Luxi Lin, Zhaohong Huang, and Xuelong Li. Visual Implicit Autoregressive Modeling. arXiv preprint arXiv:2605.01220, 2026. URL https://arxiv.org/abs/2605.01220.

[240] Pearse Jim, Steven Kolawole, Opegbemi Matthias Busoye, Glory Bagai, and Virginia Smith. What Survives When You Compress a Recursive Reasoner for the Edge? arXiv preprint arXiv:2606.26488, 2026. URL https://arxiv.org/abs/2606.26488.

[241] Hengyu Jin, Shu Yang, and Di Wang. Final Checkpoints Are Not Enough: Analyzing Latent Reasoning Faithfulness Along Training Trajectories. arXiv preprint arXiv:2607.06648, 2026. URL https://arxiv.org/abs/2607.06648.

[242] Zehao Jin, Ruixuan Deng, and Junran Wang. Transformers Stop Thinking Too Early, and a Tiny LoRA Fixes It. arXiv preprint arXiv:2609.36585, 2026. URL https://arxiv.org/abs/2609.36585.

[243] Zijian Jin, Yunbei Zhang, Yuanzhe Liu, Ming Liu, Baian Chen, Weirui Ye, Shilong Liu, and Marco Pavone. DeepJEPA: Scaling World Models from Within. arXiv preprint arXiv:2610.00368, 2026. URL https://arxiv.org/abs/2610.00368.

[244] Akhil Jindal and Harang Ju. SMolLM: Small Language Models Learn Small Molecular Grammar. arXiv preprint arXiv:2605.06322, 2026. URL https://arxiv.org/abs/2605.06322.

[245] Alexia Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. arXiv preprint arXiv:2510.04871, 2025. URL https://arxiv.org/abs/2510.04871.

[246] Caleb Jore and Jialin Liu. Bifurcation Models: Learning Set-Valued Solution Maps with Weight-Tied Dynamics. arXiv preprint arXiv:2605.07277, 2026. URL https://arxiv.org/abs/2605.07277.

[247] Da Ju, Stephen Roller, Sainbayar Sukhbaatar, and Jason Weston. Staircase Attention for Recurrent Processing of Sequences. In *Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2106.04279.

[248] Georgios Kaissis, David Mildenberger, Juan Felipe Gomez, Martin J. Menten, and Eleni Triantafillou. Step-resolved data attribution for looped transformers. arXiv preprint arXiv:2602.10097, 2026. URL https://arxiv.org/abs/2602.10097.

[249] Yacouba Kaloga, Shashi Kumar, Shakeel A. Sheikh, Driss Khalil, Petr Motlicek, and Ina Kodrasi. Test-Time Compute Scaling for ASR with Depth-Conditioned Looped Transformers. arXiv preprint arXiv:2606.04678, 2026. URL https://arxiv.org/abs/2606.04678.

[250] Shunsuke Kamiya, Masanori Koyama, Seongcheol Jeong, Fumiya Uchiyama, Kenji Kubo, Kohei Hayashi, Masahiro Suzuki, and Yutaka Matsuo. Steering Recurrent Reasoners at Inference Time with Readout Feedback. arXiv preprint arXiv:2608.24136, 2026. URL https://arxiv.org/abs/2608.24136.

[251] Ferdinand Kapl, Emmanouil Angelis, Kaitlin Maile, Johannes von Oswald, and Stefan Bauer. From Growing to Looping: A Unified View of Iterative Computation in LLMs. arXiv preprint arXiv:2602.16490, 2026. URL https://arxiv.org/abs/2602.16490.

[252] Aras Kavuncu. LSTM-UT and Recurrent-Depth Transformers on Cellular Automata. arXiv preprint arXiv:2609.19521, 2026. URL https://arxiv.org/abs/2609.19521.

[253] Aras Kavuncu, Bryan Vullo, and Alberto Berni. Reproducing Recurrent Transformers: The CoTFormer. arXiv preprint arXiv:2607.19405, 2026. URL https://arxiv.org/abs/2607.19405.

[254] Yekun Ke, Xiaoyu Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. Advancing the Understanding of Fixed Point Iterations in Deep Neural Networks: A Detailed Analytical Study. arXiv preprint arXiv:2410.11279, 2024. URL https://arxiv.org/abs/2410.11279.

[255] Iurii Kemaev, Daniil Polykovskiy, and Dmitry Vetrov. ReSet: Learning Recurrent Dynamic Routing in ResNet-like Neural Networks. In *Asian Conference on Machine Learning*, 2018. URL https://arxiv.org/abs/1811.04380.

[256] Bum Jun Kim, Kohei Hayashi, Shunsuke Kamiya, Masanori Koyama, Yusuke Iwasawa, and Yutaka Matsuo. Looped Transformers with Source-Centered State Evolution. arXiv preprint arXiv:2607.27656, 2026. URL https://arxiv.org/abs/2607.27656.

[257] Gunn Kim. Dynamical phase selection controls compute scaling in looped transformers. arXiv preprint arXiv:2608.26556, 2026. URL https://arxiv.org/abs/2608.26556.

[258] Jiwon Kim, Jung Kwon Lee, and Kyoung Mu Lee. Deeply-Recursive Convolutional Network for Image Super-Resolution. In *Computer Vision and Pattern Recognition*, 2015. URL https://arxiv.org/abs/1511.04491.

[259] Junseo Kim, Uraz Odyurt, and Amirreza Yousefzadeh. MergeOver: Post-Training Token Merging for Recursive Vision Transformers. arXiv preprint arXiv:2608.13141, 2026. URL https://arxiv.org/abs/2608.13141.

[260] Jan Kirin. Relational Preference Encoding in Looped Transformer Internal States. arXiv preprint arXiv:2604.09870, 2026. URL https://arxiv.org/abs/2604.09870.

[261] Jan Kirin. Operational Proto-Introspection in Looped Language Models: Process-Quality Taps, Executable Branching, and the Readout-Control Boundary. arXiv preprint arXiv:2607.18553, 2026. URL https://arxiv.org/abs/2607.18553.

[262] Tuğçe Kızıltepe and Hacer Yalim Keles. ODE-Based Transformer Decoders for Iterative Sign Language Translation. arXiv preprint arXiv:2608.11352, 2026. URL https://arxiv.org/abs/2608.11352.

[263] Jonas Knupp, Jan Hendrik Metzen, Jeremias Bohn, Georg Groh, and Kristian Kersting. Depth-Recurrent Attention Mixtures: Giving Latent Reasoning the Attention it Deserves. arXiv preprint arXiv:2601.21582, 2026. URL https://arxiv.org/abs/2601.21582.

[264] Brandon Knutson, Amandin Chyba Rabeendran, Michael Ivanitskiy, Jordan Pettyjohn, Cecilia Diniz-Behn, Samy Wu Fung, and Daniel McKenzie. On Logical Extrapolation for Mazes with Recurrent and Implicit Networks. In *AAAI Conference on Artificial Intelligence*, 2024. URL https://arxiv.org/abs/2410.03020.

[265] Gregor Koehler, Tassilo Wald, Constantin Ulrich, David Zimmerer, Paul F. Jaeger, Jörg K. H. Franke, Simon Kohl, Fabian Isensee, and Klaus H. Maier-Hein. RecycleNet: Latent Feature

Recycling Leads to Iterative Decision Refinement. arXiv preprint arXiv:2309.07513, 2023. URL https://arxiv.org/abs/2309.07513.

[266] Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers. arXiv preprint arXiv:2604.07822, 2026. URL https://arxiv.org/abs/2604.07822.

[267] Yeskendir Koishekenov, Aldo Lipani, and Nicola Cancedda. Encode, Think, Decode: Scaling test-time reasoning with recursive latent thoughts. arXiv preprint arXiv:2510.07358, 2025. URL https://arxiv.org/abs/2510.07358.

[268] Steven Kolawole, Pearse Jim, Opegbemi M. Busoye, Glory Bagai, and Virginia Smith. A Tilted Bowl Is Not a Slippery Slope: Compressing Looped Models. arXiv preprint arXiv:2609.39277, 2026. URL https://arxiv.org/abs/2609.39277.

[269] Mieszko Komisarczyk, Saurabh Mathur, Maurice Kraus, Sriraam Natarajan, and Kristian Kersting. Recursive Inference Machines for Neural Reasoning. arXiv preprint arXiv:2603.05234, 2026. URL https://arxiv.org/abs/2603.05234.

[270] Okan Köpüklü, Maryam Babaee, Stefan Hörmann, and Gerhard Rigoll. Convolutional Neural Networks with Layer Reuse. In *International Conference on Information Photonics*, 2019. URL https://arxiv.org/abs/1901.09615.

[271] Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, Mark Chen, Alan Cooney, Allan Dafoe, Anca Dragan, Scott Emmons, Owain Evans, David Farhi, Ryan Greenblatt, Dan Hendrycks, Marius Hobbhahn, Evan Hubinger, Geoffrey Irving, Erik Jenner, Daniel Kokotajlo, Victoria Krakovna, Shane Legg, David Lindner, David Luan, Aleksander Mądry, Julian Michael, Neel Nanda, Dave Orr, Jakub Pachocki, Ethan Perez, Mary Phuong, Fabien Roger, Joshua Saxe, Buck Shlegeris, Martín Soto, Eric Steinberger, Jasmine Wang, Wojciech Zaremba, Bowen Baker, Rohin Shah, and Vlad Mikulik. Chain of Thought Monitorability: A New and Fragile Opportunity for AI Safety. arXiv preprint arXiv:2507.11473, 2025. URL https://arxiv.org/abs/2507.11473.

[272] Hare Krishna, Shubham Singh, Stephen Ebert, and Hao-Yu Sun. When Recursive Models Finish Computing. arXiv preprint arXiv:2609.26487, 2026. URL https://arxiv.org/abs/2609.26487.

[273] Nicholas Kuang, Vanessa Scalon, and Ji Yu. UCell: rethinking generalizability and scaling of bio-medical vision models. arXiv preprint arXiv:2604.00243, 2026. URL https://arxiv.org/abs/2604.00243.

[274] Kenji Kubo, Shunsuke Kamiya, Masanori Koyama, Kohei Hayashi, Yusuke Iwasawa, and Yutaka Matsuo. C-voting: Confidence-Based Test-Time Voting without Explicit Energy Functions. arXiv preprint arXiv:2604.13521, 2026. URL https://arxiv.org/abs/2604.13521.

[275] Hsun-Yu Kuo, El Mahdi Chayti, Patrik Reizinger, Wieland Brendel, and Martin Jaggi. Stabilizing Extrapolation in Looped Transformers via Learned Stochastic Stopping. arXiv preprint arXiv:2606.29983, 2026. URL https://arxiv.org/abs/2606.29983.

[276] Nahomi Kusunoki, Yosuke Higuchi, Tetsuji Ogawa, and Tetsunori Kobayashi. Hierarchical Multi-Task Learning with CTC and Recursive Operation. Interspeech (USB), 2024.

[277] Dohyeon Kwon and Youngjin Park. CHERRY: Compressed Hierarchical Experts with Recurrent Representational Yield. arXiv preprint arXiv:2606.31796, 2026. URL https://arxiv.org/abs/2606.31796.

[278] Alexander Kyuroson, Denis Kleyko, and Marcus Liwicki. TIDE: Asymmetric Neural Circuits for Stabilized Temporal Inhibitory-Excitatory Dynamics. arXiv preprint arXiv:2605.19403, 2026. URL https://arxiv.org/abs/2605.19403.

[279] Asher Labovich. Stability and Generalization in Looped Transformers. arXiv preprint arXiv:2604.15259, 2026. URL https://arxiv.org/abs/2604.15259.

[280] Jeffrey Lai, Anthony Bao, John Quinn, and William Gilpin. Fractal basins trap latent reasoning. arXiv preprint arXiv:2609.04963, 2026. URL https://arxiv.org/abs/2609.04963.

[281] Venkat Akhil Lakkapragada. CosmicFish-HRM: Adaptive Reasoning via Hierarchical Recurrent Mechanisms in Compact Language Models. arXiv preprint arXiv:2605.28919, 2026. URL https://arxiv.org/abs/2605.28919.

[282] Wai-Lun Lam. Energy-Entropy Regularization: The True Power of Minimal Looped Transformers. arXiv preprint arXiv:2601.09588, 2026. URL https://arxiv.org/abs/2601.09588.

[283] Simon Lam-Muir. The Ignition Is Real, and It Lives at the Readout: Latent composition, difficulty-clocked ignition, and the interface-constituted commit in a recurrent-depth reasoner. arXiv preprint arXiv:2608.03263, 2026. URL https://arxiv.org/abs/2608.03263.

[284] Simon Lam-Muir. Behaviour Is an Incomplete Measure of Reasoning Development: Cross-surface pre-arrival accessibility and the limits of developmental inference in a recurrent-depth reasoner. arXiv preprint arXiv:2608.16085, 2026. URL https://arxiv.org/abs/2608.16085.

[285] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1909.11942.

[286] Clément Laroche and Riccardo Miccini. Does per-frame early exit pay? A compute-matched study of dynamic depth for on-device speech enhancement. arXiv preprint arXiv:2609.29867, 2026. URL https://arxiv.org/abs/2609.29867.

[287] Byung-Kwan Lee, Sangyun Chung, Chae Won Kim, Beomchan Park, and Yong Man Ro. TroL: Traversal of Layers for Large Language and Vision Models. In *Conference on Empirical Methods in Natural Language Processing*, 2024. URL https://arxiv.org/abs/2406.12246.

[288] Ryan Lee, Jacob Biloki, Edward J. Hu, and Jonathan May. Sparse Layers are Critical to Scaling Looped Language Models. arXiv preprint arXiv:2605.09165, 2026. URL https://arxiv.org/abs/2605.09165.

[289] Sang In Lee and Jihun Park. Soft Mixture-of-Recursions: Going Deeper with Recursive Vision Transformers. arXiv preprint arXiv:2607.00774, 2026. URL https://arxiv.org/abs/2607.00774.

[290] Sangho Lee, Youngjae Yu, Gunhee Kim, Thomas Breuel, Jan Kautz, and Yale Song. Parameter Efficient Multimodal Transformers for Video Representation Learning. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/2012.04124.

[291] Sanghyun Lee, Chunsan Hong, Seungryong Kim, Jonghyun Lee, Jongho Park, and Dongmin Park. Looped Diffusion Language Models. arXiv preprint arXiv:2605.26106, 2026. URL https://arxiv.org/abs/2605.26106.

[292] Wen-Ze Lee, Darren Tan, Ian McLoughlin, Aik Beng Ng, Zheng-Kui Wang, et al. SkipFold: Test-time dynamic depth for scalable biomolecular inference. In *2026 6th International Conference on Advanced Algorithms and Neural Networks (AANN)*, 2026. doi: 10.1109/AANN71145.2026.11709365.

[293] Kart leong Lim. Recursive transformers for semiconductor thermo-mechanical reliability. arXiv preprint arXiv:2607.27251, 2026. URL https://arxiv.org/abs/2607.27251.

[294] Sam Leroux, Pavlo Molchanov, Pieter Simoens, Bart Dhoedt, Thomas Breuel, and Jan Kautz. IamNN: Iterative and Adaptive Mobile Neural Network for Efficient Image Classification. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1804.10123.

[295] Mosh Levy, Zohar Elyoseph, Shauli Ravfogel, and Yoav Goldberg. State over Tokens: Characterizing the Role of Reasoning Tokens. arXiv preprint arXiv:2512.12777, 2025. URL https://arxiv.org/abs/2512.12777.

[296] Bingcong Li, Xin Tang, Xianbiao Qi, Yihao Chen, and Rong Xiao. Hamming OCR: A Locality Sensitive Hashing Neural Network for Scene Text Recognition. arXiv preprint arXiv:2009.10874, 2020. URL https://arxiv.org/abs/2009.10874.

[297] Chloe Li, Mary Phuong, and Noah Y. Siegel. LLMs Can Covertly Sandbag on Capability Evaluations Against Chain-of-Thought Monitoring. In *IJCNLP-AACL*, 2025. URL https://arxiv.org/abs/2508.00943.

[298] Guanghao Li, Wenhao Jiang, Mingfeng Chen, Yan Li, Hao Yu, Shuting Dong, Tao Ren, Ming Tang, and Chun Yuan. SCOUT: Teaching Pre-trained Language Models to Enhance Reasoning via Flow Chain-of-Thought. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.24181.

[299] Guanghao Li, Wenhao Jiang, Li Shen, Ming Tang, and Chun Yuan. Zero Token-Driven Deep Thinking in LLMs: Unlocking the Full Potential of Existing Parameters via Cyclic Refinement. arXiv preprint arXiv:2502.12214, 2025. URL https://arxiv.org/abs/2502.12214.

[300] Guanghao Li, Wenhao Jiang, Li Shen, Ming Tang, and Chun Yuan. Efficient Transformer Parameter Reuse via Zero-Token Mechanism. Annual Meeting of the Association for Computational Linguistics, 2026.

[301] Guanghao Li, Zihan Su, Hao Yu, Jinyang Jiang, Tao Ren, Zehao Li, Feng Lu, Ming Tang, and Chun Yuan. Shallow Queries, Mature Values: Depth-Asynchronous Self-Speculation for Looped Transformers. arXiv preprint arXiv:2609.34538, 2026. URL https://arxiv.org/abs/2609.34538.

[302] GuoLiang Li and Yiyang Li. Recurrent multiple shared layers in Depth for Neural Machine Translation. arXiv preprint arXiv:2108.10417, 2021. URL https://arxiv.org/abs/2108.10417.

[303] James Li, Philip H. W. Leong, and Thomas Chaffey. Quantization Robustness of Monotone Operator Equilibrium Networks. *IEEE Control Systems Letters*, 2026. URL https://arxiv.org/abs/2603.10562.

[304] Jin Li. Universal Transforming Geometric Network. arXiv preprint arXiv:1908.00723, 2019. URL https://arxiv.org/abs/1908.00723.

[305] Jindong Li, Yali Fu, Li Fan, Jiahong Liu, Yao Shu, Chengwei Qin, Menglin Yang, Irwin King, and Rex Ying. Implicit Reasoning in Large Language Models: A Comprehensive Survey. arXiv preprint arXiv:2509.02350, 2025. URL https://arxiv.org/abs/2509.02350.

[306] Nizhang Li and Ian G. Harris. SafeDepth: Safety-Aware Token-Level Adaptive Computation. arXiv preprint arXiv:2610.00815, 2026. URL https://arxiv.org/abs/2610.00815.

[307] Nux Li. Quantizing Looped Transformers: Feedback Exposure and Calibration Blindness. arXiv preprint arXiv:2609.30820, 2026. URL https://arxiv.org/abs/2609.30820.

[308] Shuzhen Li, Yifan Zhang, Jiacheng Guo, Quanquan Gu, and Mengdi Wang. DeepLoop: Depth Scaling for Looped Transformers. arXiv preprint arXiv:2607.13491, 2026. URL https://arxiv.org/abs/2607.13491.

[309] Yingcong Li, Xiangyu Chang, Muti Kara, Xiaofeng Liu, Amit Roy-Chowdhury, and Samet Oymak. When and How Unlabeled Data Provably Improve In-Context Learning. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2506.15329.

[310] Zeyu LI, Yipu ZHANG, Jintao Chen, Xin LI, and Wei ZHANG. Loopy: Low-Bit Quantization Framework for Looped Language Models. arXiv preprint arXiv:2610.05265, 2026. URL https://arxiv.org/abs/2610.05265.

[311] Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of Thought Empowers Transformers to Solve Inherently Serial Problems. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2402.12875.

[312] Zijian Li, Xiangchen Song, Gongxu Luo, Jie Qiao, Ruichu Cai, Zhenhao Chen, Xinshuai Dong, Fan Feng, Guangyi Chen, and Kun Zhang. TAFFY: A Task-Adaptive Tabular Foundation Model with In-Context Diversity. arXiv preprint arXiv:2610.07559, 2026. URL https://arxiv.org/abs/2610.07559.

[313] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a Layer or Loop it? Test-Time Depth Adaptation of Pretrained LLMs. arXiv preprint arXiv:2507.07996, 2025. URL https://arxiv.org/abs/2507.07996.

[314] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a Layer or Loop It? Learning Program-of-Layers in LLMs. arXiv preprint arXiv:2606.06574, 2026. URL https://arxiv.org/abs/2606.06574.

[315] Jia Liang, Xi Jin, and Liangming Pan. Beyond the Training Horizon: Mechanisms and Limits of Length Generalization in Looped Transformers. arXiv preprint arXiv:2609.33144, 2026. URL https://arxiv.org/abs/2609.33144.

[316] Yingyu Liang, Zhizhou Sha, Zhenmei Shi, Zhao Song, and Yufa Zhou. Looped ReLU MLPs May Be All You Need as Practical Programmable Computers. In *International Conference on Artificial Intelligence and Statistics*, 2024. URL https://arxiv.org/abs/2410.09375.

[317] Yuanchu Liang, Saeed Anwar, and Yang Liu. DRT: A Lightweight Single Image Deraining Recursive Transformer. arXiv preprint arXiv:2204.11385, 2022. URL https://arxiv.org/abs/2204.11385.

[318] Qianli Liao and Tomaso Poggio. Bridging the Gaps Between Residual Learning, Recurrent Neural Networks and Visual Cortex. arXiv preprint arXiv:1604.03640, 2016. URL https://arxiv.org/abs/1604.03640.

[319] Chien-Yu Lin, Anish Prabhu, Thomas Merth, Sachin Mehta, Anurag Ranjan, Maxwell Horton, and Mohammad Rastegari. SPIN: An Empirical Evaluation on Sharing Parameters of Isotropic Networks. In *European Conference on Computer Vision*, 2022. URL https://arxiv.org/abs/2207.10237.

[320] Ruhai Lin, Yiyang Guo, Rui-Jie Zhu, Hao Ye, and Jason K. Eshraghian. Allocating Recurrent Compute in Looped Language Models. arXiv preprint arXiv:2608.18230, 2026. URL https://arxiv.org/abs/2608.18230.

[321] Ye Lin, Mingxuan Wang, Zhexi Zhang, Xiaohui Wang, Tong Xiao, and Jingbo Zhu. Understanding Parameter Sharing in Transformers. arXiv preprint arXiv:2306.09380, 2023. URL https://arxiv.org/abs/2306.09380.

[322] Yeh-Sheng Lin and Shu-Chuan Tseng. Classifying Speech Intelligibility Levels of Children in Two Continuous Speech Styles. IEEE International Conference on Acoustics Speech and Signal Processing, 2021.

[323] Drew Linsley, Alekh Karkada Ashok, Lakshmi Narasimhan Govindarajan, Rex Liu, and Thomas Serre. Stable and expressive recurrent vision models. In *Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.11362.

[324] Samuel Lippl, Benjamin Peters, and Kriegeskorte Nikolaus. Iterative convergent computation may not be a useful inductive bias for residual neural networks. bioRxiv (Cold Spring Harbor Laboratory), 2023.

[325] Aiwei Liu, Cheng Shi, Chuhan Wu, Ci Lei, Di Lu, Donald He, Fan Zhang, Fanhao Kong, Feifei Zhang, Guan Wang, Haicheng Wang, Haoyu Liu, Houjin Yu, Jiachen Ding, Jiayi Feng, Jie Zhou, Jijun Chi, Jindi Shi, Jing Lei, Junjie Zhang, Laiyi Li, Le Tian, Linhao Zhang, Miao Fan, Sijun Zhang, Wei Jia, Weiwei Shi, Wenhan Li, Wentao Zhao, Wenteng Liang, Xiao Zhou, Xiaojin Zhou, Xihuai Wang, Xinyu Gao, Xuanliang Wang, Xuyang Ao, Yang Yu, Yangxiu You, Yinuo Zhao, Yufei Kuang, Yufei Wang, Yuan Liu, Yuan Liu, Yuwen Chen, Zhencong Tian, Zhongyin Zhao, Zilin Yu, and Zitao Wang. Hidden Decoding at Scale: Latent Computation Scaling for Large Language Models. arXiv preprint arXiv:2607.08186, 2026. URL https://arxiv.org/abs/2607.08186.

[326] Enze Liu, Bowen Zheng, Xiaolei Wang, Wayne Xin Zhao, Jinpeng Wang, Sheng Chen, and Ji-Rong Wen. LARES: Latent Reasoning for Sequential Recommendation. arXiv preprint arXiv:2505.16865, 2025. URL https://arxiv.org/abs/2505.16865.

[327] Weihao Liu, Huangjie Zheng, Tianrong Chen, Rohit Dilip, Richard He Bai, Yizhu Jiao, Yuyang Wang, and Ruixiang Zhang. Decoding Looped Transformers Better for (Almost) Free. arXiv preprint arXiv:2610.02185, 2026. URL https://arxiv.org/abs/2610.02185.

[328] Yijin Liu, Fandong Meng, Yufeng Chen, Jinan Xu, and Jie Zhou. Depth-Adaptive Graph Recurrent Network for Text Classification. arXiv preprint arXiv:2003.00166, 2020. URL https://arxiv.org/abs/2003.00166.

[329] Yuxi Liu. Perfect diffusion is TC^0 – Bad diffusion is Turing-complete. arXiv preprint arXiv:2507.12469, 2025. URL https://arxiv.org/abs/2507.12469.

[330] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2402.14905.

[331] Joe Logan. Per-Token Fixed-Point Convergence in Depth-Recurrent Transformers. arXiv preprint arXiv:2607.14427, 2026. URL https://arxiv.org/abs/2607.14427.

[332] Binchun Lu, Lidan Fu, Juntao Ren, Yixuan Pan, and Yonggui Dong. TranIU-Net: Indicative Electrical Tomography Imaging Based on Implicit Unrolling Transformer. IEEE Transactions on Image Processing, 2026.

[333] Hongyuan Adam Lu, Z. L. Victor Wei, Qun Zhang, Jinrui Zeng, Bowen Cao, Lingwei Meng, Mocheng Li, Zezhong Wang, Haonan Yin, Naifu Xue, Minyu Chen, Cenyuan Zhang, Zefan Zhang, Hao Wei, Jiawei Zhou, Haoran Xu, Hao Yang, Ronglai Zuo, Tongda Xu, Yonghao Li, Jian Chen, Hebin Wang, Zeyu Gao, Yang Li, Wei Zhao, Qimin Zhong, Siqi Liu, Yumeng Zhang, Leyan Cui, Zhangyu Wang, and Wai Lam. Looped World Models. arXiv preprint arXiv:2606.18208, 2026. URL https://arxiv.org/abs/2606.18208.

[334] Wenquan Lu, Yuechuan Yang, Kyle Lee, Yanshu Li, and Enqi Liu. Latent Chain-of-Thought? Decoding the Depth-Recurrent Transformer. arXiv preprint arXiv:2507.02199, 2025. URL https://arxiv.org/abs/2507.02199.

[335] Xikun Lu, Yujian Ma, Yunda Chen, Xianquan Jiang, and Jinqiu Sang. Adaptive Depth and Expert Refinement for Efficient Speech Enhancement. arXiv preprint arXiv:2609.22824, 2026. URL https://arxiv.org/abs/2609.22824.

[336] Zhilin Lu, Rongpeng Li, Ming Lei, Chan Wang, Zhifeng Zhao, and Honggang Zhang. Self-Critical Alternate Learning based Semantic Broadcast Communication. *IEEE Transactions on Communications*, 2023. URL https://arxiv.org/abs/2312.01423.

[337] Zhiying Lu. LoopVAE: Recurrent Depth Across Scales for Visual Tokenization. arXiv preprint arXiv:2609.11516, 2026. URL https://arxiv.org/abs/2609.11516.

[338] Xinyue Luo and Fei Yu. Prediction Dynamics in Depth-Recurrent Language Models. arXiv preprint arXiv:2609.21383, 2026. URL https://arxiv.org/abs/2609.21383.

[339] Jonathan Lys, Vincent Gripon, Bastien Pasdeloup, Axel Marmoret, Lukas Mauch, Fabien Cardinaux, and Ghouthi Boukli Hacene. Inner Loop Inference for Pretrained Transformers: Unlocking Latent Capabilities Without Training. arXiv preprint arXiv:2602.14759, 2026. URL https://arxiv.org/abs/2602.14759.

[340] Lufan Ma, Tiancai Wang, Bin Dong, Jiangpeng Yan, Xiu Li, and Xiangyu Zhang. Implicit Feature Refinement for Instance Segmentation. In *ACM Multimedia*, 2021. URL https://arxiv.org/abs/2112.04709.

[341] Alexander V. Mantzaris. Exploring the Hierarchical Reasoning Model for Small Natural-Image Classification Without Augmentation. arXiv preprint arXiv:2510.03598, 2025. URL https://arxiv.org/abs/2510.03598.

[342] Omid Nejati Manzari, Guillaume Lajoie, and Hassan Rivaz. Recursive Vision Language Models for General Symbolic Reasoning. arXiv preprint arXiv:2608.01534, 2026. URL https://arxiv.org/abs/2608.01534.

[343] Andrei Marchenko, Viacheslav Bezrukov, Oleg Kashurin, Inessa Fedorova, Dmitry Bocharov, Yuliana Shakhvalieva, Maria Tikhonova, and Valerii Ternovskii. Closing the Loop: Practical Training Recipes for Looped Language Models. arXiv preprint arXiv:2610.00673, 2026. URL https://arxiv.org/abs/2610.00673.

[344] Sibylle Marcotte and Joan Bruna. Universal interpolation for deep residual self-attention networks. arXiv preprint arXiv:2610.01981, 2026. URL https://arxiv.org/abs/2610.01981.

[345] Ronan Killian McGovern. Test-time Adaptation of Tiny Recursive Models. arXiv preprint arXiv:2511.02886, 2025. URL https://arxiv.org/abs/2511.02886.

[346] Lane McIntosh, Niru Maheswaranathan, David Sussillo, and Jonathon Shlens. Recurrent Segmentation for Variable Computational Budgets. arXiv preprint arXiv:1711.10151, 2017. URL https://arxiv.org/abs/1711.10151.

[347] Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers Can Do Arithmetic with the Right Embeddings. In *Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.17399.

[348] Sean McLeish, Ang Li, John Kirchenbauer, Dayal Singh Kalra, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Jonas Geiping, Tom Goldstein, and Micah Goldblum. Teaching Pretrained Language Models to Think Deeper with Retrofitted Recurrence. arXiv preprint arXiv:2511.07384, 2025. URL https://arxiv.org/abs/2511.07384.

[349] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2403.03853.

[350] William Merrill and Ashish Sabharwal. A Little Depth Goes a Long Way: The Expressive Power of Log-Depth Transformers. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2503.03961.

[351] William Merrill and Ashish Sabharwal. Exact Expressive Power of Transformers with Padding. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.18948.

[352] Nicola Messina, Giuseppe Amato, Fabio Carrara, Claudio Gennaro, and Fabrizio Falchi. Recurrent Vision Transformer for Solving Visual Reasoning Problems. In *International Conference on Image Analysis and Processing*, 2021. URL https://arxiv.org/abs/2111.14576.

[353] Andrea Miele, Yiming Qin, Alba Carballo-Castro, Justin Deschenaux, and Pascal Frossard. Fixed-Point Masked Generative Modeling. arXiv preprint arXiv:2605.31215, 2026. URL https://arxiv.org/abs/2605.31215.

[354] Liana Mikaelyan, Ayyoob Imani, Mathew Salvaris, Parth Pathak, and Mohsen Fayyaz. DeltaLLM: Compress LLMs with Low-Rank Deltas between Shared Weights. arXiv preprint arXiv:2501.18596, 2025. URL https://arxiv.org/abs/2501.18596.

[355] Avni Mittal, Avinash Anand, Ashutosh Kumar, Dikshant Kukreja, Kritarth Prasad, Sushane Dulloo, Erik Cambria, Timothy Liu, Zhengkui Wang, and Rajiv Ratn Shah. TextNCA: Neural Cellular Automata for Language Modeling via Hierarchical Local Attention. arXiv preprint arXiv:2608.02050, 2026. URL https://arxiv.org/abs/2608.02050.

[356] Yosuke Miyanishi and Tetsuro Morimura. Interaction Locality in Hierarchical Recursive Reasoning. arXiv preprint arXiv:2605.20784, 2026. URL https://arxiv.org/abs/2605.20784.

[357] Jama Hussein Mohamud and Mirco Ravanelli. Random Recursive Models. arXiv preprint arXiv:2610.00541, 2026. URL https://arxiv.org/abs/2610.00541.

[358] Amirkeivan Mohtashami, Matteo Pagliardini, and Martin Jaggi. CoTFormer: A Chain-of-Thought Driven Architecture with Budget-Adaptive Computation Cost at Inference. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2310.10845.

[359] Giovanni Monea, Nathan Godey, Kianté Brantley, and Yoav Artzi. The State-Prediction Separation Hypothesis. arXiv preprint arXiv:2607.01218, 2026. URL https://arxiv.org/abs/2607.01218.

[360] Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. The Surprising Effectiveness of Shared Memory in Looped Transformers. arXiv preprint arXiv:2610.02383, 2026. URL https://arxiv.org/abs/2610.02383.

[361] Ibraheem Muhammad Moosa, Suhas Lohit, Ye Wang, Moitreya Chatterjee, and Wenpeng Yin. Understanding Dynamic Compute Allocation in Recurrent Transformers. arXiv preprint arXiv:2602.08864, 2026. URL https://arxiv.org/abs/2602.08864.

[362] Sajad Movahedi, Felix Sarnthein, Nicola Muca Cirone, and Antonio Orvieto. Fixed-Point RNNs: Interpolating from Diagonal to Dense. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2503.10799.

[363] Sajad Movahedi, Vera Milovanović, Shlomo Libo Feigin, Alexander Theus, Thomas Hofmann, Valentina Boeva, T. Konstantin Rusch, and Antonio Orvieto. Fixed-Point Reasoners: Stable and Adaptive Deep Looped Transformers. arXiv preprint arXiv:2606.18206, 2026. URL https://arxiv.org/abs/2606.18206.

[364] Michael C. Mozer, Denis Kazakov, and Robert V. Lindsey. State-Denoised Recurrent Neural Networks. arXiv preprint arXiv:1805.08394, 2018. URL https://arxiv.org/abs/1805.08394.

[365] Michael C. Mozer, Shoaib Ahmed Siddiqui, and Rosanne Liu. The Topological Trouble With Transformers. arXiv preprint arXiv:2604.17121, 2026. URL https://arxiv.org/abs/2604.17121.

[366] Tiberiu Musat. Neural Weight Norm = Kolmogorov Complexity. arXiv preprint arXiv:2605.10878, 2026. URL https://arxiv.org/abs/2605.10878.

[367] Seungjun Nah, Sanghyun Son, and Kyoung Mu Lee. Recurrent Neural Networks With Intra-Frame Iterations for Video Deblurring. IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2019.

[368] Nanbeige Lab, Chen Yang, Chengrui Huang, Fufeng Lan, Hanhui Chen, Hao Zhou, Huatong Song, Jiaqi Cao, Jiaying Zhu, Jinlin Niu, Kai Wang, Lisheng Huang, Qiliang Liang, Ran Le, Ruixiang Feng, Shuang Sun, Tao Gu, Tao Zhang, Tianyu Luo, Yang Song, Yun Xing, Yuntao Wen, Ziyao Xu, Zongchao Chen, and Zongqiang Li. Nanbeige4.2-3B: Unlocking Agentic Capabilities in a Compact Model. arXiv preprint arXiv:2607.22083, 2026. URL https://arxiv.org/abs/2607.22083.

[369] James O' Neill and Fergal Reid. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. arXiv preprint arXiv:2607.15456, 2026. URL https://arxiv.org/abs/2607.15456.

[370] Mark Neumann, Pontus Stenetorp, and Sebastian Riedel. Learning to Reason With Adaptive Computation. In *Neural Information Processing Systems*, 2016. URL https://arxiv.org/abs/1610.07647.

[371] Kei-Sing Ng and Qingchen Wang. Loop Neural Networks for Parameter Sharing. arXiv preprint arXiv:2409.14199, 2024. URL https://arxiv.org/abs/2409.14199.

[372] Kwok-Ho Ng, Tingting Song, Bingwen Feng, and Peiya Li. REIMU: Efficient Heterogeneous Hierarchical Reasoning for SSL-Based Speech Deepfake Detection. arXiv preprint arXiv:2608.00857, 2026. URL https://arxiv.org/abs/2608.00857.

[373] ngocdaobao. Looped transformers: A survey of recurrent-depth architectures for language models (paper list). GitHub repository, 2026. URL https://github.com/ngocdaobao/A-Survey-on-Looped-Transformers. Accessed 2026-10-05.

[374] Anthony Nguyen and Wenjun Lin. Intra-Layer Recurrence in Transformers for Language Modeling. *Canadian AI*, 2025. URL https://arxiv.org/abs/2505.01855.

[375] Minh Chau Nguyen, Truong Giang Le, and Viet Sang Dinh. HyperPolypDEQ: A Lightning Transformer-Based Deep Equilibrium Model for Colon Polyp Segmentation. Smart Systems and Devices, 2024.

[376] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large Language Diffusion Models. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.09992.

[377] Ying Nie, Kai Han, Hongguang Li, Hang Zhou, Tianyu Guo, Enhua Wu, Xinghao Chen, and Yunhe Wang. VersatileFFN: Achieving Parameter Efficiency in LLMs via Adaptive Wide-and-Deep Reuse. arXiv preprint arXiv:2512.14531, 2025. URL https://arxiv.org/abs/2512.14531.

[378] Lu ning Zhang, Jian wei Liu, Zhi yan Song, and Xin Zuo. Universal Transformer Hawkes Process with Adaptive Recursive Iteration. *Engineering applications of artificial intelligence*, 2021. URL https://arxiv.org/abs/2112.14479.

[379] Lu ning Zhang, Jian wei Liu, Zhi yan Song, Xin Zuo, Weimin Li, and Zeyu Liu. Universal Transformer Hawkes process. IEEE International Joint Conference on Neural Network, 2021.

[380] Yanan Niu. Repeated Shared Access Enables Grokking, but Edit Propagation Depends on an Addressable Memory. arXiv preprint arXiv:2606.20737, 2026. URL https://arxiv.org/abs/2606.20737.

[381] Mohammadmahdi Nouriborji, Omid Rohanian, Samaneh Kouchaki, and David A. Clifton. MiniALBERT: Model Distillation via Parameter-Efficient Recursive Transformers. In *Conference of the European Chapter of the Association for Computational Linguistics*, 2022. URL https://arxiv.org/abs/2210.06425.

[382] Mohammadmahdi Nouriborji, Morteza Rohanian, and Omid Rohanian. Improving Recursive Transformers with Mixture of LoRAs. arXiv preprint arXiv:2512.12880, 2025. URL https://arxiv.org/abs/2512.12880.

[383] Elnaz Nowrouzi, Zhiqun Zuo, Xueru Zhang, and Mohammad Mahdi Khalili. BRAID: Learning Equilibrium Maps in Interdependent Security Games via Weight-Tied Iterative Graph Neural Networks. arXiv preprint arXiv:2608.14856, 2026. URL https://arxiv.org/abs/2608.14856.

[384] Pawel Olszowiec, Michal Byra, Grzegorz Gruszczynski, Grzegorz Stefanski, and Alberto Presta. Same path, different: a mechanistic comparison of looped and stacked transformer encoders on 12-lead ECG. arXiv preprint arXiv:2609.15498, 2026. URL https://arxiv.org/abs/2609.15498.

[385] Kishan Padayachy, Ronald Richman, and Mario V. Wüthrich. Tab-TRM: Tiny Recursive Model for Insurance Pricing on Tabular Data. arXiv preprint arXiv:2601.07675, 2026. URL https://arxiv.org/abs/2601.07675.

[386] Yu Pan, Ye Yuan, Yichun Yin, Jiaxin Shi, Zenglin Xu, Ming Zhang, Lifeng Shang, Xin Jiang, and Qun Liu. Preparing Lessons for Progressive Training on Language Models. In *AAAI Conference on Artificial Intelligence*, 2024. URL https://arxiv.org/abs/2401.09192.

[387] Abhishek Panwar, Maheep Singh, and Saksham Bansal. Think Deep, Speak Once: Relit, A Recursive Latent Implicit Transformer Framework. arXiv preprint arXiv:2608.08113, 2026. URL https://arxiv.org/abs/2608.08113.

[388] Francesco Pappone, Donato Crisostomi, and Emanuele Rodolà. Two-Scale Latent Dynamics for Recurrent-Depth Transformers. arXiv preprint arXiv:2509.23314, 2025. URL https://arxiv.org/abs/2509.23314.

[389] Taekhyun Park, Yongjae Lee, Dohee Kim, and Hyerim Bae. LoopUS: Recasting Pretrained LLMs into Looped Latent Refinement Models. arXiv preprint arXiv:2605.11011, 2026. URL https://arxiv.org/abs/2605.11011.

[390] Liang Peng, Chenxiao Li, Libo Zhang, Xingping Dong, and Heng Fan. LoopTrack: A Simple Baseline for Parameter-Efficient Transformer Tracking. arXiv preprint arXiv:2609.33306, 2026. URL https://arxiv.org/abs/2609.33306.

[391] Jacob Pfau, William Merrill, and Samuel R. Bowman. Let's Think Dot by Dot: Hidden Computation in Transformer Language Models. arXiv preprint arXiv:2404.15758, 2024. URL https://arxiv.org/abs/2404.15758.

[392] Telmo Pessoa Pires, António V. Lopes, Yannick Assogba, and Hendra Setiawan. One Wide Feedforward is All You Need. In *Conference on Machine Translation*, 2023. URL https://arxiv.org/abs/2309.01826.

[393] Bryan A. Plummer, Nikoli Dryden, Julius Frost, Torsten Hoefler, and Kate Saenko. Neural Parameter Allocation Search. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/2006.10598.

[394] Ashwini Pokle, Zhengyang Geng, and Zico Kolter. Deep Equilibrium Approaches to Diffusion Models. In *Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2210.12867.

[395] Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Adaptive Depth in Looped Transformers: Diagnosing Learned Halting Gates and Trajectory Readouts. arXiv preprint arXiv:2607.20519, 2026. URL https://arxiv.org/abs/2607.20519.

[396] Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Looped Language Models Improve Compositional Tool Calling. arXiv preprint arXiv:2608.18171, 2026. URL https://arxiv.org/abs/2608.18171.

[397] Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling Laws For Stable Looped Language Models. arXiv preprint arXiv:2604.12946, 2026. URL https://arxiv.org/abs/2604.12946.

[398] Rojalina Priyadarshini and Subham Divakar. Rethinking Attention: Entropy-Guided Iterative Refinement for Language Models. Transactions on Artificial Intelligence, 2026.

[399] Muxin Pu, Mei Kuan Lim, Chun Yong Chong, and Chen Change Loy. LA-Sign: Looped Transformers with Geometry-aware Alignment for Skeleton-based Sign Language Recognition. arXiv preprint arXiv:2603.29057, 2026. URL https://arxiv.org/abs/2603.29057.

[400] Kaleem Ullah Qasim and Jiashu Zhang. Accelerating Training Speed of Tiny Recursive Models with Curriculum Guided Adaptive Recursion. arXiv preprint arXiv:2511.08653, 2025. URL https://arxiv.org/abs/2511.08653.

[401] Zhe Qian, Ziyang Gong, Zhongxing Xu, Hehan Li, Zhonghua Wang, Fei Luo, Mingxuan Wang, Xue Yang, Shiwei liu, Yanbiao Ma, Junchi Yan, and Jungong Han. LoopVL: Recurrent Visual Intelligence. arXiv preprint arXiv:2609.38426, 2026. URL https://arxiv.org/abs/2609.38426.

[402] Bosheng Qin, Haoji Roland Hu, and Yueting Zhuang. Deep Residual Weight-Sharing Attention Network With Low-Rank Attention for Visual Question Answering. IEEE Transactions on Multimedia, 2022.

[403] Zhijin Qin, Huiqiang Xie, and Xiaoming Tao. Mem-DeepSC: A Semantic Communication System with Memory. International Conference on Communications, 2023.

[404] Eric S. Qiu, Utku Umur Acikalin, Justin Lovelace, Christian Belardi, Arjun B. Mulchandani, Carla P. Gomes, and Kilian Q. Weinberger. MoRE: Mixture of Reused Experts. arXiv preprint arXiv:2609.18176, 2026. URL https://arxiv.org/abs/2609.18176.

[405] Saman Rahbar. The Ignition Index: Measuring Global Workspace Dynamics in Language Models. arXiv preprint arXiv:2608.05160, 2026. URL https://arxiv.org/abs/2608.05160.

[406] Sharan Ramjee. Ulterior Motives: Detecting Misaligned Reasoning in Continuous Thought Models. arXiv preprint arXiv:2604.23460, 2026. URL https://arxiv.org/abs/2604.23460.

[407] Gabriel Díaz Ramos, Toros Arikan, and Richard G. Baraniuk. MazeNet: An Accurate, Fast, and Scalable Deep Learning Solution for Steiner Minimum Trees. arXiv preprint arXiv:2410.18832, 2024. URL https://arxiv.org/abs/2410.18832.

[408] Zaccharie Ramzi, Pierre Ablin, Gabriel Peyré, and Thomas Moreau. Test like you Train in Implicit Deep Learning. arXiv preprint arXiv:2305.15042, 2023. URL https://arxiv.org/abs/2305.15042.

[409] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. arXiv preprint arXiv:2404.02258, 2024. URL https://arxiv.org/abs/2404.02258.

[410] Paulius Rauba, Claudio Fanconi, and Mihaela van der Schaar. Tiny Autoregressive Recursive Models. arXiv preprint arXiv:2603.08082, 2026. URL https://arxiv.org/abs/2603.08082.

[411] Machel Reid, Edison Marrese-Taylor, and Yutaka Matsuo. Subformer: Exploring Weight Sharing for Parameter Efficiency in Generative Transformers. In *Conference on Empirical Methods in Natural Language Processing*, 2021. URL https://arxiv.org/abs/2101.00234.

[412] Hangliang Ren. LSRL: Process-Supervised GRPO on Latent Recurrent States Improves Mathematical Reasoning. Conference on Empirical Methods in Natural Language Processing, 2025.

[413] Zirui Ren and Ziming Liu. Are Your Reasoning Models Reasoning or Guessing? A Mechanistic Analysis of Hierarchical Reasoning Models. arXiv preprint arXiv:2601.10679, 2026. URL https://arxiv.org/abs/2601.10679.

[414] Ivan Rodkin, Daniil Orel, Konstantin Smirnov, Arman Bolatov, Bilal Elbouardi, Besher Hassan, Yuri Kuratov, Aydar Bulatov, Preslav Nakov, Timothy Baldwin, Artem Shelmanov, and Mikhail Burtsev. Beyond Memorization: Extending Reasoning Depth with Recurrence, Memory and Test-Time Compute Scaling. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL https://arxiv.org/abs/2508.16745.

[415] Leo Raphael Rodrigues and Jian Kang. Dissecting Hierarchical Reasoning Models: A Mechanistic Study. arXiv preprint arXiv:2609.22197, 2026. URL https://arxiv.org/abs/2609.22197.

[416] Thomas Rolland and Alberto Abad. Exploring Shared-Weight Mechanisms in Transformer and Conformer Architectures for Automatic Speech Recognition. Interspeech (USB), 2025.

[417] Thomas Rolland, Carlos Carvalho, and Alberto Abad. Rethinking Depth: A study of the Recursive-Transformer for Speech Recognition. arXiv preprint arXiv:2606.09357, 2026. URL https://arxiv.org/abs/2606.09357.

[418] Thomas Manuel Rost. Inference-Path Optimization via Circuit Duplication in Frozen Visual Transformers for Marine Species Classification. arXiv preprint arXiv:2604.03428, 2026. URL https://arxiv.org/abs/2604.03428.

[419] Antonio Roye-Azar, Santiago Vargas-Naranjo, Dhruv Ghai, Nithin Balamurugan, and Rayan Amir. Tiny Recursive Models on ARC-AGI-1: Inductive Biases, Identity Conditioning, and Test-Time Compute. arXiv preprint arXiv:2512.11847, 2025. URL https://arxiv.org/abs/2512.11847.

[420] Alon Rozental, Zohar Kelrich, and Daniel Fleischer. Latent Universal Task-Specific BERT. arXiv preprint arXiv:1905.06638, 2019. URL https://arxiv.org/abs/1905.06638.

[421] Benjamin Rozonoyer, Jacopo Minniti, Dhruvesh Patel, Neil Band, Avishek Joey Bose, Tim G. J. Rudner, and Andrew McCallum. Learned Relay Representations for Forward-Thinking Discrete Diffusion Models. arXiv preprint arXiv:2605.22967, 2026. URL https://arxiv.org/abs/2605.22967.

[422] T. Konstantin Rusch, Tim Seyde, Jared Boyer, Zach J. Patterson, and Daniela Rus. Looped Actor: Depth-Recurrent Reasoning Models for Reinforcement Learning. arXiv preprint arXiv:2609.37432, 2026. URL https://arxiv.org/abs/2609.37432.

[423] Lucas Saldyt and Subbarao Kambhampati. Algorithmic Language Models with Neurally Compiled Libraries. arXiv preprint arXiv:2407.04899, 2024. URL https://arxiv.org/abs/2407.04899.

[424] Grigory Sapunov. Universal Transformers Need Memory: Depth-State Trade-offs in Adaptive Recursive Reasoning. arXiv preprint arXiv:2604.21999, 2026. URL https://arxiv.org/abs/2604.21999.

[425] Swayam Prabha Satpathy, Akash Sahoo, Rabi Mahapatra, and Amita Sahoo. Intent Reasoning from User Locale Data using Tiny Recursive Models. International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT), 2026.

[426] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2502.17416.

[427] Pedro Savarese. Principled Approximation Methods for Efficient and Scalable Deep Learning. arXiv preprint arXiv:2509.00174, 2025. URL https://arxiv.org/abs/2509.00174.

[428] Peter Schneider-Kamp, Jacob Nielsen, Gianluca Barmina, Kenneth Enevoldsen, and Lukas Galke Poech. DFM Mimir v1: An Open HRM Delivering Frontier Performance at 1B Parameters Using Only Permissible Post-Training Data. arXiv preprint arXiv:2608.13517, 2026. URL https://arxiv.org/abs/2608.13517.

[429] Mark Schöne, Babak Rahmani, Heiner Kremer, Fabian Falck, Hitesh Ballani, and Jannes Gladrow. Implicit Language Models are RNNs: Balancing Parallelization and Expressivity. In *International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2502.07827.

[430] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident Adaptive Language Modeling. In *Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2207.07061.

[431] Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can You Learn an Algorithm? Generalizing from Easy to Hard Problems with Recurrent Networks. In *Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2106.04537.

[432] Avi Schwarzschild, Arjun Gupta, Amin Ghiasi, Micah Goldblum, and Tom Goldstein. The Uncanny Similarity of Recurrence and Depth. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2102.11011.

[433] Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. arXiv preprint arXiv:2604.21106, 2026. URL https://arxiv.org/abs/2604.21106.

[434] Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. arXiv preprint arXiv:2608.09444, 2026. URL https://arxiv.org/abs/2608.09444.

[435] Lee Seung-woo and Bowen Qi. AdaLoop: Adaptive-Depth Latent Reasoning for Audio Language Models. arXiv preprint arXiv:2610.06949, 2026. URL https://arxiv.org/abs/2610.06949.

[436] Amin Sghaier, Ali Parviz, and Alexia Jolicoeur-Martineau. Probabilistic Tiny Recursive Model. arXiv preprint arXiv:2605.19943, 2026. URL https://arxiv.org/abs/2605.19943.

[437] Pervez Shaik, Prosenjit Biswas, Abhinav Thorat, Ravi Kolla, and Niranjan Pedanekar. RecRec: Recursive Refinement for Sequential Recommendation. arXiv preprint arXiv:2607.10541, 2026. URL https://arxiv.org/abs/2607.10541.

[438] Yuliana Shakhvalieva, Dmitrii Kharchev, Viacheslav Bezrukov, Inessa Fedorova, Dmitry Bocharov, Ivan Oseledets, and Valerii Ternovskii. What Limits Recursive Reasoning Models: Optimization, Architecture and Test-Time Scaling. arXiv preprint arXiv:2609.39967, 2026. URL https://arxiv.org/abs/2609.39967.

[439] Mark Shapiro. Retrofitting Recurrent Depth into a Pretrained Language Model: Installation, Extrapolation, Transfer, and Retention at Two Parameter Budgets. arXiv preprint arXiv:2608.11233, 2026. URL https://arxiv.org/abs/2608.11233.

[440] Rituraj Sharma and Tu Vu. Dense Supervision Is Not Enough: The Readout Blind Spot in Looped Language Models. arXiv preprint arXiv:2606.24898, 2026. URL https://arxiv.org/abs/2606.24898.

[441] Peter Shaw, James Cohan, Jacob Eisenstein, Kenton Lee, Jonathan Berant, and Kristina Toutanova. ALTA: Compiler-Based Analysis of Transformers. *Trans. Mach. Learn. Res.*, 2024. URL https://arxiv.org/abs/2410.18077.

[442] Boyang Shen, Kaixiang Yang, Hao Wang, Qiuyu Yu, Qiang Xie, Qiang Li, and Zhiwei Wang. LoopVLA: Learning Sufficiency in Recurrent Refinement for Vision-Language-Action Models. arXiv preprint arXiv:2605.09948, 2026. URL https://arxiv.org/abs/2605.09948.

[443] Jucheng Shen, Barbara Su, and Anastasios Kyrillidis. One Model, Two Roles: Emergent Specialization in a Shared Recurrent Transformer. arXiv preprint arXiv:2605.17811, 2026. URL https://arxiv.org/abs/2605.17811.

[444] Zhiqiang Shen, Zechun Liu, and Eric Xing. Sliced Recursive Transformer. In *European Conference on Computer Vision*, 2021. URL https://arxiv.org/abs/2111.05297.

[445] Zekai Shi. Think Sparse, Predict Dense: Continuous Thought Machines for Image Super-Resolution. arXiv preprint arXiv:2607.18856, 2026. URL https://arxiv.org/abs/2607.18856.

[446] Kyuhong Shim, Jinkyu Lee, and Hyunjae Kim. Leveraging Adapter for Parameter-Efficient ASR Encoder. Interspeech (USB), 2024.

[447] Zachary Shinnick, Christian Internò, Hemanth Saratchandran, Anton van den Hengel, and Damien Teney. Procedural Core: A Compact Recurrent Initialization for Vision Transformers. arXiv preprint arXiv:2609.37631, 2026. URL https://arxiv.org/abs/2609.37631.

[448] Behzad Shomali, Markus Frey, David Berghaus, Joachim Koehler, and Mehdi Ali. LoopMTP: A looped transformer guided by latent multi-token prediction. arXiv preprint arXiv:2608.03624, 2026. URL https://arxiv.org/abs/2608.03624.

[449] Wen-Jie Shu, Xuerui Qiu, Rui-Jie Zhu, Harold Haodong Chen, Yexin Liu, and Harry Yang. LoopViT: Scaling Visual ARC with Looped Transformers. arXiv preprint arXiv:2602.02156, 2026. URL https://arxiv.org/abs/2602.02156.

[450] Prateek Kumar Sikdar. A rigor-matched audit of periodic-step layer skipping for efficient LLM inference: conflayers versus swift, with a supplemental analysis of trained routing alternatives. arXiv preprint arXiv:2608.28846, 2026. URL https://arxiv.org/abs/2608.28846.

[451] Antoine Simoulin and Benoît Crabbé. How Many Layers and Why? An Analysis of the Model Depth in Transformers. Annual Meeting of the Association for Computational Linguistics, 2021.

[452] Akshit Singh, Shyam Marjit, Wei Lin, Leonid Karlinsky, and M. Jehanzeb Mirza. Architectural Sampling: Test-Time Scaling via Computational Diversity in Frozen Vision-Language Models. arXiv preprint arXiv:2610.01687, 2026. URL https://arxiv.org/abs/2610.01687.

[453] Anjani Sirivella, Aanisha Newaz, and Glaucia Melo. Evaluating Tiny Recursive Models Across Training for Code Generation. arXiv preprint arXiv:2608.29376, 2026. URL https://arxiv.org/abs/2608.29376.

[454] Said Slaoui. S-AI-Recursive: Convergent Recursive Reasoning. arXiv preprint arXiv:2605.13872, 2026. URL https://arxiv.org/abs/2605.13872.

[455] James Seale Smith, Chi-Heng Lin, Shikhar Tuli, Haris Jeelani, Shangqian Gao, Yilin Shen, Hongxia Jin, and Yen-Chang Hsu. FlexiGPT: Pruning and Extending Large Language Models with Low-Rank Weight Sharing. In *North American Chapter of the Association for Computational Linguistics*, 2025. URL https://arxiv.org/abs/2501.14713.

[456] Haobo Song, Soumajit Majumder, and Tao Lin. Revisiting Implicit Models: Sparsity Trade-offs Capability in Weight-tied Model for Vision Tasks. arXiv preprint arXiv:2307.08013, 2023. URL https://arxiv.org/abs/2307.08013.

[457] Shixiang Song, He Li, Zitong Wang, Boyi Zeng, Feichen Song, Yixuan Wang, Zhiqin John Xu, Ziwei He, and Zhouhan Lin. AdaPonderLM: Gated Pondering Language Models with Token-Wise Adaptive Depth. arXiv preprint arXiv:2603.01914, 2026. URL https://arxiv.org/abs/2603.01914.

[458] Yiran Song and Lizhuang Ma. CLMAE: A Liter and Faster Masked Autoencoders. IEEE International Conference on Acoustics Speech and Signal Processing, 2023.

[459] Asael Sorensen, Charles Brock, David Chamberlain, Jennifer Minnich, Matthew Hoffman, and Ramyaa Ramyaa. Stream Recursion Model (SRM). arXiv preprint arXiv:2609.28809, 2026. URL https://arxiv.org/abs/2609.28809.

[460] Indro Spinelli, Simone Scardapane, and Aurelio Uncini. Adaptive Propagation Graph Convolutional Network. *IEEE Transactions on Neural Networks and Learning Systems*, 2020. URL https://arxiv.org/abs/2002.10306.

[461] Ayhan Suleymanzade, Chanhyuk Lee, Floor Eijkelboom, Nicholas M. Boffi, İsmail İlkan Ceylan, and Jinwoo Kim. Thinking with Looped Flows. arXiv preprint arXiv:2609.11801, 2026. URL https://arxiv.org/abs/2609.11801.

[462] Haotian Sun, Rushi Qiang, Yuqian Zheng, and Bo Dai. Forward-Free Diffusion Language Models with BPTT-Free Looped Refinement. arXiv preprint arXiv:2606.08357, 2026. URL https://arxiv.org/abs/2606.08357.

[463] Qi Sun, Marc Pickett, Aakash Kumar Nain, and Llion Jones. Transformer Layers as Painters. In *AAAI Conference on Artificial Intelligence*, 2024. URL https://arxiv.org/abs/2407.09298.

[464] Wenfang Sun, Xinyuan Song, Pengxiang Li, Lu Yin, Yefeng Zheng, and Shiwei Liu. The Curse of Depth in Large Language Models. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.05795.

[465] Yang Sun, Fajie Yuan, Min Yang, Guoao Wei, Zhou Zhao, and Duo Liu. A Generic Network Compression Framework for Sequential Recommender Systems. In *Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020. URL https://arxiv.org/abs/2004.13139.

[466] Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. Universal YOCO for Efficient Depth Scaling. arXiv preprint arXiv:2604.01220, 2026. URL https://arxiv.org/abs/2604.01220.

[467] Yuwei Sun, Yuxuan Yao, Hui Li, and Siyu Zhu. The Thinking Pixel: Recursive Sparse Reasoning in Multimodal Diffusion Latents. arXiv preprint arXiv:2604.25299, 2026. URL https://arxiv.org/abs/2604.25299.

[468] Anej Svete and Ashish Sabharwal. On the Reasoning Abilities of Masked Diffusion Language Models. arXiv preprint arXiv:2510.13117, 2025. URL https://arxiv.org/abs/2510.13117.

[469] Anej Svete, William Merrill, Ryan Cotterell, and Ashish Sabharwal. Revisiting Padded Transformer Expressivity: Which Architectural Choices Matter and Which Don't. arXiv preprint arXiv:2605.30523, 2026. URL https://arxiv.org/abs/2605.30523.

[470] Sho Takase and Shun Kiyono. Lessons on Parameter Sharing across Layers in Transformers. *SUSTAINLP*, 2021. URL https://arxiv.org/abs/2104.06022.

[471] Shota Takashiro, Masanori Koyama, Takeru Miyato, Yusuke Iwasawa, Yutaka Matsuo, and Kohei Hayashi. Exploration of Fast-Slow Latent Recurrence for Train-Short, Test-Long Generalization. arXiv preprint arXiv:2604.01577, 2026. URL https://arxiv.org/abs/2604.01577.

[472] Aviv Tamar, Yi Wu, Garrett Thomas, Sergey Levine, and Pieter Abbeel. Value Iteration Networks. In *Neural Information Processing Systems*, 2016. URL https://arxiv.org/abs/1602.02867.

[473] Cao Thien Tan, Phan Thi Thu Trang, Do Nghiem Duc, Ho Ngoc Anh, Hanyang Zhuang, and Nguyen Duc Dung. UCAN: Unified Convolutional Attention Network for Expansive Receptive Fields in Lightweight Super-Resolution. arXiv preprint arXiv:2603.11680, 2026. URL https://arxiv.org/abs/2603.11680.

[474] Jia Huei Tan, Ying Hua Tan, Chee Seng Chan, and Joon Huang Chuah. ACORT: A Compact Object Relation Transformer for Parameter Efficient Image Captioning. *Neurocomputing*, 2022. URL https://arxiv.org/abs/2202.05451.

[475] Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. Sparse Universal Transformer. In *Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2310.07096.

[476] Zheyue Tan, Zhiyuan Li, Tao Yuan, Dong Zhou, Weilin Liu, Yueqing Zhuang, Yadong Li, Guowei Niu, Cheng Qin, Zhuyu Yao, Congyi Liu, Haiyang Xu, Boxun Li, Guohao Dai, Bo Zhao, and Yu Wang. ReXMoE: Reusing Experts with Minimal Overhead in Mixture-of-Experts. arXiv preprint arXiv:2510.17483, 2025. URL https://arxiv.org/abs/2510.17483.

[477] Guo Tang, Shixin Jiang, Heng Chang, Nuo Chen, Yuhan Li, Huiming Fan, Jia Li, Ming Liu, and Bing Qin. LoopRPT: Reinforcement Pre-Training for Looped Language Models. arXiv preprint arXiv:2603.19714, 2026. URL https://arxiv.org/abs/2603.19714.

[478] Hao Tang, Zhiao Huang, Jiayuan Gu, Bao-Liang Lu, and Hao Su. Towards Scale-Invariant Graph-related Problem Solving by Iterative Homogeneous Graph Neural Networks. arXiv preprint arXiv:2010.13547, 2020. URL https://arxiv.org/abs/2010.13547.

[479] Haoyu Tang, Zhaoyi Liu, Chang Zeng, and Xinfeng Li. Beyond Universal Transformer: block reusing with adaptor in Transformer for automatic speech recognition. In *International Symposium on Neural Networks*, 2023. URL https://arxiv.org/abs/2303.13072.

[480] Jiakai Tang, Runfeng Zhang, Weiqiu Wang, Yifei Liu, Chuan Wang, Xu Chen, Yeqiu Yang, Jian Wu, Yuning Jiang, and Bo Zheng. LoopCTR: Unlocking the Loop Scaling Power for Click-Through Rate Prediction. arXiv preprint arXiv:2604.19550, 2026. URL https://arxiv.org/abs/2604.19550.

[481] Yi Tay, Mostafa Dehghani, Samira Abnar, Hyung Won Chung, William Fedus, Jinfeng Rao, Sharan Narang, Vinh Q. Tran, Dani Yogatama, and Donald Metzler. Scaling Laws vs Model Architectures: How does Inductive Bias Influence Scaling? In *Conference on Empirical Methods in Natural Language Processing*, 2022. URL https://arxiv.org/abs/2207.10551.

[482] Roblex Nana Tchakoute and Claude Tadonki. Energy-Aware Deep Learning on GPUs through Parameter Sharing and Mixed Precision Training. 2025 IEEE, 2025.

[483] TechCrunch. OpenAI's new reasoning technique alarms AI safety experts. TechCrunch, September 2026. URL https://techcrunch.com/2026/09/02/openais-new-reasoning-technique-alarms-ai-safety-experts/. Accessed 2026-10-05.

[484] Zachary Teed and Jia Deng. RAFT: Recurrent All-Pairs Field Transforms for Optical Flow. In *European Conference on Computer Vision*, 2020. URL https://arxiv.org/abs/2003.12039.

[485] Xinyu Tian and Xiaotong Shen. Statistical Gains from Looped Estimation under Parameter Budgets. arXiv preprint arXiv:2609.25778, 2026. URL https://arxiv.org/abs/2609.25778.

[486] Akiyoshi Tomihari and Ryo Karakida. Recurrent Self-Attention Dynamics: An Energy-Agnostic Perspective from Jacobians. In *Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.19458.

[487] Kohei Tsuchiyama, Andre Roehm, Takatomo Mihana, and Ryoichi Horisaki. Time-multiplexed layer reuse for physical neural networks. arXiv preprint arXiv:2511.00044, 2025. URL https://arxiv.org/abs/2511.00044.

[488] Yalcin Tur, Jalal Naghiyev, Haoquan Fang, Wei-Chuan Tsai, Jiafei Duan, Dieter Fox, and Ranjay Krishna. Recurrent-Depth VLA: Implicit Test-Time Compute Scaling of Vision-Language-Action Models via Latent Iterative Reasoning. arXiv preprint arXiv:2602.07845, 2026. URL https://arxiv.org/abs/2602.07845.

[489] Mehmet Kerem Turkcan. Loom: A Scalable Analytical Neural Computer Architecture. arXiv preprint arXiv:2604.08816, 2026. URL https://arxiv.org/abs/2604.08816.

[490] Cem Üyük, Mike Lasby, Mohamed Yassin, Utku Evci, and Yani Ioannou. Learning Fine-grained Parameter Sharing via Sparse Tensor Decomposition. *Trans. Mach. Learn. Res.*, 2024. URL https://arxiv.org/abs/2411.09816.

[491] Geoffroy Vanderreydt, Jenthe Thienpondt, and Kris Demuynck. Memory-efficient Model Growth for Finetuning Automatic Speech Recognition Models. International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA), 2025.

[492] Vijay Veerabadran, Srinivas Ravishankar, Yuan Tang, Ritik Raina, and Virginia R. de Sa. Adaptive recurrent vision performs zero-shot computation scaling to unseen difficulty levels. In *Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2311.06964.

[493] Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. arXiv preprint arXiv:2605.07721, 2026. URL https://arxiv.org/abs/2605.07721.

[494] Neha Verma, Kenton Murray, and Kevin Duh. Merging Feed-Forward Sublayers for Compressed Transformers. arXiv preprint arXiv:2501.06126, 2025. URL https://arxiv.org/abs/2501.06126.

[495] Ivan Viakhirev, Kirill Borodin, Amirah Almutairi, Serguei Barannikov, Maxim Abramov, and Grach Mkrtchian. Think Shallow, Solve Deep: Controlling Recurrent Dynamics for Reliable Test-Time Depth. arXiv preprint arXiv:2608.18222, 2026. URL https://arxiv.org/abs/2608.18222.

[496] Ke Wan and Chen Chen. Attention Routing Stabilizes Early: Working-Set Inference for Recurrent Language Models. arXiv preprint arXiv:2609.27373, 2026. URL https://arxiv.org/abs/2609.27373.

[497] Binshuai Wang, Peng Wei, and Mahyar Ghazanfari. Linear Reusable Neural Bases Architecture for Network Compression. arXiv preprint arXiv:2609.01550, 2026. URL https://arxiv.org/abs/2609.01550.

[498] Boyuan Wang, Chengyao Yu, Jiaxi Ren, Hongxin Wei, Bingyi Jing, and Yuxin Tao. Scheduling Recursive Reasoning in Looped Transformers. arXiv preprint arXiv:2609.36653, 2026. URL https://arxiv.org/abs/2609.36653.

[499] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical Reasoning Model. arXiv preprint arXiv:2506.21734, 2025. URL https://arxiv.org/abs/2506.21734.

[500] Guan Wang, Changling Liu, Chenyu Wang, Cai Zhou, Yuhao Sun, Yifei Wu, Shuai Zhen, Luca Scimeca, and Yasin Abbasi Yadkori. HRM-Text: Efficient Pretraining Beyond Scaling. arXiv preprint arXiv:2605.20613, 2026. URL https://arxiv.org/abs/2605.20613.

[501] Guangzhi Wang, Kai Li, Yinghao Jiao, and Zhi Liu. Refine Thought: A Test-Time Inference Method for Embedding Model Reasoning. arXiv preprint arXiv:2511.13726, 2025. URL https://arxiv.org/abs/2511.13726.

[502] Han Wang, Ishwar B Balappanawar, and Huan Zhang. On the Chain-of-Thought Monitorability of Looped Language Models. arXiv preprint arXiv:2610.02741, 2026. URL https://arxiv.org/abs/2610.02741.

[503] Haoyu Wang, Siyuan Wang, Yaguang Gong, and Wei-Qiang Zhang. DistilALHuBERT: A Distilled Parameter Sharing Audio Representation Model. International Conference on Signal Processing and Machine Learning, 2023.

[504] Jiayun Wang, Yubei Chen, Stella X. Yu, Brian Cheung, and Yann LeCun. Compact and Optimal Deep Learning with Recurrent Parameter Generators. arXiv preprint arXiv:2107.07110, 2021. URL https://arxiv.org/abs/2107.07110.

[505] Jingcun Wang, Yu-Guang Chen, Ing-Chao Lin, Bing Li, and Grace Li Zhang. Basis Sharing: Cross-Layer Parameter Sharing for Large Language Model Compression. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2410.03765.

[506] Shaowen Wang, Bingrui Li, Ge Zhang, Wenhao Huang, Shen Yan, and Jian Li. On the Residual Scaling of Looped Transformers: Stability and Transferability. arXiv preprint arXiv:2606.18524, 2026. URL https://arxiv.org/abs/2606.18524.

[507] Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers. arXiv preprint arXiv:2609.01343, 2026. URL https://arxiv.org/abs/2609.01343.

[508] Shijie Wang, Xiangzhao Hao, Yueti Li, Guangyu Cao, Xinyu Tang, and Haiyun Guo. ReLoopUME: Recurrent Depth with Learnable Retrieval Registers for Universal Multimodal Embedding. arXiv preprint arXiv:2607.28751, 2026. URL https://arxiv.org/abs/2607.28751.

[509] Shouren Wang, Chuang Ma, Mohsen Hariri, Debargha Ganguly, Wang Yang, Xiaoqing Tong, Qianying Liu, Xiaotian Han, and Vipin Chaudhary. How to Loop MoE: Flatten the Experts, Untie the Attention. arXiv preprint arXiv:2609.35751, 2026. URL https://arxiv.org/abs/2609.35751.

[510] Shuai Wang, Yao Teng, and Limin Wang. Deep Equilibrium Object Detection. In *IEEE International Conference on Computer Vision*, 2023. URL https://arxiv.org/abs/2308.09564.

[511] Shuai Wang, Yao Teng, and Limin Wang. Deep Equilibrium Object Detection and Segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2025.

[512] Wenlong Wang and Fergal Reid. Tiny Recursive Reasoning with Mamba-2 Attention Hybrid. arXiv preprint arXiv:2602.12078, 2026. URL https://arxiv.org/abs/2602.12078.

[513] Wenlong Wang and Fergal Reid. Looped Transformers under the Jacobian Lens: Does the Global Workspace Survive Recurrence? arXiv preprint arXiv:2609.01924, 2026. URL https://arxiv.org/abs/2609.01924.

[514] Xi Wang, Ziyang Cai, Zheng Zhan, Harry Dong, Ying Fan, Gustavo de Rosa, Tim Pearce, and John Langford. Full-bandwidth transformer. arXiv preprint arXiv:2608.08888, 2026. URL https://arxiv.org/abs/2608.08888.

[515] Xun Wang, Ruishuo Chen, Yu Chen, Zhuoran Li, and Longbo Huang. FlexLoop: Depth-Elastic Looped Policies for Adaptive Test-Time Computation in Deep RL. arXiv preprint arXiv:2609.34488, 2026. URL https://arxiv.org/abs/2609.34488.

[516] Yiming Wang and Jinyu Li. ResidualTransformer: Residual Low-Rank Learning with Weight-Sharing for Transformer Layers. In *IEEE International Conference on Acoustics, Speech, and Signal Processing*, 2023. URL https://arxiv.org/abs/2310.02489.

[517] Yixuan Wang, Haoyu Qiao, Lujun Li, Qingfu Zhu, and Wanxiang Che. CommonKV: Compressing KV Cache with Cross-layer Parameter Sharing. arXiv preprint arXiv:2508.16134, 2025. URL https://arxiv.org/abs/2508.16134.

[518] Yuxiang Wang, Kunyu Feng, Yingda Shen, Haoning Xu, Junyu Wang, and Zhizheng Wu. RecurTrace: Adaptive Latent Reasoning with Loop-Time Memory. arXiv preprint arXiv:2609.03379, 2026. URL https://arxiv.org/abs/2609.03379.

[519] Zihan Wang, Rui Pan, Jiarui Yao, Robert Csordas, Linjie Li, Lu Yin, Jiajun Wu, Tong Zhang, Manling Li, and Shiwei Liu. Chain-of-Experts: Unlocking the Communication Power of Mixture-of-Experts Models. arXiv preprint arXiv:2506.18945, 2025. URL https://arxiv.org/abs/2506.18945.

[520] Ziyu Wang, Shuangpeng Han, and Mengmi Zhang. PRISM: Progressive Reasoning through Iterative Slot Memory for Vision. arXiv preprint arXiv:2605.30942, 2026. URL https://arxiv.org/abs/2605.30942.

[521] Artit Wangperawong. Attending to Mathematical Language with Transformers. arXiv preprint arXiv:1812.02825, 2018. URL https://arxiv.org/abs/1812.02825.

[522] Sachini Weerasekara, Sagar Kamarthi, and Jacqueline Isaacs. REFINEPPO: Learning Continuous Control Policies by Iterative Action Refinement. arXiv preprint arXiv:2609.21108, 2026. URL https://arxiv.org/abs/2609.21108.

[523] Xilin Wei, Xiaoran Liu, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Jiaqi Wang, Xipeng Qiu, and Dahua Lin. SIM-CoT: Supervised Implicit Chain-of-Thought. arXiv preprint arXiv:2509.20317, 2025. URL https://arxiv.org/abs/2509.20317.

[524] Zichao Wei. On the Spatiotemporal Dynamics of Generalization in Neural Networks. arXiv preprint arXiv:2602.01651, 2026. URL https://arxiv.org/abs/2602.01651.

[525] Zichao Wei. On the Mirage of Long-Range Dependency, with an Application to Integer Multiplication. arXiv preprint arXiv:2603.29069, 2026. URL https://arxiv.org/abs/2603.29069.

[526] Justus Westerhoff, Stephan Olbrich, Hatem Oraby, Matthew Evan Larkum, and Felix Alexander Gers. Programs-of-Layers in LLMs through the Lens of Cortical Areas. arXiv preprint arXiv:2609.31360, 2026. URL https://arxiv.org/abs/2609.31360.

[527] Sergio Ryan Wibisono and Achmad Imam Kistijantoro. Log Anomaly Detection Using Adaptive Universal Transformer. 2019 International Conference of Advanced Informatics: Concepts, Theory and Applications (ICAICTA), 2019.

[528] Nugroho Wicaksono, Lailil Muflikhah, and Novanto Yudistira. IDiffPose: Equilibrium Substitution for Lightweight Diffusion-Graph-Based Pose Estimation. ACM International Conference on Multimedia in Asia (MMAsia), 2025.

[529] Minduli Wijayatunga and Roberto Armellin. Tiny Recursive Models for Solving the J2-Perturbed Lambert Problem. arXiv preprint arXiv:2606.00895, 2026. URL https://arxiv.org/abs/2606.00895.

[530] Jonathan Williams and Esin Tureci. Prioritize the Process, Not Just the Outcome: Rewarding Latent Thought Trajectories Improves Reasoning in Looped Language Models. arXiv preprint arXiv:2602.10520, 2026. URL https://arxiv.org/abs/2602.10520.

[531] Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. arXiv preprint arXiv:2510.24824, 2025. URL https://arxiv.org/abs/2510.24824.

[532] Jiaju Wu, Yi Hu, and Muhan Zhang. Shared Weights, Selected Computations: How Looped Transformers Route What Each Loop Does. arXiv preprint arXiv:2609.39892, 2026. URL https://arxiv.org/abs/2609.39892.

[533] Lyumin Wu, Chenyang Zhang, and Yuan Cao. Looped Transformers with Layer Normalization Provably Learn the Power Method. arXiv preprint arXiv:2606.00605, 2026. URL https://arxiv.org/abs/2606.00605.

[534] Heming Xia, Yongqi Li, Jun Zhang, Cunxiao Du, and Wenjie Li. SWIFT: On-the-Fly Self-Speculative Decoding for LLM Inference Acceleration. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2410.06916.

[535] Shi-Yu Xia, Wenxuan Zhu, Xu Yang, and Xin Geng. Exploring Learngene via Stage-wise Weight Sharing for Initializing Variable-sized Models. In *International Joint Conference on Artificial Intelligence*, 2024. URL https://arxiv.org/abs/2404.16897.

[536] He Xian, Zixin Wang, Honggang Yuan, Ting Wang, and Yuanming Shi. Latency-Minimal Decentralized LAM Training with Looped Transformers in Heterogeneous LEO Satellite Constellations. Globecom, 2025.

[537] Liu Xiao. Associative-State Universal Transformers: Sparse Retrieval Meets Structured Recurrence. arXiv preprint arXiv:2604.25930, 2026. URL https://arxiv.org/abs/2604.25930.

[538] Huiqiang Xie, Zhijin Qin, and Geoffrey Ye Li. Semantic Communication with Memory. *IEEE Journal on Selected Areas in Communications*, 2023. URL https://arxiv.org/abs/2303.12335.

[539] Yucheng Xin, Runci Bai, Yongcong Wang, Guangwei Gao, Jiao Liu, Dianjie Lu, Linwei Fan, and Zhuoran Zheng. LoopMoEVR: Loop-Based Degradation-Aware Mixture-of-Experts for Unified UHD Video Restoration. arXiv preprint arXiv:2610.05109, 2026. URL https://arxiv.org/abs/2610.05109.

[540] Haofei Xu, Daniel Barath, Andreas Geiger, and Marc Pollefeys. ReSplat: Learning Recurrent Gaussian Splatting. arXiv preprint arXiv:2510.08575, 2025. URL https://arxiv.org/abs/2510.08575.

[541] Kevin Xu and Issei Sato. On Expressive Power of Looped Transformers: Theoretical Analysis and Enhancement via Timestep Encoding. In *International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2410.01405.

[542] Kevin Xu and Issei Sato. To CoT or To Loop? A Formal Comparison Between Chain-of-Thought and Looped Transformers. arXiv preprint arXiv:2505.19245, 2025. URL https://arxiv.org/abs/2505.19245.

[543] Kevin Xu and Issei Sato. A Formal Comparison Between Chain of Thought and Latent Thought. arXiv preprint arXiv:2509.25239, 2025. URL https://arxiv.org/abs/2509.25239.

[544] Ruihan Xu, Yuting Gao, Lan Wang, Jianing Li, Weihao Chen, Qingpei Guo, Ming Yang, and Shiliang Zhang. Looping Back to Move Forward: Recursive Transformers for Efficient and Flexible Large Multimodal Models. arXiv preprint arXiv:2602.09080, 2026. URL https://arxiv.org/abs/2602.09080.

[545] Yudong W. Xu, Wenhao Li, Scott Sanner, and Elias B. Khalil. Self-Supervised Transformers as Iterative Solution Improvers for Constraint Satisfaction. In *International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2502.15794.

[546] Yunqiu Xu, Ling Chen, Meng Fang, Yang Wang, and Chengqi Zhang. Deep Reinforcement Learning with Transformers for Text Adventure Games. IEEE Conference on Games (CoG), 2020.

[547] Fuzhao Xue, Ziji Shi, Futao Wei, Yuxuan Lou, Yong Liu, and Yang You. Go Wider Instead of Deeper. In *AAAI Conference on Artificial Intelligence*, 2021. URL https://arxiv.org/abs/2107.11817.

[548] Yihao Xue, Allan Zhang, Jianhao Huang, Amit Sahai, and Baharan Mirzasoleiman. Beyond What Seems Necessary: Hidden Gains from Scaling Training-Time Reasoning Length under Outcome Supervision. arXiv preprint arXiv:2602.00927, 2026. URL https://arxiv.org/abs/2602.00927.

[549] Yuanyi Yan, Xinzhe Rao, Canyu Shen, Yang Chen, Yunlu Chen, Meng Tang, Teng Long, and Vincent Tao Hu. Training-Free Hidden-State Refinement for Flow-Matching Image Generators. arXiv preprint arXiv:2608.29160, 2026. URL https://arxiv.org/abs/2608.29160.

[550] Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. arXiv preprint arXiv:2606.18023, 2026. URL https://arxiv.org/abs/2606.18023.

[551] Jiaxi Yang, Binyuan Hui, Min Yang, Bailin Wang, Bowen Li, Binhua Li, Fei Huang, and Yongbin Li. Iterative Forward Tuning Boosts In-Context Learning in Language Models. In *Annual Meeting of the Association for Computational Linguistics*, 2023. URL https://arxiv.org/abs/2305.13016.

[552] Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped Transformers are Better at Learning Learning Algorithms. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2311.12424.

[553] Ruotong Yang, Hongdong Zhu, Qi Gao, Yin Ma, Hai Wei, and Kai Wen. Q-DEQ: Discrete Solving and Quantization for Deep Equilibrium Models in Time Series Forecasting under Edge Deployment Coding Constraints. arXiv preprint arXiv:2609.24042, 2026. URL https://arxiv.org/abs/2609.24042.

[554] Shuo Yang, Le Hou, Xiaodan Song, Qiang Liu, and Denny Zhou. Speeding up Deep Model Training by Sharing Weights and Then Unsharing. arXiv preprint arXiv:2110.03848, 2021. URL https://arxiv.org/abs/2110.03848.

[555] Wanqi Yang and Shiwei Liu. FlashLoop: Fast and Memory-Efficient Looped Transformers via Lazy Updates. arXiv preprint arXiv:2609.29812, 2026. URL https://arxiv.org/abs/2609.29812.

[556] Xiao-Wen Yang, Ziyu Han, Xi-Hua Zhang, Wen-Da Wei, Jie-Jing Shao, Lan-Zhe Guo, and Yu-Feng Li. Stabilizing Recurrent Dynamics for Test-Time Scalable Latent Reasoning in Looped Language Models. arXiv preprint arXiv:2605.26733, 2026. URL https://arxiv.org/abs/2605.26733.

[557] Zhun Yang, Adam Ishay, and Joohyung Lee. Learning to Solve Constraint Satisfaction Problems with Recurrent Transformer. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2307.04895.

[558] Lin Yao. Don't Commit Alone: Joint Token Commitment in Diffusion Language Models. arXiv preprint arXiv:2607.04469, 2026. URL https://arxiv.org/abs/2607.04469.

[559] Yuekun Yao, Yupei Du, Dawei Zhu, Michael Hahn, and Alexander Koller. Language models can learn implicit multi-hop reasoning, but only if they have lots of training data. In *Conference on Empirical Methods in Natural Language Processing*, 2025. URL https://arxiv.org/abs/2505.17923.

[560] Wencheng Ye, Anning Hu, Xiangdong Zhang, Tianyi Wang, Yikang Li, Hengyu Jin, Bing Li, and Junchi Yan. Write Back the Δ: Revisiting the Same Tokens with Fresh Representations. arXiv preprint arXiv:2609.32457, 2026. URL https://arxiv.org/abs/2609.32457.

[561] Yichen You, Tianyu Fu, Aosong Feng, Xingtai Lv, Xuefei Ning, Ning Ding, and Yu Wang. Improving Test-Time Scaling with Adaptive Looped Transformers. arXiv preprint arXiv:2609.35748, 2026. URL https://arxiv.org/abs/2609.35748.

[562] Byeongho Yu, Junhyuk So, and Eunhyeok Park. LoopCD: Loop-wise Contrastive Decoding for Improving Reasoning in Looped Language Models. arXiv preprint arXiv:2609.24196, 2026. URL https://arxiv.org/abs/2609.24196.

[563] Chengting Yu, Xiaobo Shu, Yadao Wang, Yizhen Zhang, Haoyi Wu, Jiaang Li, Rujiao Long, Ziheng Chen, Yuchi Xu, Wenbo Su, and Bo Zheng. MeSH: Memory-as-State-Highways for Recursive Transformers. arXiv preprint arXiv:2510.07739, 2025. URL https://arxiv.org/abs/2510.07739.

[564] Chengting Yu, Xiaobo Shu, Yadao Wang, Yizhen Zhang, Haoyi Wu, You Wu, Rujiao Long, Ziheng Chen, Yuchi Xu, Wenbo Su, and Bo Zheng. SpiralFormer: Looped Transformers Can Learn Hierarchical Dependencies via Multi-Resolution Recursion. arXiv preprint arXiv:2602.11698, 2026. URL https://arxiv.org/abs/2602.11698.

[565] Mingqian Yu, Wenpeng Zhang, Shaobo Cui, and Peilin Zhao. T-LoopFormer: Token-Level Elastic-Depth Looped Transformers for Latent Reasoning with Dynamic Routing. arXiv preprint arXiv:2609.15160, 2026. URL https://arxiv.org/abs/2609.15160.

[566] Nanxi Yu, Kang Li, Ye Du, Xiaowei Hu, Weihua Yang, and Shujun Wang. Learning When to Recur: Token-Adaptive Recursion for Imbalanced Ophthalmic Domain Incremental Learning. arXiv preprint arXiv:2609.32785, 2026. URL https://arxiv.org/abs/2609.32785.

[567] Qifan Yu, Zhenyu He, Sijie Li, Xun Zhou, Jun Zhang, Jingjing Xu, and Di He. Enhancing Auto-regressive Chain-of-Thought through Loop-Aligned Reasoning. In *Conference of the European Chapter of the Association for Computational Linguistics*, 2025. URL https://arxiv.org/abs/2502.08482.

[568] Yi Yu and Karl Borjesson. Chemical transformer compression for accelerating both training and inference of molecular modeling. *Machine Learning: Science and Technology*, 2022. URL https://arxiv.org/abs/2205.07582.

[569] Zhenxuan Yu, Takeshi Kojima, Yutaka Matsuo, and Yusuke Iwasawa. CHASE: Cache-Hole-Adapted Skip Exit for Looped State-Space Language Models. arXiv preprint arXiv:2607.10110, 2026. URL https://arxiv.org/abs/2607.10110.

[570] Honggang Yuan, Zixin Johnson Wang, Yuning Jiang, Xin Liu, Yuanming Shi, and Ting Wang. SAI: Latency-Aware Satellite Edge LAM Inference with Looped Transformer. International Conference on Communications, 2025.

[571] Jeongin Yun, Jaeri Lee, Jongjin Kim, Minjun Kim, Jinho Song, and U Kang. SharVeT: Similarity-aware Parameter Sharing with Vector-based Tuning for Efficient LLM Compression. Annual Meeting of the Association for Computational Linguistics (ACL), 2026.

[572] Pourya Zamanvaziri, Amirhossein Sadr, Aida Pakniyat, and Dara Rahmati. ITS-Mina: A Harris Hawks Optimization-Based All-MLP Framework with Iterative Refinement and External Attention for Multivariate Time Series Forecasting. arXiv preprint arXiv:2604.27981, 2026. URL https://arxiv.org/abs/2604.27981.

[573] Abbas Zeitoun, Lucas Torroba-Hennigen, and Yoon Kim. Hyperloop Transformers. arXiv preprint arXiv:2604.21254, 2026. URL https://arxiv.org/abs/2604.21254.

[574] Boyi Zeng, He Li, Shixiang Song, Yixuan Wang, Zitong Wang, Ziwei He, Xinbing Wang, and Zhouhan Lin. PonderLM-2: Pretraining LLM with Latent Thoughts in Continuous Space. arXiv preprint arXiv:2509.23184, 2025. URL https://arxiv.org/abs/2509.23184.

[575] Boyi Zeng, Shixiang Song, Siyuan Huang, Yixuan Wang, He Li, Ziwei He, Xinbing Wang, Zhiyu Li, and Zhouhan Lin. PonderLM: Pretraining Language Models to Ponder in Continuous Space. arXiv preprint arXiv:2505.20674, 2025. URL https://arxiv.org/abs/2505.20674.

[576] Boyi Zeng, Yiqin Hao, He Li, Shixiang Song, Feichen Song, Zitong Wang, Siyuan Huang, Yi Xu, ZiWei He, Xinbing Wang, and Zhouhan Lin. Pretraining with Token-Level Adaptive Latent Chain-of-Thought. arXiv preprint arXiv:2602.08220, 2026. URL https://arxiv.org/abs/2602.08220.

[577] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive Language Models. arXiv preprint arXiv:2512.24601, 2025. URL https://arxiv.org/abs/2512.24601.

[578] Boyang Zhang and Lianlei Shan. MPCoT: Reward-Guided Multi-Path Latent Reasoning for Test-Time Scalable Vision-Language-Action. arXiv preprint arXiv:2606.06245, 2026. URL https://arxiv.org/abs/2606.06245.

[579] Boyang Zhang, Daning Cheng, and Yunquan Zhang. Rethinking Parameter Sharing as Graph Coloring for Structured Compression. arXiv preprint arXiv:2511.06786, 2025. URL https://arxiv.org/abs/2511.06786.

[580] Chenyang Zhang and Yuan Cao. Transformers Efficiently Perform In-Context Logistic Regression via Normalized Gradient Descent. arXiv preprint arXiv:2605.06609, 2026. URL https://arxiv.org/abs/2605.06609.

[581] Ganlin Zhang, Weirong Chen, Daniel Cremers, and Xi Wang. BA-T: An Iterative Transformer for Two-View Bundle Adjustment. arXiv preprint arXiv:2606.03287, 2026. URL https://arxiv.org/abs/2606.03287.

[582] Haozhou Zhang. Chain-of-Thought and Compressed Looped Transformers: A Memory-Budget Separation. arXiv preprint arXiv:2605.30757, 2026. URL https://arxiv.org/abs/2605.30757.

[583] Jifan Zhang, Zhe Wu, Xinfeng Zhang, Guoli Song, Yaowei Wang, and Jie Chen. Recurrent Fine-Grained Self-Attention Network for Video Crowd Counting. IEEE International Conference on Acoustics Speech and Signal Processing, 2023.

[584] Jinnian Zhang, Houwen Peng, Kan Wu, Mengchen Liu, Bin Xiao, Jianlong Fu, and Lu Yuan. MiniViT: Compressing Vision Transformers with Weight Multiplexing. In *Computer Vision and Pattern Recognition*, 2022. URL https://arxiv.org/abs/2204.07154.

[585] Jinxi Zhang and Chunling Li. Learning Source Representation via Iterative Feature Refinement for Data-Informed Neural Machine Translation. Data Intelligence, 2026.

[586] Jinyang Zhang, Hongxin Ding, Yue Fang, Weibin Liao, Muyang Ye, Junfeng Zhao, and Yasha Wang. The Tell-Tale Norm: ℓ_2 Magnitude as a Signal for Reasoning Dynamics in Large Language Models. arXiv preprint arXiv:2606.06188, 2026. URL https://arxiv.org/abs/2606.06188.

[587] Lida Zhang, Abdolghani Ebrahimi, and Diego Klabjan. Layer Flexible Adaptive Computational Time. In *International Joint Conference on Neural Networks (IJCNN)*, 2021. URL https://arxiv.org/abs/1812.02335.

[588] Tong Zhang, Junhao Hu, Yun Peng, and Tao Xie. When Does Recurrence Become an Algorithm? Convergence Selection in Weight-Tied Looped Transformers. arXiv preprint arXiv:2607.20594, 2026. URL https://arxiv.org/abs/2607.20594.

[589] Yi Zhang, Arturs Backurs, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, and Tal Wagner. Unveiling Transformers with LEGO: a synthetic reasoning task. arXiv preprint arXiv:2206.04301, 2022. URL https://arxiv.org/abs/2206.04301.

[590] Yimeng Zhang, Harold Rockwell, Sicheng Dai, Ge Huang, Stephen Tsou, Yuanyuan Wei, and Tai Sing Lee. Recurrent networks improve neural response prediction and provide insights into underlying cortical circuits. arXiv preprint arXiv:2110.00825, 2021. URL https://arxiv.org/abs/2110.00825.

[591] Yiming Zhang, Qiangyu Yan, Borui Jiang, and Kai Han. Multimodal Latent Reasoning via Hierarchical Visual Cues Injection. arXiv preprint arXiv:2602.05359, 2026. URL https://arxiv.org/abs/2602.05359.

[592] Yun Zhang. UT-ATD: Universal Transformer for Anomalous Trajectory Detection by Embedding Trajectory Information. Proceedings of the International DMS Conference on Visualization and Visual Languages (DMSVIVA), 2021.

[593] Zhengxi Zhang and Paul Swoboda. Compute Time Scaling with Recursive Models for Combinatorial Optimization. arXiv preprint arXiv:2609.34585, 2026. URL https://arxiv.org/abs/2609.34585.

[594] Zhilong Zhang, Xinhui Zhang, Gongyu Jin, Sihua Wang, Danpu Liu, and Changchuan Yin. Recursive Vision Transformer with Dynamic Depth and Width Adjustment for Resource-Efficient Image Semantic Communication. arXiv preprint arXiv:2606.00114, 2026. URL https://arxiv.org/abs/2606.00114.

[595] Yifei Zhao and Ying Tang. Energy-guided Recursive Model. arXiv preprint arXiv:2607.10128, 2026. URL https://arxiv.org/abs/2607.10128.

[596] Yingzhu Zhao, Chongjia Ni, Cheung-Chi Leung, Shafiq Joty, Eng Siong Chng, and Bin Ma. Universal Speech Transformer. Interspeech (USB), 2020.

[597] Zixuan Zhao, Samuel Wheeler, Neil Getty, Xiaotian Duan, Rick Stevens, and Fangfang Xia. Structured Sparse Memory for Recurrent Reasoning. arXiv preprint arXiv:2609.33270, 2026. URL https://arxiv.org/abs/2609.33270.

[598] Zaixiang Zheng, Yi Zhou, and Hao Zhou. Deep Equilibrium Non-Autoregressive Sequence Learning. Annual Meeting of the Association for Computational Linguistics, 2023.

[599] Cai Zhou, Chenxiao Yang, Yi Hu, Chenyu Wang, Chubin Zhang, Muhan Zhang, Lester Mackey, Tommi Jaakkola, Stephen Bates, and Dinghuai Zhang. Coevolutionary Continuous Discrete Diffusion: Make Your Diffusion Language Model a Latent Reasoner. arXiv preprint arXiv:2510.03206, 2025. URL https://arxiv.org/abs/2510.03206.

[600] Qingyang Zhou, Rongpeng Li, Zhifeng Zhao, Chenghui Peng, and Honggang Zhang. Semantic Communication with Adaptive Universal Transformer. *IEEE Wireless Communications Letters*, 2021. URL https://arxiv.org/abs/2108.09119.

[601] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. BERT Loses Patience: Fast and Robust Inference with Early Exit. In *Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2006.04152.

[602] Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-Connections. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2409.19606.

[603] Rui-Jie Zhu, Tianhao Peng, Tianhao Cheng, Xingwei Qu, Jinfa Huang, Dawei Zhu, Hao Wang, Kaiwen Xue, Xuanliang Zhang, Yong Shan, Tianle Cai, Taylor Kergan, Assel Kembay, Andrew Smith, Chenghua Lin, Binh Nguyen, Yuqi Pan, Yuhong Chou, Zefan Cai, Zhenhe Wu, Yongchi Zhao, Tianyu Liu, Jian Yang, Wangchunshu Zhou, Chujie Zheng, Chongxuan Li, Yuyin Zhou, Zhoujun Li, Zhaoxiang Zhang, Jiaheng Liu, Ge Zhang, Wenhao Huang, and Jason Eshraghian. A Survey on Latent Reasoning. arXiv preprint arXiv:2507.06203, 2025. URL https://arxiv.org/abs/2507.06203.

[604] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models. arXiv preprint arXiv:2510.25741, 2025. URL https://arxiv.org/abs/2510.25741.

[605] Ruike Zhu, Hanwen Zhang, Kevin Li, Tianyu Shi, Yiqun Duan, Chi Wang, Tianyi Zhou, Arindam Banerjee, and Zengyi Qin. Beyond Parameters: Exploring Virtual Logic Depth for Scaling Laws. arXiv preprint arXiv:2506.18233, 2025. URL https://arxiv.org/abs/2506.18233.

[606] Tongtian Zhu. Mobius Learning: Cyclic Depth Folding in Transformers. arXiv preprint arXiv:2607.17843, 2026. URL https://arxiv.org/abs/2607.17843.

[607] Zhirui Zhu, Li Feng, and Qinglin Zhao. Depth-selective LoRA: Augmenting mixture-of-recursions with sparse low-rank adaptation. Expert Systems with Applications, 2026.

[608] Zirui Zhu, Hailun Xu, Xuanlei Zhao, Yong Liu, Yingxuan Ren, Kanchan Sarkar, Kun Xu, and Yang You. Loop Dropout: Regularizing Shared Updates in Looped Language Models. arXiv preprint arXiv:2609.34218, 2026. URL https://arxiv.org/abs/2609.34218.

[609] Xinlin Zhuang, Siyuan Wang, Imran Razzak, and Weiyang Liu. What Makes Recurrence Effective in Looped Language Models? arXiv preprint arXiv:2609.36636, 2026. URL https://arxiv.org/abs/2609.36636.

[610] Magauiya Zhussip, Dmitriy Shopkhoev, Ammar Ali, and Stamatios Lefkimmiatis. Share Your Attention: Transformer Weight Sharing via Matrix-based Dictionary Learning. In *AAAI Conference on Artificial Intelligence*, 2025. URL https://arxiv.org/abs/2508.04581.

[611] Artur Zolkowski, Wen Xing, David Lindner, Florian Tramèr, and Erik Jenner. Can Reasoning Models Obfuscate Reasoning? Stress-Testing Chain-of-Thought Monitorability. arXiv preprint arXiv:2510.19851, 2025. URL https://arxiv.org/abs/2510.19851.

A Corpus Growth Data

This appendix gives the monthly counts of core papers by area that Figure 2 plots.

Table 23: Core papers per month by area (data behind Figure 2).

Month	Architecture & adaptive depth	Training, scaling & systems	Theory & mechanisms	Capabilities & recursive reasoning	Beyond language	Safety & other	Total
before 2023	41	3	10	2	36	4	96
2023-01	0	0	1	0	0	1	2
2023-02	0	0	0	0	0	0	0
2023-03	0	0	0	0	2	0	2
2023-04	0	0	0	0	0	0	0
2023-05	0	1	0	0	1	0	2
2023-06	1	0	0	0	3	0	4
2023-07	1	0	1	0	2	0	4
2023-08	0	0	0	0	1	0	1
2023-09	2	0	0	0	1	0	3
2023-10	5	2	0	0	0	0	7
2023-11	1	0	1	0	0	0	2
2023-12	0	0	0	0	2	0	2
2024-01	2	0	0	0	1	0	3
2024-02	3	0	2	0	0	0	5
2024-03	0	0	0	0	0	0	0
2024-04	2	0	0	0	0	0	2
2024-05	1	0	2	0	0	0	3
2024-06	0	1	0	0	1	0	2
2024-07	0	1	0	0	0	0	1
2024-08	0	0	0	0	0	0	0
2024-09	2	0	1	0	3	0	6
2024-10	2	0	10	0	0	0	12
2024-11	1	0	0	0	1	0	2
2024-12	0	0	0	0	0	0	0
2025-01	1	3	1	0	0	0	5
2025-02	8	0	2	1	0	0	11
2025-03	0	0	1	0	0	0	1
2025-04	0	0	0	0	0	0	0
2025-05	2	2	2	1	1	0	8
2025-06	2	2	1	1	0	0	6
2025-07	2	1	1	0	1	0	5
2025-08	2	1	1	1	4	0	9
2025-09	1	1	2	1	0	0	5
2025-10	4	7	2	3	5	0	21
2025-11	3	1	0	3	0	0	7
2025-12	2	1	1	2	2	0	8
2026-01	1	3	3	2	1	0	10
2026-02	3	2	3	1	6	0	15
2026-03	5	1	2	6	4	0	18
2026-04	7	5	5	3	6	1	27
2026-05	8	7	9	8	15	1	48
2026-06	5	9	3	4	7	0	28
2026-07	9	6	3	4	8	0	30
2026-08	4	6	7	7	12	0	36
2026-09	12	17	11	12	16	0	68
2026-10	1	6	1	1	5	2	16

B Reporting Checklist for Looped-Model Papers

We recommend that empirical papers on looped models report the following. Items 1–4 make results comparable across papers, and items 5–8 make them interpretable.

1. **Architecture as an instance of the template** (Section 3): k_P, k_R and k_C, the injection operator and the initial state. Also report the sharing map (strict, relaxed, MoE) and the loop-count policy at training and at test time.
2. **Parameters**: unique, active per iteration (for MoE), and embedding parameters, reported separately.
3. **Compute**: training tokens and training FLOPs, inference FLOPs per token at every evaluated loop count, and the FLOP-counting convention (including attention terms).
4. **Memory**: KV-cache bytes per token at every loop count and whether caches are shared across iterations. Also report training activation memory and the gradient method (full or truncated backpropagation, one-step gradients).

5. **Baselines and regime**: which of parameters, effective depth, training FLOPs, inference FLOPs and data are matched for every baseline (Section 3.5). Also report whether the authors trained the baselines on the same data.
6. **Depth curves**: quality as a function of loop count below, at and beyond the training horizon, in addition to quality at the best loop count.
7. **Latency**: wall-clock latency and throughput on stated hardware and batch sizes, in addition to any FLOP or average-depth proxy. For adaptive depth, also report the overhead of the exit policy or search separately, and include as a baseline a fixed-loop model retrained at the same realized compute [286, 450].
8. **Variance**: number of seeds and the spread of results, especially for small-data puzzle benchmarks where protocol choices (augmentation, voting, pass@k) dominate the results.

C Claims Ledger Excerpt

For each priority-1 core paper, Table 24 lists up to two claims that compare a looped model with a non-looped one. For each claim, the table gives the comparison regime and the largest scale at which the claim is shown. We release the full ledger with the survey, and it includes the verbatim quotation and location behind every claim.

Table 24: Claims-ledger excerpt: up to two architecture-comparison claims per priority-1 core paper. Regimes: P matched parameters, D matched effective depth, IF matched inference FLOPs, TF matched training FLOPs, Dat matched data, U unmatched. Claim wording is paraphrased from the extraction records; each is backed by a verified quotation in the released ledger.

Paper	Claim	Regime	Scale
ACT [187]	On parity, the network without ACT fails (about 40% error) while ACT with low time penalty gets below 5% error.	P, Dat	128-unit RNN
ACT [187]	For character-level language modeling ACT gives little gain; ponder is concentrated at spaces and sentence ends.	P, Dat	1500-cell LSTM
[158]	Adaptive looping (Nmax=3) reduces math BPB by 22% vs the base model and beats a 36-layer iso-FLOP model on math BPB by 6.4% with one-third of the layers.	P, D	~200M
[158]	More loops add little beyond Nmax=3 (Loop-7 improves 1.7% over Loop-3) and commonsense slightly declines with more loops; the 36-layer iso-FLOP model remains better on commonsense.	P	~200M
ALBERT [285]	At matched depth/width (fewer unique parameters), sharing all layers hurts downstream average by 1.5 (E=128) to 2.5 (E=768) points vs not-shared.	D, Dat	ALBERT-base (12M-108M)
ALBERT [285]	With roughly matched training wall-clock time, ALBERT-xxlarge beats BERT-large by 1.5 average points despite fewer parameters (but it is ~3x slower per step).	TF, U	ALBERT-xxlarge 233M vs BERT-large 334M
Attractor Models [152]	On Sudoku-Extreme/Maze-Hard, Attractor Models improve with scale while TRM collapses at 27M.	P, U	27M
Attractor Models [152]	At matched parameters, data, optimizer and schedule, Attractor Models beat both a standard Transformer and the looped Parcae on perplexity and downstream accuracy at 140M-770M.	P, Dat	770M
CoTFormer [358]	At equal parameters and equal number of repeats, CoTFormer beats Block Universal Transformer in perplexity, and the gap widens with more repeats (at the cost of extra attention compute).	P, D, Dat	24-layer base x 5 repeats
CoTFormer [358]	Router-based token allocation gives a much better compute-accuracy trade-off than uniformly truncating repeats at inference; but adaptive training lags a fixed-5-repeat model at equal inference compute (23.83 vs 23.19).	IF	24 layers, max 5 repeats

Paper	Claim	Regime	Scale
DEQ [30]	At similar parameter counts DEQs match or slightly beat Transformer-XL and TrellisNet on WikiText-103 while using 80-88% less memory.	P, Dat	172M total / 43M non-embedding
DEQ [30]	DEQs are slower than the corresponding explicit deep networks (runtime ratios above 1x in Table 4), though computation can be traded via tolerance or Broyden iteration limits.	P	medium
DT-Recall (Deep Thinking with recall) [38]	Trained on 9x9 mazes, DT-Recall with progressive loss solves 74% of 201x201 mazes; feed-forward and prior DT systems completely fail on 59x59.	D, Dat	conv net
DT-Recall (Deep Thinking with recall) [38]	Recall is what prevents overthinking; recalled models appear to converge to a fixed point rather than becoming unstable as iterations grow.	Dat	conv net
ETD [267]	The best block position depends on task category: 7-4*2-5 is best for math, while an earlier block (3-4*2-9) is best for reading comprehension.	P, Dat, IF, TF	1B
ETD [267]	Which layers to loop matters: at equal FLOPs, looping the 4 knee-selected layers beats looping all layers or the middle 12 layers.	P, IF, Dat	1B
HRM [499]	On Sudoku-Extreme-Full, increasing width does not help while depth helps; standard (and recurrent) Transformers saturate with depth while HRM keeps improving.	D	872M (largest width-scaled baseline)
HRM [499]	Against a same-size Transformer trained with the identical setup ('Direct pred'), swapping in HRM's hierarchy plus ACT more than doubles ARC-AGI-1 performance.	P, Dat	27M
HRM-Text [500]	Looped Transformers and RINS beat same-size Transformers, but their advantage over a larger FLOP-matched Transformer is inconsistent.	P, TF	1B looped vs 3B dense
HRM-Text [500]	TRM (shared H/L weights) trained at 1B with reduced recursions is unstable; at 0.6B with the same H2L3 recursion it roughly matches HRM 0.6B but needs about 2x the training FLOPs.	P, TF	1B
Huginn [175]	Against a non-recurrent twin trained on the same 180B tokens, the recurrent model is much better on harder tasks (ARC-C, GSM8K), with similar recall tasks; parameter/FLOP matching of the twin is not stated.	Dat	3.5B
Huginn [175]	Test-time recurrence improves reasoning benchmarks up to compute equivalent to a 50B-parameter model; comparisons are to public models trained on other data.	U	3.5B
Iso-Depth phi [433]	At matched training compute and matched effective depth, replacing unique blocks with shared recurrences raises validation loss; one recurrence is worth r^0.46 in unique-parameter terms.	TF, D	2.15e19 FLOPs grid; 47B-token extrapolation
Iso-Depth phi [433]	The compute-optimal loss gap of looped vs non-looped grows with r and persists at ~20x the grid compute (0.061 nats at r=4, 47B tokens).	D, Dat	s=34 (d_model 2,176), 47B tokens
Loop Scaling Laws [84]	At trillion-token scale and matched training compute, a 1.3B-total looped MoE with law-derived recurrence matches a 2.9B-total non-looped MoE on reasoning but trails on the 14-benchmark overall score, and uses more inference compute.	TF, IF, Dat	1.3B total / 0.3B active, ~3T tokens
Loop Scaling Laws [84]	Recurrence adds effective parameters only up to a finite asymptote; MoE sparsity raises that asymptote and delays saturation (routing reaches different experts across passes).	P, Dat	1.0B active, 500B tokens
LoopCoder-v2 [550]	With identical parameters, 18T-token pretraining data and SFT, the two-loop PLT improves broadly over the non-looped baseline on code generation, code reasoning, agentic SWE and tool use.	P, Dat	7B, 18T tokens
LoopCoder-v2 [550]	Loop count is strongly non-monotonic: three or more loops regress, often below the non-looped baseline (R=3 drops to 27.6% on SWE-bench Verified).	P, Dat	7B, 18T tokens
[396]	Under matched SFT, looped models do better on compositional (multi-call, nested) tool calling, while gains on isolated single API calls are smaller and model-dependent; the Ouro comparison is against non-looped models with different pretraining.	U	2.6B

Paper	Claim	Regime	Scale
[396]	Retrofitted looped models improve several compositional categories over their parents but remain much weaker than natively pretrained looped models on deeply nested workflows.	U	1B
Looped TF (ICL) [552]	At matched effective depth (b=12 loops of 1 layer vs 12 distinct layers), the looped model does not match the standard transformer; it needs more loops than layers (about 20 iterations of 1 layer to match 8 layers).	D	9.48M
Looped TF (ICL) [552]	Input injection is necessary for stable extrapolation in loop count: without it, the weight-tied model diverges beyond the trained number of loops; with it, the model reaches a fixed point that persists past training iterations.	P	0.79M
Looped TF length-gen (n-RASP-L) [146]	Fixed-depth weight tying (NTP-Loop) and pause tokens help somewhat but are clearly worse than adaptive depth; pause tokens add parallel not sequential compute.	P, D	small
Looped TF length-gen (n-RASP-L) [146]	Looped transformers with step-dependent final-answer supervision and adaptive loop count length-generalize far beyond training length (parity: trained to 20 digits, near-perfect at 40+).	D, Dat	small GPT-2 blocks (256-dim)
Loopie [169]	Layer-loop (per-layer repetition) beats model-loop (whole-stack repetition) on MoE backbones at the same parameters and FLOPs, but only after about 1.2T tokens; model-loop leads early.	P, IF, Dat	6B-A0.6B
Loopie [169]	Marginal benefit of extra loops decays fast under fixed pre-training compute; R=2 chosen; the N x layer vs N x loop figure is not compute-matched for N=2.	P, D	0.25B active
LoopLM (Closing the Loop) / Looped Qwen [343]	At matched inference compute (3 loops vs a 72-layer dense twin on the same data), the dense model is ahead or level on every benchmark; the looped model's advantage is parameter count.	IF, D, Dat	1.4B looped vs 3.9B dense
LoopLM (Closing the Loop) / Looped Qwen [343]	Think-at-Hard (2511.08577) could not be reproduced on this training mixture: repeated-step loss never fell below first-step loss.	U	not stated
LOTUS [147]	Trained loop depth matters strongly and extra test-time loops beyond the trained depth do not help.	P, Dat	3B
LOTUS [147]	At 3B, the looped padded model with parallel gold-CoT supervision brings latent reasoning to within about 1 point of explicit CoT on GSM8K (same backbone, same data), unlike prior latent methods whose gap widens with scale.	P, Dat	3B
LPT / H-LPT [360]	Memory sharing improves quality over a matched-depth looped baseline while reducing KV memory 2.0-4.7x with under 1% more FLOPs.	P, Dat	1B
LPT / H-LPT [360]	Hybrid H-LPT beats a same-size standard Transformer while using 76-79% less context memory at five recursions.	P, Dat	1B
MoEUT [108]	MoEUT slightly outperforms dense Transformers with the same parameter count on C4 perplexity, and the gap grows with scale (up to about 1B).	P, Dat	1040M
MoEUT [108]	MoEUT slightly outperforms dense Transformers with the same parameter count (same d_model and number of layers, same training steps), and the gap grows with scale, while using fewer MACs.	P, Dat	1040M
MoR [25]	At equal training FLOPs, MoR (expert-choice, Nr=2) beats the vanilla Transformer in average few-shot accuracy with about half the parameters, because routing lets it process more tokens in the same budget.	P, TF, Dat	360M base (167M MoR)
MoR [25]	In isoFLOP sweeps MoR beats fixed-depth recursive at every size but underperforms vanilla at 135M; it matches or exceeds vanilla only above 360M.	TF	1.7B
Nanbeige4.2-3B [368]	Two passes give the best trade-off, retaining about 75% of a standard Transformer's token efficiency; more passes add little, slow training and destabilize optimization.	P	
Nanbeige4.2-3B [368]	Sharing the KV cache across loop passes halves KV memory but consistently yields lower gains than the full non-sharing loop, so it was not used.	P	

Paper	Claim	Regime	Scale
[502]	Increasing Ouro loop depth from the default T=4 to T=8 lowers CoT monitorability on Logic, Science and Engineering Cue Answer tasks under stress-test prompts, while other tasks are stable or change differently.	P	Ouro-2.6B-Thinking
[502]	The monitorability decline is not explained by changes in verification pass rate across the 32 model x task x stress-test cells.	P	Ouro-2.6B-Thinking
Ouro [604]	On the Mano modular-arithmetic manipulation task, looped models beat iso-parameter non-looped models for all k and often beat the iso-FLOP (12-layer, iso-effective-depth) baseline.	P, D	12 effective layers, hidden 1024 (synthetic)
Ouro [604]	In the small-scale controlled sweep (20B tokens), a standard transformer whose layer count is multiplied by the recurrent step always beats the LoopLM; the gap grows with loops and shrinks with model size.	D, Dat	1.36B
Parcae [397]	At fixed parameter count and fixed training FLOPs, the compute-optimal looped frontier (fewer tokens, more recurrence) achieves strictly lower loss and higher Core than fixed-depth models that spend the FLOPs on more data.	P, TF	370M, up to 1.28e20 FLOPs
Parcae [397]	Compute-optimal training increases mean recurrence and data in tandem following power laws.	P, TF	370M
PLT [531]	Sequential looping roughly doubles decode latency and KV memory at L=2; CLP keeps accuracy and cuts latency 37% vs naive loop at bs=4.	P, Dat	680M activated / 13B total MoE
PLT [531]	Sharing the first loop's KV cache halves KV memory but costs 3.4 average points; gated SWA recovers it with 1.4% extra KV.	P, Dat	680M activated / 13B total MoE
PonderLM [575]	PonderPythia trained with the same architecture, tokenizer, data and hyperparameters as official Pythia outperforms it at every size; 2.8B surpasses Pythia-6.9B (but each token costs about s+1 forward passes).	P, Dat	2.8B, 300B tokens
PonderLM [575]	At equal extra steps, pondering beats a plain looped Transformer, pause tokens and hidden-state feedback; 3 pondering steps beat 6-loop baselines.	P, Dat	70M, 30B tokens
PonderNet [37]	On parity extrapolation (more ones than in training), PonderNet reaches near-perfect accuracy while ACT stays at chance, and it increases thinking steps to about 5 (roughly twice interpolation).	P	128-unit RNN
PonderNet [37]	With the same UT transformer architecture, PonderNet solves all 20 bAbI tasks with far fewer steps than UT with ACT (1658 vs 10161).	P	UT small
Relaxed Recursive Transformers [24]	With LoRA rank 512, the relaxed recursive Gemma matches the original full-size Gemma 2B (58.4 vs 58.6) with fewer parameters and 60B uptraining tokens.	D, U	Gemma 2B
Relaxed Recursive Transformers [24]	Continuous depth-wise batching with early exiting gives up to 2.66x simulated generation speedup over vanilla models, but this rests on oracle exits and hypothetical throughput.	D	Gemma 2B -> 1B
Retrofitted Recurrence [348]	A curriculum that slowly raises the mean recurrence keeps loss while reducing training cost; scheduling the max backprop depth instead is FLOP-efficient but worse per step.	TF, Dat	~1B, ~1e20 FLOPs runs
Retrofitted Recurrence [348]	At equal training FLOPs on the same data order, retrofitted recurrent models with fewer unique parameters beat the non-recurrent parent on GSM8K/MATH, and keep improving where the parent plateaus.	TF, Dat	~1B, ~50B tokens
RINS [12]	Stochastic RINS with linear adapters improves the LM even when recursion is disabled at test time (same training compute, parameters and inference cost).	P, TF, IF	600M
RINS [12]	At matched parameter count and training FLOPs, RINS outperforms all other sharing strategies and longer-context baselines, and its advantage grows with compute.	P, TF	600M (sweep), 1B
Saunshi et al. (k⊗L looping) [426]	At LM scale, looped models have worse perplexity than the iso-depth baseline but recover far more of the gap on reasoning-heavy downstream groups (open-book QA, math word problems, reasoning primitives) than on memorization (closed-book QA).	P, D, Dat	24-layer 1B (1.5B in appendix)

Paper	Claim	Regime	Scale
Saunshi et al. (k⊗L looping) [426]	On synthetic reasoning tasks, a k-layer block looped L times nearly matches a kL-layer non-looped model and far outperforms the k-layer model.	P, D	12-layer synthetic models
SMELT [507]	With per-token FLOPs, total parameters and KV cache matched, the looped MoE (SMELT) reaches lower loss than the unlooped MoE at every scale and sparsity level.	P, TF, IF, Dat	1.6B active / 54B total, ~215B tokens
SMELT [507]	Downstream gains exceed what the validation-loss gap predicts, and this excess grows with scale; at 1.6B the largest excess shifts to Symbolic Problem Solving.	P, IF, Dat	1.6B active / 54B total
Sparse Layers / Looped-MoE [288]	At matched training compute and effective depth, dense looping scales strictly worse than the dense baseline, while Looped-MoE scales better than the dense baseline.	TF, D	305M active, 1e18 FLOPs
Sparse Layers / Looped-MoE [288]	Looping the same architecture always reduces quality at matched compute: non-looped MoE beats Looped-MoE and Base beats Looped; all four share roughly the same slope and differ in offset.	TF, D	305M active
Takase-Kiyono sharing (SEQUENCE/ CYCLE/ CYCLE(REV)) [470]	Universal (one shared layer, widened to match parameters) beats Vanilla in MT but trains more than twice as slow; partial sharing (M=6, N=12) is faster than Universal and slightly better.	P, Dat	61M
Takase-Kiyono sharing (SEQUENCE/ CYCLE/ CYCLE(REV)) [470]	In language modeling, full one-layer sharing is worse than no sharing, while partial sharing (N=12, M=6) beats the 6-layer Vanilla at equal parameters; SEQUENCE is best.	P, Dat	121M (234M for SEQUENCE large)
[432]	Recurrent models show the same hierarchical behaviors and depth benefits as feed-forward networks despite reusing filters.	D	
[432]	Recurrent networks can closely emulate non-recurrent deep models, often with far fewer parameters.	D	
TRM [245]	Backpropagating through a full recursion process instead of HRM's 1-step gradient is the single largest gain (56.5% to 87.4% on Sudoku-Extreme).	Dat	5M
TRM [245]	Fewer layers with proportionally more recursions generalizes better: 2 layers beat 4 layers at roughly matched emulated depth; adding layers overfits.	Dat	10M vs 5M
URM [168]	At equal FLOPs (32x), a weight-shared UT (4 layers x 8 loops) beats a 32-layer vanilla Transformer on ARC-AGI-1 pass@1 (40.0 vs 23.75); vanilla depth/width scaling saturates or degrades.	IF, D	32 layers x 512 hidden vanilla
URM [168]	URM reaches 53.8% pass@1 on ARC-AGI-1 and 16.0% on ARC-AGI-2, above re-run TRM (40.0) and HRM (34.4).	Dat	4 layers, hidden 512
UT [120]	Dynamic halting outperforms fixed-depth UTs even when the fixed models use as many or more steps than the halting model's average (8 and 9 vs 8.2), suggesting halting acts as a regularizer.	D	UT base
UT [120]	UT base beats Transformer base by 0.9 BLEU on WMT14 En-De at approximately the same parameter count (so UT spends more compute per parameter).	P	Transformer-base parameter count (not stated numerically)
[609]	With extra test-time loops, looped models trained at the same effective depth and training FLOPs as a 20-layer NonLoop exceed its reasoning score (NonLoop 30.25; 5x4 reaches 31.88 at L(r)=30), using fewer unique layers but more inference compute.	TF, IF, D	Llama3.1-1B config
[609]	A non-recurrent coda mitigates knowledge collapse under under-unrolling; prelude-heavy allocation better supports reasoning extrapolation; near the horizon allocation matters little.	P, D	Llama3.1-1B and Qwen3-0.6B configs

D Core Papers by Topic

This appendix cites every core paper of the corpus, grouped by its main topic and ordered by date within each topic (546 papers). The topic labels come from screening and are coarse, so a paper that spans several topics appears under one of them. A further 11 core papers have no bibliographic metadata that we could resolve, and they are listed with titles and screening reasons in the released corpus.

Architecture, topology and sharing (89). [12, 13, 24, 43, 64, 75, 82, 87, 89, 94, 108, 121, 151, 152, 159, 160, 167, 169, 174, 175, 178, 181, 204, 205, 209, 211, 217, 220, 223, 229, 230, 232, 233, 236, 247, 252, 253, 256, 263, 299, 300, 302, 320, 321, 324, 330, 354, 355, 357, 358, 371, 374, 377, 381, 382, 386, 392, 393, 398, 404, 411, 420, 427, 429, 466, 470, 475, 476, 490, 497, 504, 505, 509, 516, 519, 535, 537, 547, 560, 563, 564, 571, 573, 575, 579, 606, 607, 609, 610]

Vision (60). [8–10, 15, 29, 31, 46, 56, 58, 62, 66, 68, 95, 97, 99, 130, 138, 166, 183, 190, 201, 203, 207, 258, 259, 265, 270, 278, 289, 296, 317, 319, 337, 340, 341, 346, 352, 367, 375, 390, 399, 444, 445, 447, 449, 456, 458, 473, 484, 510, 511, 520, 528, 539, 540, 566, 581, 583, 584, 594]

Recursive reasoning models (54). [3, 19, 21, 26, 27, 44, 53, 63, 69, 73, 92, 104, 114, 116, 118, 123, 125, 128, 135, 137, 140, 157, 165, 168, 173, 197, 199, 210, 218, 245, 250, 269, 272, 274, 281, 342, 345, 387, 400, 410, 419, 428, 436, 438, 443, 459, 461, 499, 500, 512, 545, 593, 595, 597]

Adaptive depth, halting and routing (41). [2, 7, 23, 25, 33, 34, 39, 57, 79, 88, 100, 105, 127, 142, 156, 158, 162, 194, 208, 214, 235, 255, 294, 328, 331, 361, 363, 370, 395, 424, 451, 454, 457, 460, 492, 518, 561, 565, 569, 587, 601]

Algorithmic and length generalization (31). [16, 38, 40, 42, 49, 67, 74, 103, 106, 107, 139, 146, 189, 228, 264, 266, 275, 315, 347, 407, 414, 471, 478, 495, 521, 524, 525, 557, 567, 588, 589]

Inference systems (30). [61, 81, 90, 98, 111, 149, 176, 196, 198, 212, 227, 240, 268, 301, 307, 310, 327, 360, 369, 434, 482, 493, 496, 517, 531, 536, 553, 555, 562, 570]

Science and other domains (29). [20, 35, 36, 126, 206, 213, 221, 244, 273, 292, 293, 304, 312, 332, 336, 378, 379, 383, 385, 403, 425, 487, 527, 529, 538, 568, 572, 592, 600]

Retrofitting and training-free looping (28). [41, 45, 47, 76, 86, 91, 141, 202, 226, 267, 277, 298, 313, 314, 339, 348, 389, 418, 423, 439, 452, 455, 494, 501, 526, 551, 558, 586]

Mechanisms and interpretability (26). [48, 72, 117, 195, 231, 242, 248, 251, 257, 260, 261, 280, 283, 284, 334, 338, 356, 380, 384, 388, 405, 413, 415, 513, 532, 590]

Theory: expressivity (22). [1, 4, 70, 182, 237, 246, 254, 316, 344, 351, 362, 366, 426, 441, 468, 469, 485, 489, 541–543, 582]

Speech and audio (19). [6, 11, 51, 59, 102, 153, 249, 276, 322, 335, 372, 416, 417, 435, 446, 479, 491, 503, 596]

Training, stability and gradients (17). [55, 161, 185, 219, 238, 279, 282, 308, 323, 343, 440, 448, 486, 498, 506, 554, 556]

Reinforcement learning, robotics and control (15). [14, 115, 179, 192, 234, 243, 333, 422, 442, 472, 488, 515, 522, 546, 578]

Lineage and foundations (14). [18, 30, 37, 101, 112, 113, 120, 131, 187, 285, 318, 364, 431, 432]

Diffusion and generative backbones (11). [22, 32, 65, 96, 177, 186, 239, 291, 353, 467, 549]

Scaling laws (10). [84, 93, 193, 288, 397, 433, 481, 507, 548, 605]

Multimodal models (10). [5, 262, 287, 290, 401, 402, 474, 508, 544, 591]

Theory: in-context learning and optimization (10). [71, 119, 170, 171, 180, 224, 309, 533, 552, 580]

Capabilities of looped language models (10). [50, 80, 145, 164, 172, 368, 396, 453, 550, 604]

Post-training, reinforcement learning and distillation (6). [129, 147, 412, 477, 530, 608]

Recommendation and retrieval (5). [122, 326, 437, 465, 480]

Other (5). [110, 144, 150, 184, 585]

Diffusion language models (2). [148, 598]

Safety and trust (1). [502]

Surveys and position papers (1). [365]